System-level mapping of automotive applications onto zonal E/E architectures: allocation, contention, LET, TSN and the life of the platform

Morgan Gauthier

Abstract. Vehicles are moving from a hundred function-specific electronic control units (ECUs) to zonal electrical/electronic (E/E) architectures: a central computer and a few zone controllers, linked by a Time-Sensitive Networking (TSN) backbone, run most of the software. Before such a platform is built, its designers must know how many cores each ECU needs, whether every task-chain still meets its end-to-end deadline once its tasks are spread over cores and over the network, and how much room is left for the software updates of the coming years. Task allocation, fixed priorities, shared-memory contention, Logical Execution Time (LET) communication between cores and network delays are coupled, and, to our knowledge, no existing method treats them together. This paper states the problem formally and gives a system-level method that does. Task-chains are cut into sub-chains that respect the bindings of sensors and actuators to ECUs, allocated to heterogeneous cores by bin packing or by a new communication-aware rule, consolidated onto fewer cores, and prioritized by an optimal assignment that respects precedence. Response times include shared-memory contention. The maximum reaction time of every task-chain is bounded, under implicit communication or strict LET, with Network Calculus bounds for credit-based, time-aware and asynchronous traffic shaping and for frame replication. Relocation and deadline-driven moves, with an exact incremental re-analysis, repair unschedulable cores and late task-chains. An exact constraint-programming formulation with lazy schedulability cuts measures the optimality gap of the allocation, and a lifecycle analysis adds software updates to the hardware fixed at the start of production. On synthetic applications that follow published automotive statistics, no simulated delay exceeds its bound, for 3377 network flows and for 9000 task-chains simulated as a whole system. The heuristic is 2.3 to 2.9 % above the optimal allocation on average and proven optimal in 81 % of 140 instances of up to 300 task-chains; on the default platform it is as close as simulated annealing run for seconds, in milliseconds, and closer than a genetic algorithm, which in turn recovers most of the gap of the heuristic on harder platforms. Communication awareness removes 64 % of the inter-ECU communications for 5 more cores, strict LET shrinks the network bounds by 15–19 % but costs 30 points of deadlines, and each core bought at the start of production buys about half a software update. The same method maps domain, hybrid and distributed architectures, within 1.1–1.2 and 4.4–4.7 % of the optimum on the hybrid and domain platforms, and a whole-vehicle application of 2000 task-chains in about a second. **This project** originated from an idea conceived during the author's PhD that could not be pursued at the time. It is an independent continuation built with the help of AI tools, which assisted in developing the method, code, and paper. The code and the scripts that reproduce every experiment are at `https://github.com/ifupauline/mapping`.

Keywords. automotive embedded systems; software-defined vehicle; zonal E/E architecture; real-time systems; task allocation; task-chains; cause-effect chains; logical execution time; fixed-priority scheduling; time-sensitive networking; network calculus; asynchronous traffic shaping; frame replication; memory contention; design-space exploration

1 Introduction

For three decades, a new vehicle function usually meant a new electronic control unit (ECU). Premium cars already carried 70 to 100 ECUs in 2009 [1], with a wiring harness, a supply chain and an integration effort to match [2, 3]. The industry is now consolidating this software onto a few powerful computers. A *central computer* runs most functions, and a handful of *zone controllers* connect the sensors and actuators of their physical area of the car. An Ethernet backbone with Time-Sensitive Networking (TSN) links them [4, 5]. This *zonal* electrical/electronic (E/E) architecture shortens the harness and decouples software from hardware [2, 6].

It also turns a question that each ECU supplier used to answer alone into a system-wide one: the design question in the box.

Design question

How many processor cores does the software need, and on which ECU? Does every function still meet its timing requirement once its tasks are spread over cores and over the network? And how much room is left for the functions that software updates will add during the life of the vehicle?

Why the question is hard. Vehicle functions are *task-chains* (also called cause-effect chains): a sensor is sampled, data flows through a sequence of tasks, and an actuator is commanded [7, 8]. Their timing requirement is end to end, typically a bound on the *maximum reaction time*: the longest time from an external event to the actuation that accounts for it [7, 9, 10]. On a zonal platform this reaction time depends on four decisions, and they interact.

(i) The *allocation* of tasks to cores. It is constrained by the wiring (a sensor driver runs in the zone of its sensor), by memory, by safety (redundant functions

on different ECUs, safety tasks on lock-step cores) and by cores of different types and speeds.

(ii) The *priorities* on each core, which set the response times of the tasks.

(iii) The *communication between cores*. With the Logical Execution Time (LET) paradigm [11–13], data crosses a core boundary only at fixed instants. This makes timing predictable, but every crossing adds sampling delays and every core pays for the copy task.

(iv) The *network*. Every task-chain that crosses ECUs adds a stream of frames, which shares output ports with the other streams. Its delay depends on the traffic shaping of the switches [5, 14].

Packing tasks onto fewer cores saves hardware but lengthens response times. Spreading a task-chain over ECUs frees cores but adds LET sampling delays and network streams, and these streams delay other task-chains on the same ports. The platform also has to survive its own life: the hardware is frozen at the start of production (SOP), while software updates keep adding functions for years.

What exists. Each piece is well studied on its own. Allocation and priority assignment for distributed real-time systems goes back to Tindell et al. [15] (see [16–18]). The end-to-end analysis of task-chains has matured [9, 19–21]. LET deployment on multicores has been optimized [13, 22, 23]. TSN has well-established Network Calculus bounds [24–28], multicore response-time analysis accounts for shared-memory interference [29], and compositional system-level analysis chains the analyses of processors and buses for a given mapping [30]. Studies of zonal architectures, however, focus on the harness and the network [31] or schedule tasks without a schedulability analysis [32], and syntheses of E/E architectures survey the mapping of software without a system-level timing analysis [3]. What is missing is the system level. To our knowledge, and as Section 3 details, no published method decides the allocation of an automotive application while it analyses, together and with the same granularity, the fixed-priority scheduling of every core, the shared-memory contention between its cores, LET-based communication and the delays of shaped TSN streams, under the spatial constraints of a vehicle, and sizes the cores from that analysis.

Contributions. This paper gives such a system-level method, in enough detail to re-implement it from the text, with the code as a reference.

C1. A formal model and problem statement (Sections 4 and 5). The model covers zonal, domain, hybrid and distributed E/E architectures (Section 4.5), heterogeneous cores, a utilization cap, communication between cores at LET instants (implicit or strict LET), multi-rate task-chains, shared-memory contention, and a TSN backbone with four TSN mechanisms: credit-based shaping, time-aware shaping, asynchronous traffic shaping and frame replication, on a star, a tree, a ring or any switched topology. It also covers the spatial constraints of a vehicle: allowed ECUs, memory, separation of redundant task-chains, co-location and allowed core types. The objective is to use the fewest cores, then to meet as many end-to-end deadlines as possible, optionally for a target timing margin.

C2. Timing analyses with proofs (Section 6). They are an optimal priority assignment that respects precedence, response times with shared-memory contention, a bound on the maximum reaction time of a task-chain under implicit communication and under strict LET, Network Calculus bounds for the four TSN mechanisms, and a lower bound on the number of cores of any mapping of the sub-chains.

C3. A mapping heuristic (Section 7), given as pseudo-code. Its new parts are a communication-aware allocation rule, a core consolidation that preserves schedulability, a reservation of zone memory for pinned tasks, a deadline-driven local search with an exact incremental re-analysis, and the sizing of the platform for a target margin.

C4. An exact formulation (Section 8): a constraint program that finds the fewest cores, with schedulability added as lazy constraints. It measures the optimality gap of the heuristic for the sub-chains of the partitioning, without contention.

C5. A lifecycle analysis (Section 9). The platform is sized at the SOP, and then software updates add task-chains to the fixed hardware. This trades the hardware cost against the number of updates the platform absorbs.

C6. An evaluation (Section 11). The synthetic applications follow the statistics of an industrial engine-management software [8]. The evaluation adds a small detection chain and a whole-vehicle application of 2000 task-chains. A frame-level network simulation and a full-system simulation check the bounds. The method is compared with the optimum, with simulated annealing and a genetic algorithm, and with the rules of practice; each of its parts is removed in turn; strict LET is compared with implicit communication; the same software is mapped onto four architectures; and each assumption of the generator is varied.

Central claim

A system-level method that couples allocation, fixed-priority scheduling, shared-memory contention, LET and TSN bounds sizes the cores of a vehicle platform close to the optimal allocation of its sub-chains (2.3 to 2.9 % above it on average, proven optimal in 81 % of 140 instances) in milliseconds, and no end-to-end bound it reported was exceeded when the network and the whole system were simulated.

The method is implemented as an open, deterministic C library and command-line tool, which reads the application and the platform from JSON files. A separate user manual describes the tool, five use cases, and how to reproduce every number of this paper.

How to read this paper. Section 2 recalls the terms and mechanisms the paper builds on. A reader who knows LET, TSN and Network Calculus can skip it. Section 3 reviews the state of the art. Sections 4 and 5 define the model and the problem. Section 6 gives the timing analyses, Section 7 the heuristic, Section 8 the exact formulation and Section 9 the lifecycle analysis. Section 10 follows one task-chain through all the steps. Section 11 reports the experiments, Section 12 discusses design rules and limits, and Section 13 concludes. Every section starts with a short summary of what it contains.

2 Background and terminology

In brief. This section recalls the standard notions the paper uses: task-chains and their end-to-end metrics, fixed-priority scheduling, LET, TSN and Network Calculus. Table 1 lists the terms.

2.1 Task-chains and their end-to-end metrics

A *task* is released periodically and runs for at most its worst-case execution time (WCET). A *task-chain* is a sequence of tasks in which each task reads the output of the previous one. Automotive task-chains are multi-rate: the tasks of a chain may have different periods. Consecutive tasks with the same period form an *activation pattern*. They are released together and run in order. Between two patterns, the tasks communicate through a shared variable (a register): the consumer reads the latest value at its own release [8, 12].

Two end-to-end metrics are standard [7, 9, 19]. The *maximum reaction time* is the longest time from an external event (a change seen by the sensor) to the end of the first job of the last task whose output accounts for it. The *maximum data age* is the longest time a sensor value is used, from its sampling to the end of the last job of the last task that uses it. Günzel et al. [10] proved that the two are equal for cause-effect chains of periodic tasks. In this paper, the end-to-end deadline of a task-chain bounds its maximum reaction time, and the bound therefore also holds for the data age. The *end-to-end response time* of holistic analysis [33] is a third quantity: the sum of the response times and network delays along the chain, without the sampling and copy delays. It is one part of the reaction time, not a time of the system.

2.2 Fixed-priority scheduling on partitioned multicores

Each core runs its tasks with preemptive fixed priorities, and a task never migrates (partitioned scheduling [17]). The worst-case response time (WCRT) of a task is the longest time from its release to its completion; it is found by the classical response-time analysis [34, 35]. A core is *schedulable* if every task meets its implicit deadline, its period. Audsley's algorithm [36] assigns the priorities from the lowest level up; for schedulability tests that depend only on the set of higher-priority tasks, such as response-time analysis, it finds a schedulable order whenever one exists [18]. The *critical scaling factor* of a core is the largest factor by which all WCETs can be multiplied with the core still schedulable [37]; it measures the timing margin. Allocating tasks to cores under a utilization cap is a bin-packing problem; the classical heuristics take the tasks by decreasing utilization and put each one on the first (first fit), fullest (best fit) or emptiest (worst fit) core that can take it [17, 38].

2.3 Logical execution time

With the LET paradigm [11, 39], a task reads its inputs at its release and publishes its outputs at its next release (or deadline), whatever its actual execution. Between cores, the copies are made by a communication task at fixed instants [12, 13]. The timing of the data flow then no longer depends on execution times, at the price of sampling delays and of the copy task. With *implicit communication*, the AUTOSAR alternative, a task works on local copies of its data, which are written back when it finishes [12]: the delays are shorter, but they depend on response times. System-level LET extends LET across ECUs, with synchronized clocks [40].

2.4 Time-Sensitive Networking

TSN is a set of IEEE 802.1 standards for bounded-latency Ethernet [5]. A switch has one output queue per traffic class at each port. The paper uses four mechanisms.

- *Credit-based shaper* (CBS, 802.1Qav): a class may send only while its credit is non-negative; the credit grows at the *idle slope* while frames wait and drains while the class sends. It reserves a share of the link for the class and spreads its frames out.
- *Time-aware shaper* (TAS, 802.1Qbv): a cyclic gate schedule opens the queue of each class in its own time window.
- *Asynchronous traffic shaping* (ATS, 802.1Qcr [41, 42]): at every switch, a *regulator* holds each frame until the stream again conforms to its token-bucket description, and the classes are then served in strict priority. Bursts therefore do not grow from hop to hop.
- *Frame replication and elimination for reliability* (FRER, 802.1CB [43]): a stream is sent as two *member streams* over disjoint paths; the receiving side keeps the first copy of each frame and discards the other, so a single link or switch failure loses no frame.

Frame preemption (802.1Qbu) lets a time-critical frame interrupt a best-effort frame [44].

2.5 Network Calculus

Network Calculus bounds worst-case delays with curves [45, 46]. An *arrival curve* α bounds the traffic of a flow: in any window of length t, the flow sends at most $\alpha(t)$ bits. The token bucket $\alpha(t) = b + rt$ (burst b, rate r) is the usual one. A *service curve* β guarantees the service a port gives to a class; the rate-latency curve $\beta(t) = R\,[t - L]^+$ (rate R after a latency L) is the usual one. The delay of a flow at the port is then at most the horizontal distance between α and β, here $L + b/R$; and the flow leaves the port with a larger burst, $b + rd$ for a delay bound d. Along a route,

Table 1. Terms used in the paper.

term	meaning
task-chain	sequence of tasks, sensor to actuator (cause-effect chain)
activation pattern	maximal run of consecutive tasks of a chain with one period
sub-chain	run of consecutive tasks of a chain, the unit of allocation
segment	maximal run of consecutive sub-chains of a chain on one core
pinned	bound to one ECU (sensor and actuator drivers)
central computer	the ECU that runs most functions
zone controller (ZCU)	ECU of one zone of the car, wired to its devices
host	ECU that free sub-chains go to first: the central computers, else the ECUs with the fastest cores
inter-ECU communication	data sent between two segments on two ECUs
LET task	copy task of a core, run at its LET instants
implicit / strict LET	output copied after its producer ends / at the end of its period
utilization cap φ	largest load allowed on a core
maximum reaction time	event to the first actuation that accounts for it
end-to-end response time	sum of the response times and network delays of a chain (holistic)
critical scaling factor	largest WCET factor that keeps the cores schedulable
design point	one run of the method for one application and one set of parameters
SOP	start of production: the hardware is frozen

bursts therefore grow from port to port, unless regulators reshape them.

3 State of the art

In brief. The problem sits where six lines of work meet: zonal architectures, task allocation, task-chain analysis and LET, TSN analysis, shared-resource contention, and system-level analysis and the joint design of computation and communication. Table 2 places the closest works against the features of the problem. None covers allocation, priorities, task-chains, LET and network bounds together on a zonal platform, and none sizes the cores.

3.1 Zonal and centralized E/E architectures

Bandur et al. [2] make the case for centralized automotive architectures: fewer, more powerful computers, a service-based software platform and a backbone network. Askaripoor et al. [3] survey E/E architecture synthesis, of which the allocation of software to computing units is one step. Mauser and Wagner [6] combine a literature review and industrial interviews to identify seven system properties that limit current E/E architectures (bus load, functional safety and computing power among them) and use them to assess the potential of centralization.

Quantitative studies of zonal platforms have so far looked at the network and the harness. Park et al. [31] simulate the end-to-end delay of TSN streams for 2 to 10 zones and estimate the length and weight of the harness. Aghababaiyan et al. [32] schedule tasks between zone ECUs and a central unit over several topologies, and report deadline satisfaction from simulation, without a worst-case analysis. Zaheri and Reuss [47] allocate tasks to ECUs with a mixed-integer program under ISO 26262 constraints [48] (ASIL decomposition, memory) on a case of six tasks. None of these works sizes the cores of the ECUs under a worst-case analysis of the task-chains and the network.

3.2 Task allocation and partitioned scheduling

Allocating periodic tasks to processors is a bin-packing problem, and the decision version of bin packing is NP-complete in the strong sense [49]. Partitioned scheduling therefore relies on bin-packing heuristics (first, best or worst fit by decreasing utilization) with a per-core schedulability test [17], and task splitting extends them [50]. First fit decreasing uses at most 11/9 of the optimal number of bins plus 6/9 [51]. On heterogeneous processors the size of an item depends on the bin [52].

For distributed systems, Tindell et al. [15] searched allocations by simulated annealing, with response-time analysis of tasks and messages. Zheng et al. [53] and Zhu et al. [16] optimize the allocation and the priorities together with mixed-integer programming, under end-to-end deadlines on a CAN-based platform. Zhu et al. [54] optimize the *extensibility* of such a system, the room left for WCET increases. Cambazard et al. [55] solve a hard real-time allocation problem exactly by decomposition: a master problem allocates, and a schedulability subproblem returns explanations that cut the master; Section 8 follows the same idea. Davare et al. [56] optimize periods for latency. Bhat et al. [57] allocate primary and backup tasks for software fault tolerance in automotive systems, so that replicas never share a processor.

Within a processor, the WCRT of fixed-priority tasks is the least fixed point of a recurrence [34, 35]. Audsley's algorithm [36] finds a feasible priority order whenever one exists, for any test that does not depend on the relative order of the higher-priority tasks [18]. Davis and Burns [58] refine it to maximize the tolerance to interference, and Davis et al. [18] review its extensions. The margin of a schedulable system is measured by the critical scaling factor [37] and, more generally, by sensitivity analysis [59]. The priority orders of tasks with precedence constraints are a restriction these methods do not handle directly; Section 6.2 adapts Audsley's argument to it.

3.3 Task-chains and logical execution time

Feiertag et al. [7] defined the reaction time and the data age of task-chains whose tasks communicate through registers. Becker et al. [19] bound the maximum data age of automotive task-chains for several levels of system knowledge, including implicit and LET communication. Dürr et al. [9] and Günzel et al. [20, 60] extend the bounds to sporadic and asynchronous distributed task-chains, and Günzel et al. [10] show that maximum reaction time and maximum data age are equal.

LET originates in Giotto [39]; Kirsch and Sokolova [11]

describe the paradigm. Bosch reported its industrial use for multicore engine control [12] and published the statistics of an engine-management software as a WATERS benchmark [8]. Biondi and Di Natale [13] implement LET on multicores and measure its overhead. Martinez et al. [21, 61] characterize end-to-end delays under LET and other communication models. Pazzaglia et al. [22] optimize the deployment of functions onto the tasks and cores of one multicore under LET, and Paladino et al. [23] assign per-core priorities to minimize the latency of task-chains that span cores. Gemlau et al. [40] extend LET to distributed systems by giving the network its own logical intervals. All these works analyse or optimize task-chains on a *given* platform; the sizing of the platform is outside their scope.

3.4 Timing analysis of TSN

Network Calculus [45, 46] bounds delays with arrival and service curves. For the credit-based shaper, Queck [62] and De Azua and Boyer [24] derive service curves for the SR classes A and B; Mohammadpour et al. [25] and Zhao et al. [26] bound the credit of any number of classes and give the service curves from which Section 6.3 takes its rate-latency form. For time-aware shaping, Zhao et al. [27] give Network Calculus bounds and Craciunas et al. [63] synthesize gate schedules. Thiele and Ernst [44] analyse frame preemption, and Zhao et al. [14] compare the shapers.

Asynchronous traffic shaping comes from the urgency-based scheduler of Specht and Samii [42]. Le Boudec [28] proved that interleaved regulators placed behind a FIFO system do not increase its worst-case delay ("shaping for free"), so that the bursts of a flow can be bounded at their source value at every hop. Thomas et al. [64] analyse packet replication and elimination: the elimination increases the burstiness of a flow, and an interleaved regulator placed after it loses the shaping-for-free property. Most TSN analyses take the streams as given; in this paper the streams depend on the mapping.

3.5 Shared-resource contention

On a multicore, tasks of different cores delay each other on shared caches, buses and memory. Paolieri et al. [65] bound the delay of a request under round-robin arbitration by one request of every other core, which makes WCETs composable. Davis et al. [29] integrate such interference into response-time analysis (multicore response time analysis). Allocation studies usually assume WCETs that already include the worst interference; Section 4.1 makes the interference depend on the mapping instead. Zhu et al. [66] refine the shared-cache interference of cause-effect chains on a given mapping of chains to cores, which lowers their end-to-end latency bounds.

3.6 System-level analysis and joint design

Compositional system-level analysis, as in SymTA/S [30], propagates event models between the local analyses of processors and buses, and bounds end-to-end delays for a given mapping. Craciunas and Oliver [67] synthesize task and network schedules together for time-triggered systems. Pop et al. [68] optimize the routing and scheduling of time-triggered and AVB traffic in TSN. Both assume the tasks are already allocated. Arestova et al. [69] schedule multi-rate cause-effect chains of tasks and of their TSN flows together, by an incremental heuristic under end-to-end deadlines; they compute a schedule, where this paper allocates the tasks and bounds the delays of fixed-priority tasks and of shaped streams. Zhu et al. [16] do allocate tasks, to ECUs connected by a CAN bus with fixed-priority messages, with a mixed-integer program.

3.7 Positioning

Table 2 summarizes the gap. Works that allocate tasks either ignore the network or assume a bus with fixed-priority messages; works that bound task-chains, LET or TSN take the allocation as given; contention is analysed on a given mapping; and no work both sizes the cores of a zonal platform and bounds its task-chains with all these couplings in one system-level analysis. The aim of this paper is a sound, explainable and scalable method that a system architect can run on the software of a whole vehicle in early design. It compares platforms and design rules (core types, network rate, TSN mechanism, replication policy, provisioning for updates), and every number it reports has a worst-case analysis behind it. Its distance to the optimum is measured with an exact formulation on instances of up to 300 task-chains (Section 11.3).

4 System model

In brief. The platform is a central computer and zone controllers with typed cores, a shared memory per ECU, and a TSN network (Sections 4.1 and 4.4); Section 4.5 extends it to domain, hybrid and distributed architectures and to any network topology. The application is a set of multi-rate task-chains with bindings to ECUs and safety constraints (Section 4.2). Cores communicate at LET instants, with implicit communication or strict LET (Section 4.3). Section 4.6 defines how a mapped task-chain is cut into segments and inter-ECU communications, and which timing metrics are computed. Table 3 collects the notation.

Times are integers in microseconds and utilizations are fractions of a core. The implementation stores utilizations as integers in parts per million and rounds them up, so every rounding is conservative.

4.1 Platform

ECUs and cores. The platform has a set of ECUs $\mathcal{K} = \{k_0, k_1, \dots, k_Z\}$ (Figure 1). k_0 is the central computer and $k_1, \dots, k_Z$ are the zone controllers; the default is $Z = 5$. An ECU holds cores of the types $\Theta(k)$ it supports. The central computer offers *big* application cores and, optionally, slower *little* cores. A zone controller offers microcontroller cores (*MCU*) and, optionally, *lock-step* cores (LS), a common safety mechanism for the functions of high integrity levels under ISO 26262 [48]. A type θ has a speed s_θ, in per-

Table 2. Positioning against the closest works. ✓: covered; ∘: partly, or under strong assumptions; –: not addressed (as described in the publications). *Alloc.*: allocation of tasks to cores or ECUs; *Prio.*: fixed-priority assignment; *Chains*: end-to-end bounds of multi-rate task-chains; *LET*: LET communication; *Net.*: worst-case network bounds; *Zonal*: zonal bindings and spatial constraints (memory, replicas, co-location); *Het.*: heterogeneous core speeds or types; *Size*: minimizes the number of cores; *Exact*: exact method or optimality gap measured; *Life*: evolution of the software after the design; *Scale*: largest reported case.

work	Alloc.	Prio.	Chains	LET	Net.	Zonal	Het.	Size	Exact	Life	Scale
Tindell et al. 1992 [15]	✓	✓	∘	–	∘	–	–	–	–	–	–
Cambazard et al. 2004 [55]	✓	✓	–	–	∘	∘	–	–	✓	–	–
Zhu et al. 2009/12 [16, 54]	✓	✓	✓	–	✓ (CAN)	–	∘	–	✓	∘	–
Bhat et al. 2017 [57]	✓	–	–	–	–	∘ (replicas)	–	∘	–	–	–
Pazzaglia et al. 2019 [22]	✓	∘	✓	✓	–	–	–	–	∘	–	one multicore
Paladino et al. 2024 [23]	–	✓	✓	∘	–	–	–	–	–	–	one multicore
Gemlau et al. 2021 [40]	–	–	✓	✓	∘	–	–	–	–	–	–
Günzel et al. 2021/23 [20, 60]	–	–	✓	✓	∘	–	–	–	–	–	–
Craciunas, Oliver 2016 [67]	–	∘ (TT)	✓	–	✓ (TT)	–	–	–	✓	–	–
Zaheri, Reuss 2025 [47]	✓	∘	∘	–	–	∘ (ASIL, memory)	✓	–	✓	–	6 tasks
Aghababaiyan et al. 2025 [32]	✓	–	–	–	– (simulated)	✓	✓	–	–	–	–
This paper	✓	✓	✓	✓	✓ (4 TSN)	✓	✓	✓	✓	✓	**13 869 tasks**

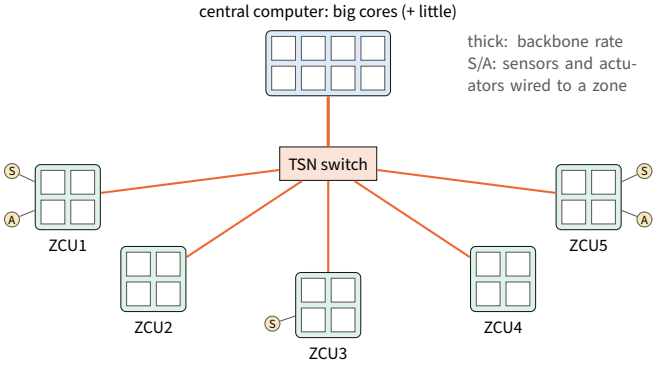


Figure 1. The zonal platform (star topology, default sizes): a central computer, five zone controllers (ZCU) of four cores, one TSN switch. Cores are added where the application needs them.

cent of a reference core. By default $s_{\text{big}} = 100$, $s_{\text{little}} = 40$ and $s_{\text{MCU}} = s_{\text{LS}} = 50$.

The initial architecture $\mathcal{P}_0$ has a number of cores of each type on each ECU: by default 8 big cores on the central computer and 4 MCU cores on each zone controller, 28 cores in all. When the application needs more, cores are added. The number of cores *used*, that is, running at least one application task, is the main sizing result, M.

Utilization cap and memory. The total utilization of the tasks of a core, its LET task included, may not exceed the utilization cap φ (default 0.8). The remaining $1 - \varphi$ is the design reserve for estimation errors and future functions. ECU k has a memory M_k for the code and data of its tasks (unlimited by default).

Execution times. Every task has a reference WCET C_τ^{ref}, measured on the reference core, and a factor $f_{\tau,\theta}$ (in percent, default 100) for each core type. The factor models tasks that gain or lose more than the speed ratio on a type, for instance through caches or vector units. On a core p of type $\theta = \theta(p)$, the WCET in isolation is

$$C_\tau^{\text{iso}}(p) = \max\left(1, \left\lceil \frac{C_\tau^{\text{ref}} f_{\tau,\theta}}{s_\theta} \right\rceil\right). \tag{1}$$

Shared-memory contention. The cores of one ECU share its memory. A job of task τ issues at most a_τ requests to it. The memory serves one request in ℓ nanoseconds, and serves the cores in round robin. Each core has at most one request pending at a time (in-order cores, blocking memory accesses). A request therefore waits at most for one request of every other core that competes for the memory [65]. If n_k cores of ECU k compete, the WCET of τ on a core p of k is

$$C_\tau(p) = C_\tau^{\text{iso}}(p) + \left\lceil \frac{a_\tau\, \ell\, (n_k - 1)}{1000} \right\rceil, \qquad U_\tau(p) = \frac{C_\tau(p)}{T_\tau}. \tag{2}$$

The waiting time does not depend on the speed of the core, which is why it is added after the scaling of Eq. (1). The number of competing cores depends on the mapping: Section 7.4 explains how the method fixes it. When no request count is given, a task gets $a_\tau = \lceil \eta\, C_\tau^{\text{ref}}/1000 \rceil$ requests, a density of η requests per millisecond of reference WCET (default 1000). With $\ell = 0$, the default, there is no contention and $C_\tau(p) = C_\tau^{\text{iso}}(p)$.

4.2 Application

> **Definition 1** (Task). *A task τ is periodic with period T_τ and an implicit deadline equal to it. Its attributes are the reference WCET C_τ^{ref} and the type factors $f_{\tau,\theta}$; the number of memory requests a_τ; the size m_τ (bytes) of the output it produces at every job; its memory μ_τ; the set $A_\tau \subseteq \mathcal{K}$ of ECUs and the set $\Theta_\tau \subseteq \Theta$ of core types it may run on; and a co-location flag g_τ. With g_τ set, the task must run on the same core as its successor in the chain.*

A task with $|A_\tau| = 1$ is *pinned* to its ECU: sensor and actuator drivers run on the zone controller their device is

Table 3. Main notation.

symbol	meaning
$\mathcal{K} = \{k_0, \dots, k_Z\}$	ECUs; on the zonal platform, k_0 the central computer, $k_1..k_Z$ the zone controllers
Θ, $\Theta(k)$	core types; types available on ECU k
s_θ, $s_{k,\theta}$	speed of type θ (on ECU k), % of the reference core
$\mathcal{P}$, $\kappa(p)$, $\theta(p)$	cores; ECU and type of core p
φ	utilization cap of a core ($0 < \varphi \leq 1$)
Ξ, ξ	application: set of task-chains; one task-chain
τ, T_τ, C_τ^{ref}	task, its period, its WCET on the reference core
U_τ^{ref}	utilization on the reference core, $C_\tau^{\text{ref}}/T_\tau$
$f_{\tau,\theta}$	WCET factor of τ on type θ (%)
$C_\tau^{\text{iso}}(p)$, $C_\tau(p)$, $U_\tau(p)$	WCET of τ on core p alone, with contention; utilization
a_τ, ℓ, n_k	memory requests per job; time per request; competing cores of ECU k
A_τ, Θ_τ	ECUs and core types τ may run on
μ_τ, M_k	memory of τ; memory of ECU k
D_ξ	end-to-end deadline of ξ (on its maximum reaction time)
$\rho(\xi)$	replica group of ξ
$\mathcal{F}$, π	sub-chains; mapping of sub-chains to cores
T_p^{LET}, C_p^{LET}	period and WCET of the LET task of core p
R_τ	worst-case response time (WCRT) of τ
σ	utilization limit of a sub-chain (fraction of a core)
$G = (V, E)$, r_e	network graph; rate of link e (bit/µs)
δ	latency of a switch
X, ϑ_x, l_x	traffic classes; period threshold of class x; its largest frame at a port
b_h, J_h, T_h	bits, release jitter and period of flow h
σ_h, ρ_h	burst and rate of flow h (unrelated to σ and $\rho(\xi)$)
$d_{e,x}$, d_h	delay bound of class x at port e; of flow h
w_h	copy-out delay of communication h (Eq. (16))
Λ_ξ, Γ_ξ, Δ_ξ	end-to-end response time (holistic sum), end-to-end delay, maximum reaction time of ξ
$M(\pi)$, $N(\pi)$	cores used; task-chains on time
$\lambda(\pi)$, $\lambda^\star$	critical scaling factor of a mapping; its target

wired to. The other tasks are *free*. A free task may still be restricted, for instance to the zone controllers only ($A_\tau = \{k_1, \dots, k_Z\}$), or kept out of the zones of its replica (below).

> **Definition 2** (Task-chain). *A task-chain $\xi = (\tau_1, \dots, \tau_n)$ is a sequence of tasks in which the output of τ_i is the input of τ_{i+1}. A maximal run of consecutive tasks with the same period is an* activation pattern *[8]. If two patterns of a task-chain have the same period, their tasks on a common core are also kept in chain order, which only restricts the priorities. The task-chain has an end-to-end deadline D_ξ on its maximum reaction time (Section 4.6) and, possibly, a replica group $\rho(\xi)$.*

The links of a task-chain come in two kinds (Figure 2).

- *Inside a pattern*, tasks are released together. On one core they execute in chain order: there is a *precedence* constraint, so a job uses the data of the job of its predecessor released with it. When the partitioning cuts a pattern, for instance at a sensor driver bound to a zone, the two parts run on different cores, and the link becomes a communication: the consumer then reads the data of an earlier job, with up to one consumer period of sampling, which Section 6.4 bounds.
- *Between two patterns*, data goes through a *register*: the consumer reads the latest value at its own release. There is no precedence, but there is a sampling delay of up to one consumer period.

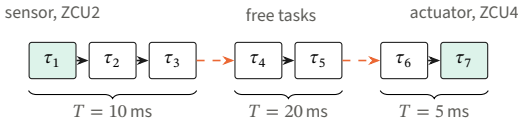


Figure 2. A task-chain of three activation patterns. Solid arrows: precedence inside a pattern; dashed arrows: registers between patterns. Shaded tasks are pinned to a zone controller.

The task-chains of one replica group are redundant implementations of one safety function, with redundant sensors and actuators. No two of them may run on the same ECU or, under a weaker policy, on the same core [57].

4.3 Scheduling and communication on the cores

Scheduling. Every core schedules its tasks with preemptive fixed priorities. Inside a pattern, the tasks mapped to the same core get priorities that decrease along the chain, which enforces the precedence on that core.

Communication between cores. Tasks on different cores never share data directly. Every core p that runs at least one application task runs one *LET task*, a copy task with the highest priority on p, released at the *LET instants* of p. It copies data between the local memory of p and the global memory, or the network interface, and

$$T_p^{\text{LET}} = \gcd\{T_\tau : \tau \text{ on } p\},$$
$$C_p^{\text{LET}} = \left\lceil \frac{100\, c_{\text{LET}}}{s_{\theta(p)}} \right\rceil + \left\lceil \frac{a_{\text{LET}}\, \ell\, (n_k - 1)}{1000} \right\rceil, \qquad (3)$$

so that every release of every task of p is a LET instant. A consumer on another core reads, at its release, the last value copied in by then. Two schemes decide when an output is copied out.

- *Implicit communication* (the default): an output is copied at the first LET instant after its producer finishes. It is the implicit communication of AUTOSAR [12], with the copies grouped at the LET instants. The delays are short, but the data flow depends on response times.
- *Strict LET* [11, 13]: an output is published at the end of the period of its producer, which is a LET instant of its core, whatever the response time. The data flow no longer depends on response times, at the price of longer delays.

Section 6.4 bounds both, and Section 11.5 compares them. Here c_{LET} is the copy time on the reference core (default 50 µs, of the order of the LET communication overheads that Biondi and Di Natale measured on a multicore [13]), and $a_{\text{LET}} = \lceil \eta\, c_{\text{LET}}/1000 \rceil$ are its memory requests. The copy time is a constant per LET instant, whatever the amount of data copied (Section 12). The LET load $C_p^{\text{LET}}/T_p^{\text{LET}}$ belongs to the core. On a reference core, a 1 ms task anywhere on it costs 5 % of LET load, while periods

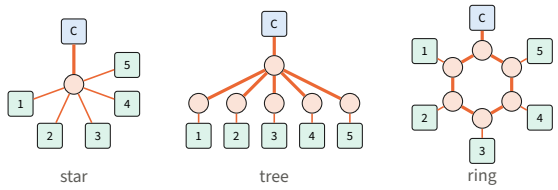


Figure 3. Topologies. C: central computer; 1–5: zone controllers; circles: switches. Thick links run at the backbone rate r_b, thin ones at r.

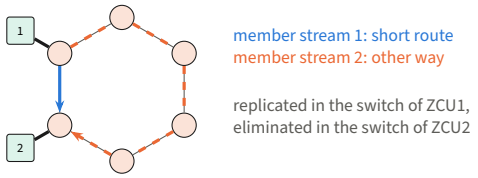


Figure 4. Frame replication on the ring for a message from ZCU1 to ZCU2. The two member streams share only the two links between the ECUs and their switches.

that are all multiples of 10 ms cost 0.5 %. The period mix of a core therefore matters, which is one reason why mapping is more than packing utilizations.

The *load* of core p under a mapping is

$$\mathcal{L}(p) = \sum_{\tau \text{ on } p} U_\tau(p) + \frac{C_p^{\text{LET}}}{T_p^{\text{LET}}} \le \varphi. \tag{4}$$

4.4 Network

Topology and routing. The network is a directed graph $G = (V, E)$ whose nodes are the ECUs and the switches. Every full-duplex cable gives two directed links, and each link e has an output port at its origin and a rate r_e. Three topologies are considered (Figure 3):

- *star*: one switch, one link per ECU;
- *tree*: one switch per ECU, and the zone switches linked to the switch of the central computer;
- *ring*: one switch per ECU, the switches in a ring.

ECU links run at the link rate r (default 1 Gbit/s). Links between switches, and the link of the central computer, which carries most of the traffic, run at the backbone rate $r_b \ge r$ (by default $r_b = r$). A message follows a shortest path. ECUs do not forward, and ties are broken by a fixed link order (clockwise on the ring). Every switch adds a constant latency δ (default 2 µs) after store-and-forward.

Frames. A message of m bytes is cut into frames of at most 1500 bytes of payload. Each frame carries 42 bytes of overhead: header 14, VLAN tag 4, checksum 4, preamble 8 and inter-frame gap 12. A payload is padded to at least 42 bytes, the 64-byte minimum frame with a VLAN tag. The bits on the wire are

$$b(m) = \sum_i 8\,(\max(m_i, 42) + 42), \tag{5}$$

over the payload chunks m_i of the message.

Traffic classes. LET traffic uses $X \in \{1, 2, 3\}$ classes of IEEE 802.1Q [5]. A stream whose producer has period T belongs to the first class x with $T \le \vartheta_x$; class 0 has the highest priority. The default thresholds are 20 ms for $X = 2$, and 5 ms and 20 ms for $X = 3$. Other traffic is best effort. It is assumed to be always present, with maximum-size frames of $L_{BE} = 1542$ bytes (12 336 bits) on the wire.

Shaping. Every port shapes the LET classes with one of three mechanisms (Section 2).

- *CBS*: one credit-based shaper per class, with idle slopes $I_{e,x}$ taken from a budget of shares of r_e (default 75 % for one class; 50 and 25 % for two; 50, 15 and 10 % for three).
- *TAS*: one exclusive window per class in a gate cycle T_c (default 500 µs), with guard bands (default windows 70 %; 40 and 30 %; 30, 20 and 20 %).
- *ATS*: strict priority between the classes, and at every switch one interleaved regulator per input port and class. Each stream is regulated to its token bucket at its source (Section 6.3).

With frame preemption (802.1Qbu), a best-effort frame blocks for at most 143 bytes [44]. A link may also carry a design limit on its LET load (default 100 %).

Frame replication. On the ring, frame replication (FRER) may be enabled for the task-chains of the replica groups, whose redundancy would otherwise end at the network. Each of their inter-ECU messages is then sent as two member streams (Figure 4). The switch of the source ECU replicates every frame onto both directions of the ring. The switch of the destination ECU keeps the first copy of each sequence number and discards the other. The two routes share only the links between the ECUs and their switches, so a single failure of a link between switches, or of a switch other than these two, leaves one member stream.

4.5 Other E/E architectures

The zonal platform above is the running example, but nothing in the model needs it. In general, a platform is a list of ECUs, and each ECU k has a *role*: central computer, zone controller, domain controller, or ECU of a distributed architecture. It also has initial cores of given types, a speed $s_{k,\theta}$ for each of its types (the speed of a type may differ from one ECU to another), a memory M_k, the type of the cores added to it (another of its types is added for a sub-chain that may not run on that one), and a largest number of cores $M_k^{\max}$ (unbounded by default). The role only matters for the sub-chains that are bound to no ECU (*free* sub-chains): they go to their *hosts* first, and the cores added for them are added to a host first. The hosts are the central computers. On a platform without a central computer, they are the ECUs whose fastest initial cores are the fastest of the platform, where a sub-chain costs the least utilization (Section 11.3 measures what this choice is worth). A free sub-chain tries the cores of the hosts first, then the initial cores of the other ECUs it is allowed on; it uses a core added to a non-host ECU only if no host admits it (the communication-aware rule CADU of Section 7 weighs the hosts against the hops

Table 4. The four platforms of the evaluation: initial cores per ECU (B: big, L: little at 40 %, M: MCU at 50 %, LS: lock-step at 50 %), star topology. Cores are added of the first type of each ECU.

platform	ECUs and initial cores
zonal	central computer 8 B; five zone controllers of 4 M; 1 Gbit/s
domain	ADAS 4 B; infotainment 2 B + 4 L; powertrain 3 M + 1 LS; chassis 2 M + 2 LS; body 2 M; 1 Gbit/s
hybrid	central computer 8 B; ADAS domain controller 4 B; four zone controllers of 4 M; 1 Gbit/s
distributed	ADAS 2 B (at most 4); ESC and EPS 1 LS each; nine ECUs of 1 M (engine, transmission, body, climate, cluster, four doors); at most 2 cores per ECU except ADAS; 100 Mbit/s

instead). A new core for it goes to the first ECU that admits it, the hosts first and then the others, each in the order of the platform, provided the ECU has fewer than $M_k^{\max}$ cores, enough memory and no replica of its task-chain. Everything else (allowed ECUs, core types, memory, replica separation, schedulability, LET, the network analysis) is defined per ECU and per core, and does not depend on the role.

The network may also be any set of switches and full-duplex links (*custom* topology), each ECU attached to one switch, with a rate per link. Routes are shortest paths. With FRER, the second member stream of a replicated flow takes the shortest route that shares no link and no intermediate switch with the first; a flow without such a route is not replicated. On the ring this is the other way around the ring, as above.

Table 4 lists the four platforms of the evaluation (Section 11.9): the zonal platform, a domain architecture of five domain controllers, a hybrid one (a central computer, an ADAS domain controller and four zone controllers), and a distributed one of twelve small ECUs.

4.6 Mapping, segments and timing metrics

A task-chain is mapped as a sequence of *sub-chains*: runs of consecutive tasks placed together on one core. Following the chain, consecutive sub-chains on the same core form a *segment*. Between two segments, data is sent either inside an ECU (shared memory) or between two ECUs, over the network. Every *inter-ECU communication* h is a network *flow*. It carries one message of $b_h = b(m)$ bits per period T_h of its producer (the last task of the source segment), released at a LET instant of the source core: after the producer has finished (implicit communication) or at the end of its period (strict LET).

Three end-to-end quantities are used for a task-chain ξ [7, 9, 19].

- The *maximum reaction time* Δ_ξ is the longest time from an external event to the end of the first job of τ_n that accounts for it, every sampling delay included. It equals the maximum data age [10], and it is the metric compared with the deadline.
- The *end-to-end delay* Γ_ξ is the longest time from the release of a job of τ_1 to the end of the first job of τ_n that uses its data, every copy and sampling delay included. It is the reaction time without the wait for the first sample.
- The *end-to-end response time* Λ_ξ is the holistic sum [33]: the response times of the runs of the chain (below) plus the network delays, without any copy or sampling delay. It is not a time of the system but a figure of merit, which shows how much of Δ_ξ is computation and network.

A task-chain *meets its deadline* when all its tasks meet theirs, every port its flows cross has a delay bound, and $\Delta_\xi \le D_\xi$. Section 6.4 bounds Γ_ξ and Δ_ξ.

5 Problem statement

In brief. A mapping cuts every task-chain into sub-chains, chooses the cores, allocates the sub-chains and sets the priorities. It must respect seven constraints (C1–C7). The objective is lexicographic: the fewest cores, then the most task-chains that meet their deadline. Two variants add a timing margin and the software updates of the platform's life. Even the first objective is strongly NP-hard.

5.1 Decisions

A *mapping* of an application Ξ onto the platform consists of four decisions.

(D1) A partition of every task-chain into sub-chains, i.e. a set $\mathcal{F}$ of runs of consecutive tasks. Every sub-chain f inherits $A_f = \bigcap_{\tau\in f} A_\tau$, $\Theta_f = \bigcap_{\tau\in f} \Theta_\tau$, $\mu_f = \sum_{\tau\in f} \mu_\tau$ and the replica group of its chain.

(D2) A set of cores $\mathcal{P} \supseteq \mathcal{P}_0$, made of the initial cores and the added ones, each with an ECU $\kappa(p)$ and a type $\theta(p) \in \Theta(\kappa(p))$.

(D3) An allocation $\pi : \mathcal{F} \to \mathcal{P}$.

(D4) A priority order on every core: the LET task first, then a total order of the application tasks.

5.2 Constraints

A mapping is *admissible* when, for all sub-chains f, f', cores p and ECUs k:

$$\kappa(\pi(f)) \in A_f, \quad \theta(\pi(f)) \in \Theta_f \tag{C1}$$

$$g_{\tau_i} = 1 \;\Rightarrow\; \tau_i, \tau_{i+1} \text{ on the same core} \tag{C2}$$

$$\mathcal{L}(p) \le \varphi \tag{C3}$$

$$\sum_{f:\kappa(\pi(f))=k} \mu_f \le M_k \tag{C4}$$

$$\rho(f) = \rho(f') \neq \varnothing,\ \xi(f) \neq \xi(f') \Rightarrow \kappa(\pi(f)) \neq \kappa(\pi(f')) \tag{C5}$$

$$\tau_i, \tau_j \in \xi \text{ on } p,\ T_{\tau_i} = T_{\tau_j},\ i < j \Rightarrow \text{prio}(\tau_i) > \text{prio}(\tau_j) \tag{C6}$$

$$R_\tau \le T_\tau \quad \text{for every application task } \tau \tag{C7}$$

In words: (C1) respects the bindings to ECUs and the allowed core types; (C2) keeps co-located tasks together, so a group of co-located tasks takes the constraints of all its tasks (the ECU of a driver in it, the common allowed ECUs and core types), and an input whose group has none in common is rejected; (C3) is the utilization cap and (C4) the memory of each ECU; (C5) keeps redundant task-chains on different ECUs (the core policy replaces $\kappa(\pi(\cdot))$ with $\pi(\cdot)$); (C6)

enforces precedence on a core, for the tasks of one chain with the same period (Section 4.3); and (C7) asks every task to meet its deadline, with the response times of Section 6.1. With contention, the WCETs in (C3) and (C7) are those of Eq. (2), so they depend on the number of cores of the ECU that run tasks. A mapping that cannot satisfy (C1)–(C5), even with added cores, is *infeasible*. This happens when a sub-chain does not fit an empty core of any allowed type, or when the pinned tasks alone exceed a memory.

5.3 Objectives

Let $M(\pi)$ be the number of cores that run at least one application task, and $N(\pi)$ the number of task-chains that meet their deadline (Section 4.6).

Problem (mapping task-chains onto a vehicle platform)

Given the platform, the network configuration and the application, find an admissible mapping that

(O1) minimizes the number of cores used, $M(\pi)$; then

(O2) maximizes the number of task-chains that meet their deadline, $N(\pi)$.

Sizing variant: given a target $\lambda^\star \geq 1$, add the constraint $\lambda(\pi) \geq \lambda^\star$ on the critical scaling factor (Section 6.5): every WCET may grow by the factor $\lambda^\star$ without a deadline miss.

Lifecycle variant: the hardware chosen for Ξ at the SOP is then fixed. Given software updates $\Xi_1, \Xi_2, \ldots$ (sets of new task-chains), maximize the number of updates that can be added, one after the other, without new cores and without moving the sub-chains already deployed (Section 9).

The order of the objectives reflects early design. The hardware is the cost, and timing is first a constraint (C7), then a quality. The two objectives conflict. Packing sub-chains tightly lowers M, but it spreads task-chains, and a task-chain spread over ECUs pays LET sampling and network delays. Section 11 therefore reports the trade-off between them, and the method offers a communication-aware rule that favours (O2).

Proposition 1 (Hardness). *Deciding whether an admissible mapping with $M(\pi) \leq m$ exists is NP-hard in the strong sense, even with one ECU, one core type, one task per task-chain, harmonic periods and no network.*

Proof. The reduction is from bin packing, whose decision version is NP-complete in the strong sense [49]: given integer sizes $s_1, \ldots, s_n$, a capacity B and a number m, can the items be packed into m bins of capacity B? Build one single-task chain per item, all with the period $T = B$ and the WCET $C_i = s_i$ on a single core type of speed 100, so that the numbers of the instance are those of the bin-packing instance. Set $c_{\mathrm{LET}} = 0$, $\ell = 0$ and $\varphi = 1$, with unlimited memory, no replicas and no co-location. A single task cannot be cut, so the sub-chains are the tasks. All tasks of a core are released together and share one period, so under any fixed-priority order the lowest-priority task completes after $\sum C_i$, and the core is schedulable if and only if $\sum C_i \leq T$, that is, if its utilization is at most 1. So (C3) and (C7) coincide, and the admissible mappings onto m cores are exactly the packings of the items into m bins. The reduction is polynomial and keeps the numbers polynomially bounded, so strong NP-hardness carries over. ∎

The problem is claimed NP-hard, not NP-complete. Membership in NP is not known: the natural certificate, a mapping, is checked with fixed-priority response times, and computing them is NP-hard in general [70].

The response times, the LET periods (a GCD of the periods of each core), the contention (the number of cores used per ECU), the multi-rate reaction times and the network fixed points are all non-linear in the decisions, and a vehicle has thousands of tasks. The method of Section 7 is therefore a decomposition heuristic. Its distance to the optimum of (O1) is measured in two ways: by a lower bound on any mapping (Section 6.6) and, on instances of up to 300 task-chains, by an exact formulation (Section 8).

6 Timing analysis

In brief. This section gives the analyses that the method calls, with the arguments that make them sound: response times on a core (Section 6.1), an optimal priority assignment that respects precedence (Section 6.2), Network Calculus bounds for CBS, TAS, ATS and FRER (Section 6.3), the end-to-end bounds of a task-chain (Section 6.4), the critical scaling factor (Section 6.5) and a lower bound on the number of cores (Section 6.6).

6.1 Response times on a core

Let p be a core, $\mathrm{hp}(\tau)$ the tasks of p with a higher priority than τ (the LET task included), and $C_j = C_j(p)$ the WCETs of Eq. (2), contention included. The WCRT of τ is the least fixed point of [34, 35]

$$R = C_\tau + \sum_{j \in \mathrm{hp}(\tau)} \left\lceil \frac{R}{T_j} \right\rceil C_j. \tag{6}$$

The right-hand side $F(R)$ is non-decreasing in R. If $R^{(0)} \leq R^*$ and $R^{(0)} \leq F(R^{(0)})$, the iterates $R^{(i+1)} = F(R^{(i)})$ are non-decreasing integers bounded by the least fixed point R^*, so they reach it in finitely many steps; both starting values of Lemma 1 satisfy the two conditions. The iteration stops early as soon as $R^{(i)} > T_\tau$ (deadline miss). With contention, Eq. (2) adds to each WCET the longest wait of its memory requests, in the spirit of multicore response-time analysis [29], with the round-robin bound of [65]; it is safe for any interleaving of the requests of the other cores.

Lemma 1 (Starting point). *With $U_{\mathrm{hp}} = \sum_{j \in \mathrm{hp}(\tau)} C_j/T_j < 1$, the least fixed point R^* satisfies $R^* \geq \max\big(C_\tau + \sum_{\mathrm{hp}} C_j,\ C_\tau/(1 - U_{\mathrm{hp}})\big)$.*

Proof. Every higher-priority task is released at the critical instant, so $\lceil R/T_j \rceil \geq 1$. Since $\lceil x \rceil \geq x$, also $R^* \geq C_\tau + U_{\text{hp}} R^*$. ∎

In floating point, the second term is computed as $\lfloor C_\tau/(1-U_{\text{hp}}+10^{-9}) \rfloor - 1$, which keeps it a valid lower bound despite rounding.

Lemma 2 (Grouping by period). *Partition* $\text{hp}(\tau)$ *into groups* $G_1, \ldots, G_q$ *of equal periods* $T^{(1)}, \ldots, T^{(q)}$*, and let* $W_g = \sum_{j \in G_g} C_j$*. Then* $F(R) = C_\tau + \sum_{g=1}^{q} \lceil R/T^{(g)} \rceil W_g$*, and the iteration produces the same integer sequence in* $O(q)$ *instead of* $O(|\text{hp}(\tau)|)$ *operations per step.*

Proof. For $j \in G_g$, $\lceil R/T_j \rceil = \lceil R/T^{(g)} \rceil$; factor it out. All quantities are integers, so the sums are exact. ∎

Automotive applications use few distinct periods; the WATERS benchmark has nine, besides its angle-synchronous tasks [8]. A core with hundreds of tasks therefore costs a handful of terms per step. To compute all the response times of a core in priority order, the groups, $\sum_{\text{hp}} C_j$ and U_{hp} are accumulated incrementally.

6.2 Optimal priority assignment under precedence

Audsley's algorithm [36] fills the priority levels from the lowest. At each level, it gives the level to any task that is schedulable there with all unassigned tasks above it. It is optimal for any test in which the schedulability of a task depends on the *set* of higher-priority tasks, but neither on their order nor on the order of the lower-priority tasks, and in which a task cannot become unschedulable when its priority rises [36]. Eq. (6) is such a test. Constraint (C6) restricts the admissible orders, so the candidates for a level must be restricted too.

Definition 3 (Candidates). *Let* S *be the unassigned application tasks of core* p*. For* $\tau \in S$*, let* $\text{next}(\tau)$ *be the next task of the same chain and period on* p*, if any. The* candidates *for the lowest free level are* $\mathcal{C}(S) = \{\tau \in S : \text{next}(\tau) \notin S\}$*. A candidate is* feasible *if its WCRT with* $\text{hp} = S \setminus \{\tau\} \cup \{\text{LET}\}$ *is at most its period.*

Theorem 1 (Optimality). *If some priority order that satisfies (C6) makes core* p *schedulable, then Algorithm 1, with or without robust, returns a schedulable order that satisfies (C6).*

Proof. *(C6) holds by construction*: a task is chosen only when its successor in the pattern already has a lower level. *Optimality*, by induction on the levels. Invariant: before the algorithm fills level v, there is a schedulable order that satisfies (C6) and agrees with the levels already assigned (below v). It holds initially by assumption. Let O be such an order, S the unassigned tasks (the levels $2..v$ of O), and suppose the algorithm picks τ^* for level v. Build O' from O by moving τ^* to level v; the tasks of S that were below it in O move up one level. (i) τ^* is schedulable at level v: its higher-priority set is $S \setminus \{\tau^*\}$ plus the LET task, exactly the set under which the algorithm checked it, and Eq. (6) depends only on that set. (ii) A task that moved up loses τ^* from its higher-priority set, and every other task keeps its set, so no response time grows. (iii) (C6) still holds: τ^* is a candidate, so no later task of its pattern on p is in S, and the other tasks keep their relative order. So O' satisfies the invariant for level $v-1$. The invariant also shows that some candidate is feasible at every level: the task at level v in O is a candidate, by (C6), and it is schedulable there. Hence, if no candidate is feasible at some level, no admissible schedulable order exists. ∎

Algorithm 1: Priorities(p, *robust*): optimal priority assignment that respects precedence

```
Input  : tasks S_0 of core p (LET excluded); flag robust
Output : a priority level for every task; schedulable
S ← S_0
for v ← |S_0| + 1 to 2 do
    C ← {τ ∈ S : next(τ) ∉ S} in order of decreasing T_τ,
      ties by decreasing index
    best ← ⊥; bs ← −1; W ← C_p^LET + Σ_{τ∈S} C_τ
    foreach τ ∈ C do
        if (T_τ − W)/T_τ < bs then break                ▷ bound
        R ← WCRT of τ below S \ {τ} and LET (Lemmas 1
          and 2)
        if R > T_τ then continue
        if not robust then best ← τ; break
        if (T_τ − R)/T_τ > bs or (equal and τ later in its
          chain) then
            best ← τ; bs ← (T_τ − R)/T_τ
    if best = ⊥ then
        give the tasks of S the levels 2..v in precedence
          order
        return unschedulable
    level(best) ← v; S ← S \ {best}
LET task ← level 1 (highest)
return schedulable
```

Robust choice. Any feasible candidate preserves optimality, but not the same margin. With *robust*, the level goes to the feasible candidate with the largest relative slack $(T-R)/T$, after Davis and Burns [58]. The tightest tasks are thereby pushed to higher levels, which tends to increase the critical scaling factor (Section 6.5); this is a heuristic choice, not a guarantee that the factor is maximal. Because $R \geq W = \sum_S C$ (Lemma 1), $(T-W)/T$ bounds the slack of every candidate. Candidates are visited by decreasing period, so the bound decreases along the scan, and the scan stops as soon as it cannot beat the best slack found. A cheaper first-fit pass (*robust* off) gives the same verdict, so the robust pass runs only on schedulable cores.

Cost. The successor links come from one sort of the tasks by (chain, period, position), in $O(n \log n)$. The eligible tasks are kept as a bit set in candidate order: choosing a task clears its bit and sets the bit of its predecessor in the pattern. The

period groups and the sums W and U are updated when a task gets a level. A level then costs only the evaluation of its candidates, $O(c\,\iota\,q)$ for c evaluated candidates, ι iterations and q periods.

6.3 Network Calculus bounds

Arrival curves. Flow h sends one message of b_h bits per period T_h, copied out at a LET instant of the source core, and leaves when the copy ends. The source core has LET instants at the release of the producer and every $T_{\text{src}}^{\text{LET}}$ after it. With implicit communication, the message of a job leaves no earlier than $T_{\text{src}}^{\text{LET}}$ and no later than $\lceil R_{\text{prod}}/T_{\text{src}}^{\text{LET}}\rceil T_{\text{src}}^{\text{LET}} + C_{\text{src}}^{\text{LET}}$ after that release, so its release jitter is at most $R_{\text{prod}} + C_{\text{src}}^{\text{LET}} \le J_h = R_{\text{prod}} + T_{\text{src}}^{\text{LET}}$. With strict LET, it leaves between T_h and $T_h + C_{\text{src}}^{\text{LET}}$ after that release, so $J_h = C_{\text{src}}^{\text{LET}}$. At its first port the flow is constrained by the token bucket $\alpha_h(t) = \sigma_h + \rho_h t$, with $\rho_h = b_h/T_h$ and $\sigma_h = b_h + \rho_h J_h$ [46]. Both use $R_{\text{prod}} \le T_h$: the WCRT of Eq. (6) is only valid up to the period, and a producer that misses it would need a busy-period analysis [71]. Such a flow is given no arrival curve instead: every port it crosses, and every port these feed, is declared unbounded, so no bound depends on it.

Port bound. Let $F(e,x)$ be the flows of class x through port e and $\sigma_h(e)$ the burst of flow h when it reaches e. If the port gives class x a rate-latency service with rate $R_{e,x}$ and latency $L_{e,x}$, if $\sum_{F(e,x)} \rho_h < R_{e,x}$, and if the LET load of the link is within its design limit, then the FIFO aggregate of the class has the delay bound [46]

$$d_{e,x} = L_{e,x} + \frac{1}{R_{e,x}} \sum_{h \in F(e,x)} \sigma_h(e). \tag{7}$$

Otherwise the port is *overloaded* and its flows have no bound. The three shapers differ by their service curves and by how bursts evolve along the route.

CBS service. At a port of rate r with classes $0..X-1$ in decreasing priority, class x with idle slope I_x receives the rate-latency service $R = I_x$, $L = L_x$ with [25, 26]

$$L_x = \frac{c_x^{\max}}{I_x} + \frac{(r - I_x)\, l_x}{r\, I_x}, \tag{8}$$

$$c_x^{\max} = \frac{I_x\big(r\,\bar{l}_x + \sum_{j<x}(r - I_j)\, l_j\big)}{r\,\big(r - \sum_{j<x} I_j\big)}, \tag{9}$$

where l_j is the largest frame of class j at the port and $\bar{l}_x = \max(L_{BE}, \max_{j>x} l_j)$ the largest frame that can block class x (non-preemptively). With frame preemption, L_{BE} drops to 143 bytes. The first term of Eq. (8) is the time to recover the largest credit. The second accounts for a frame of the class being sent at the link rate rather than at the idle slope [24].

Idle slopes. With one class, I_0 is a fixed share of r (75 %). With several classes, fixed shares waste bandwidth where a class carries little traffic, so the slopes are set per port. Let $\rho_x = \sum \rho_h$ over the flows of class x at the port, $\beta_x = \sum b_h$ their bits, $T_x^{\min}$ their shortest period, and $B = r \sum_x \text{share}_x$ the budget. Then

$$I_x = \rho_x + \Big(B - \sum_j \rho_j\Big) \frac{\beta_x/T_x^{\min}}{\sum_j \beta_j/T_j^{\min}}, \tag{10}$$

if $\sum_j \rho_j \le B$; otherwise the port is overloaded. Each class gets its own rate, plus a part of the spare budget in proportion to its burstiness per unit of time, because its delay is about its bursts over its slope (Eq. (7)).

TAS service. With an exclusive window of length w_x per gate cycle T_c, and guard bands that keep other frames out of the window, class x is served at the link rate for $\hat{w}_x = w_x - l_x/r$ per cycle, because a frame starts only if it ends within the window. A rate-latency lower bound is $R = r\hat{w}_x/T_c$, $L = T_c - \hat{w}_x$ [27]. If $\hat{w}_x \le 0$ the class has no service.

ATS service. With ATS the classes are served in strict priority, without preemption. Class x sees the higher classes through their arrival curves, so it receives the residual rate-latency service [46]

$$R_{e,x} = r - \sum_{j<x} \rho_j, \qquad L_{e,x} = \frac{\bar{l}_x + \sum_{j<x} \sum_{h \in F(e,j)} \sigma_h(e)}{R_{e,x}}, \tag{11}$$

where $\bar{l}_x$ is the largest frame of a lower class or of best effort.

Burst propagation. A flow that leaves a port with delay bound d has the output burst $\sigma + \rho d$. With CBS and TAS, along the route $e_1, \ldots, e_n$ of flow h,

$$\sigma_h(e_i) = b_h + \rho_h\Big(J_h + \sum_{j<i} d_{e_j,x}\Big). \tag{12}$$

With ATS, the regulators of every switch hold the frames of each flow until they conform again to α_h. Behind a FIFO system, such regulators do not increase the worst-case delay of the system plus regulator ("shaping for free", [28], for both interleaved and per-flow regulators). Every port of the route therefore sees the source burst, $\sigma_h(e_i) = b_h + \rho_h J_h$, and Eq. (7) with Eq. (11) bounds the delay through the port and the regulator that follows it. The end-to-end bound of a flow is, in all cases,

$$d_h = \sum_{i=1}^{n} d_{e_i,x} + (n-1)\delta. \tag{13}$$

Frame replication. A replicated flow h has two routes, P_1 and P_2, with the same first and last ports (Figure 4). Both member streams load the ports of their own route. For route m, let $\bar{D}_m$ be the delay bound from the source to the elimination: all its ports but the last, and its $n_m - 1$ switches; and let $\underline{d}_m = (n_m - 1)\delta$, a lower bound of the same delay.

Proposition 2 (Bound with replication). *After the elimination, flow h is constrained by the token bucket of rate ρ_h and burst*

$$\sigma_h(e_n) = b_h + \rho_h\Big(J_h + \max_m \bar{D}_m - \min_m \underline{d}_m\Big), \tag{14}$$

Algorithm 2: Network($\{h\}$): bounds of all flows

Input : flows with route(s), class, b_h, T_h, J_h
Output : d_h for every flow, or *unbounded*

compute ρ_x, l_x, slopes (Eq. (10)) and services at every port
$d_{e,x} \leftarrow 0$, $ok_{e,x} \leftarrow$ true for all ports
repeat
  foreach *port* (e, x) do
    $\Sigma_{e,x} \leftarrow 0$; $fed_{e,x} \leftarrow$ true
  foreach *flow* h, *each member route* $e_1..e_n$ do
    $\sigma \leftarrow b_h + \rho_h J_h$; $b \leftarrow$ true
    for $i \leftarrow 1$ to n *(to* $n-1$ *if replicated)* do
      if $i > 1$ ***or*** *first member route* then ▷ *a shared* e_1 *counts once*
        $\Sigma_{e_i,x}$ += σ; if ***not*** b then $fed_{e_i,x} \leftarrow$ false
      if ***not*** $ok_{e_i,x}$ then $b \leftarrow$ false
      else if *CBS or TAS* then σ += $\rho_h d_{e_i,x}$
  foreach *replicated flow* h do
    $\Sigma_{e_n,x}$ += Eq. (14)
  foreach *port* (e, x) do
    $ok_{e,x} \leftarrow fed_{e,x} \wedge \sum \rho < R_{e,x} \wedge$ load within limit
    $d_{e,x} \leftarrow L_{e,x} + \Sigma_{e,x}/R_{e,x}$ if $ok_{e,x}$ and $\le 10^9$, else unbounded
until *no* $d_{e,x}$ *or* $ok_{e,x}$ *changed, or* 1000 *iterations*
if *not converged* then all ports unbounded
foreach *flow* h do
  $d_h \leftarrow$ Eq. (13) (the larger route if replicated) if all its ports are bounded, else unbounded

and its delay is at most $d_h = \max_m(\bar{D}_m + d_{e_n,x})$*, also after the failure of one route.*

Proof. Let message k be released at $t_k \in [kT_h, kT_h + J_h]$. Each copy that reaches the elimination does so within $[t_k + \underline{d}_m, t_k + \bar{D}_m]$ for its route m, and the elimination forwards the first one. The forwarded message k therefore leaves the elimination within $[kT_h + \min_m \underline{d}_m,\ kT_h + J_h + \max_m \bar{D}_m]$, a window of length $J_h + \max_m \bar{D}_m - \min_m \underline{d}_m$. In any interval of length t, at most $1 + (t + J_h + \max \bar{D}_m - \min \underline{d}_m)/T_h$ messages can leave, of at most b_h bits each, which gives Eq. (14). This is corollary 1 of Thomas et al. [64] for one replication-elimination pair. The forwarded copy is never later than the copy of the slower route, which gives the delay bound. After a failure, fewer frames cross the ports, and every bound above is monotone in the traffic, so the bounds still hold. ∎

With ATS, the regulators of the destination switch act on each member stream *before* the elimination, and no regulator follows it: an interleaved regulator placed after an elimination loses the shaping-for-free property [64]. The member streams therefore keep the source bursts on their own routes, and only the last port sees the burst of Eq. (14).

Fixed point. Eqs. (7), (12) and (14) define the delays recursively, and Algorithm 2 computes them by iteration from $d \equiv 0$. Larger port delays give larger bursts, hence larger port delays, so the map is monotone and the iterates never decrease. On the star and the tree the routes are feed-forward, and the iteration settles after at most as many passes as there are ports on the longest route. On the ring, the clockwise routes of different flows can form cycles of dependencies. The iteration then computes the least fixed point of the burst equations, as in the fixed-point method for cyclic networks [46]. This fixed point is a valid bound: in a network that starts empty and whose ports each serve more than the sum of the rates of their flows, every finite nonnegative fixed point of the burst equations bounds the bursts [72, Thm. 2]. The first condition holds at start-up, and the second is the check of the sustained rate (Eq. (7)); a port that fails it is unbounded. A port whose bound exceeds 10^9 µs is declared unbounded, and if the iteration does not settle within 1000 passes, all ports are. A port whose flows arrive through an unbounded port is unbounded too (the *fed* flag). With ATS, bursts do not depend on the delays (except after an elimination, at a last port that feeds no other port), so the cycles disappear and two or three passes suffice. The bounds are compared with a frame-level simulation in Section 11.2.

6.4 End-to-end bounds of a task-chain

The mapped task-chain is a sequence of segments $S_1, \dots, S_q$ separated by communications $h_1, \dots, h_{q-1}$, inside an ECU or between ECUs (Section 4.6). Inside a segment, a *run* is a maximal sequence of consecutive tasks with the same period. Let runs(ξ) be the runs of all segments, reg(ξ) the register boundaries inside segments (consecutive tasks of different periods on the same core), cons($\cdot$) the task right after a boundary or a communication, and src(h) the source core of h. The holistic sum of Section 4.6 is

$$\Lambda_\xi = \sum_{r \in \text{runs}(\xi)} R_{\text{last}(r)} + \sum_{h \text{ inter-ECU}} d_h, \tag{15}$$

and the copy-out delay of a communication h, counted from the end of the WCRT of its producer, is

$$w_h = \begin{cases} T^{\text{LET}}_{\text{src}(h)} + C^{\text{LET}}_{\text{src}(h)} & \text{implicit,} \\ T_h - R_{\text{prod}(h)} + C^{\text{LET}}_{\text{src}(h)} & \text{strict LET.} \end{cases} \tag{16}$$

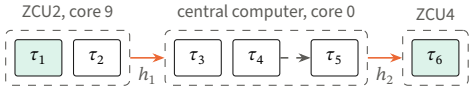
Proposition 3 (End-to-end bounds). *If every task of* ξ *meets its deadline and every inter-ECU flow has a bound* d_h*, then*

$$\Gamma_\xi \le \Gamma^{\text{b}}_\xi = \Lambda_\xi + \sum_{b \in \text{reg}(\xi)} T_{\text{cons}(b)} + \sum_h \big(w_h + T_{\text{cons}(h)}\big), \tag{17}$$

$$\Delta_\xi \le \Delta^{\text{b}}_\xi = T_{\tau_1} + \Gamma^{\text{b}}_\xi, \tag{18}$$

where the last sum of Eq. (17) runs over all communications, inside or between ECUs.

Proof. Follow a sample from the release of the job of τ_1 that takes it. *Runs.* The tasks of a run are released together and, by (C6), run in chain order. The data of a run is therefore



runs: $\tau_1\tau_2$, $\tau_3\tau_4$, τ_5, τ_6; $\Lambda = R_2 + R_4 + R_5 + R_6 + d_{h_1} + d_{h_2}$
$\Delta^{\mathrm{b}} = T_1 + \Lambda + T_5 + (w_{h_1} + T_3) + (w_{h_2} + T_6)$

Figure 5. A task-chain mapped as three segments: two inter-ECU communications, and a register between two patterns on the central computer (dashed arrow). Λ is the holistic sum of Eq. (15), Δ^{b} the bound of Eq. (18) and w_h the copy-out delay of Eq. (16).

complete when the job of its last task ends, at most R_{last} after the common release. *Registers.* The first consumer job that reads a value written at time t is released at the first consumer release after t, at most T_{cons} later. *Communications.* With implicit communication, an output completed at t is copied at the next LET instant of the source core, at most $T_{\mathrm{src}}^{\mathrm{LET}}$ later. With strict LET, the output of a producer job released at r is published at $r + T_h$, a LET instant of the source core since $T_{\mathrm{src}}^{\mathrm{LET}}$ divides T_h; the job has ended by then, since $R_{\mathrm{prod}} \le T_h$, and $r + T_h$ is $T_h - R_{\mathrm{prod}}$ after the bound $r + R_{\mathrm{prod}}$ counted by the run. The LET task has the highest priority, so the copy ends at most $C_{\mathrm{src}}^{\mathrm{LET}}$ after the instant: w_h bounds the copy-out delay in both cases. Over the network the message then takes at most d_h; its release jitter is at most J_h, as assumed in its arrival curve. The consumer reads the message at its first release after the arrival, at most T_{cons} later. This release is a LET instant of the consumer core, since T^{LET} divides every period of the core, so the LET task copies the message in at that instant, before the consumer runs. The copy-in is part of the response time of the consumer, because the LET task interferes with it. Summing the stages along the chain gives Eq. (17). An external event waits at most T_{τ_1} for the next sample, which gives Eq. (18). ∎

The bounds use only quantities of the mapping. Δ_ξ^{b} is the quantity compared with the deadline D_ξ. It bounds the maximum data age too, since the two metrics are equal [10]. Figure 5 shows a task-chain with two inter-ECU communications and one register.

6.5 Sensitivity: the critical scaling factor

The margin of a core is its critical scaling factor [37, 59]: the largest λ such that the core stays schedulable, with its priorities, when every application WCET C becomes λC (the LET task unchanged). It is found by bisection. λ starts at 1 and doubles while the core is schedulable; the bracket is then halved until it is within 0.1 %. On a core that is not schedulable, the bisection runs between 0 and 1, so $\lambda < 1$. To make small factors meaningful, the analysis runs in nanoseconds, so that a 3 µs WCET scaled by 1.05 becomes 3150 ns rather than staying at 3 µs. The factor of a mapping, $\lambda(\pi)$, is the smallest over the cores it uses, since the first core to fail limits the system; it is below 1 exactly when a core misses a deadline. A core loaded to φ typically has $\lambda \approx 1/\varphi$.

6.6 A lower bound on the number of cores

For a sub-chain f and an ECU $k \in A_f$, let $u_f(k) = \min_{\vartheta \in \Theta(k) \cap \Theta_f} \sum_{\tau \in f} U_\tau^{\mathrm{iso}}(\vartheta)$ be its utilization on the cheapest core of k it may use, in isolation (Eq. (1)). Let $\mathcal{F}_k$ be the sub-chains pinned to k and $\mathcal{F}_{\mathrm{free}}$ the others.

Proposition 4 (Lower bound). *For the sub-chains $\mathcal{F}$, every admissible mapping uses at least*

$$M_{\mathrm{LB}} = \max\Big(\sum_{k \in \mathcal{K}} \left\lceil \frac{U_k}{\varphi} \right\rceil, \left\lceil \frac{1}{\varphi} \Big(\sum_k U_k + \sum_{f \in \mathcal{F}_{\mathrm{free}}} \min_{k \in A_f} u_f(k) \Big) \right\rceil \Big) \quad (19)$$

cores, where $U_k = \sum_{f \in \mathcal{F}_k} u_f(k)$.

Proof. Every core offers at most φ of its own capacity (C3). A sub-chain placed on a core of k takes at least $u_f(k)$ of it, since the LET load and the contention are non-negative. The pinned sub-chains of k alone need at least $\lceil U_k/\varphi \rceil$ cores of k, which gives the first term. All sub-chains together need a total capacity of at least the numerator of the second term, which gives the second. Constraints (C4)–(C7) only remove mappings, so the bound holds for every admissible mapping. ∎

The bound ignores the LET load, the schedulability gaps of non-harmonic periods and the integrality of each zone, so it is not tight. The exact formulation of Section 8 gives the true optimum on smaller instances.

7 The mapping heuristic

In brief. A design point runs seven steps (Figure 6): partitioning into sub-chains, allocation by bin packing or by a communication-aware rule, core consolidation, priority assignment, relocation moves for unschedulable cores, a deadline-driven local search, and the final analysis. Contention is handled by a fixed point on the number of cores of each ECU, and an outer loop can size the platform for a target margin. Every step is deterministic.

A *design point* is one run of the method for one application and one set of parameters (Table 5). The decomposition follows the objectives. Partitioning, allocation and consolidation decide the cores (O1). Priority assignment and the moves of step 5 make every core schedulable (C7); step 5 adds no core to the platform, but may put an unused core back into use, so it can raise M. The local search of step 6 improves (O2) without changing M.

Preparation. With a WCET margin of m %, every reference WCET first becomes $\min(\lceil C^{\mathrm{ref}}(100+m)/100 \rceil, T-1)$, and the platform is sized for the larger WCETs. Section 7.9 offers an alternative that sizes for the margin directly. With contention, the tasks without a request count get $a_\tau = \lceil \eta C_\tau^{\mathrm{ref}}/1000 \rceil$ before the margin, which inflates execution, not memory traffic.

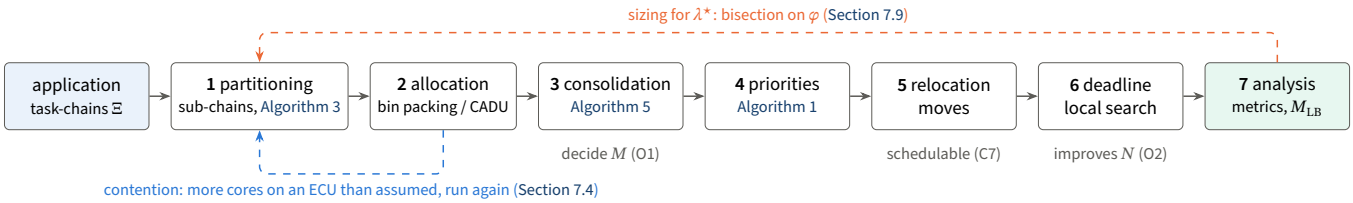


Figure 6. The steps of a design point. The network analysis (Algorithm 2) runs inside steps 6 and 7. The two loops are optional: sizing for a margin, and the fixed point of the contention.

Table 5. Parameters of a design point and their defaults.

parameter	default	meaning
rule	FFDU	allocation rule: BFDU, WFDU, FFDU, CADU
σ	0.4	utilization limit of a sub-chain, on the core type it will get
φ	0.8	utilization cap of a core
central cores	8 big (100 %)	+ little cores (40 %), none by default
zone cores	4 MCU (50 %)	per zone; + lock-step cores, none
c_{LET}	50 µs	LET copy on the reference core
M_k	unlimited	memory of each ECU
replicas	ECUs	separation policy (or cores)
affinity	on	try the cores of the task-chain first
consolidation	on	step 3 (not with WFDU)
relocation	on	step 5
swap moves	off	step 5, before relocation (Section 7.6)
local search	on	step 6
ℓ, η	0, 1000	memory latency (ns); requests per ms of WCET
network	star, 1 Gbit/s	topology, R; $R_b = R$
shaping	CBS, 1 class	idle slope 75 %, no preemption; or TAS, ATS
FRER	off	ring or custom topology: replicated task-chains
margin	0 %	WCETs increased before the mapping
$\lambda^\star$	none	sizing target (Section 7.9)

Algorithm 3: Partition(ξ, σ)

Input : task-chain $\xi = (\tau_1..\tau_n)$, limit σ
Output : sub-chains of ξ, in chain order

$F \leftarrow \langle\rangle$ ▷ *the open sub-chain*
for $i \leftarrow 1$ to n do
  if $F \neq \langle\rangle$ ***and*** $(A_{\tau_i}, \Theta_{\tau_i}, binding) \neq$ *those of* F then
    close F ▷ *forced cut*
  $g \leftarrow F \neq \langle\rangle \wedge g_{\mathrm{last}(F)}$ ▷ *glued to* F
  $\bar{u} \leftarrow (\sigma - 0.01)\, s_{\hat{\theta}}/100$, $\hat{\theta}$ the type F will get
  if $U^{\mathrm{ref}}_{\tau_i} > \bar{u}$ ***and not*** g then
    close F; $F \leftarrow \langle\tau_i\rangle$
    if ***not*** g_{τ_i} then close F
    continue
  append τ_i to F
  if g then **continue**
  if $|F| > 1$ ***and*** $(U^{\mathrm{ref}}(F) > \bar{u}$ ***or*** F *alone on a core is unschedulable)* then
    remove τ_i; close F; $i \leftarrow i - 1$
close F

7.1 Step 1: partitioning

Partitioning cuts each task-chain into sub-chains, the units of allocation. Fewer sub-chains mean fewer communications; smaller sub-chains pack better. A sub-chain must not be larger than what a core can take next to others, so its utilization on the core type it will run on is limited to σ (minus 1 % kept for the LET task): on a core of speed s, that is a utilization of $(\sigma - 0.01)\, s/100$ of the reference core, as Algorithm 3 computes. Its tasks must share their binding and their allowed sets: a sensor driver never drags the computation into its zone. Tasks that must be co-located (C2) are never separated, since they have the same constraints (Section 5). Algorithm 3 gives the procedure. A sub-chain of free tasks is sized for the core it is most likely to get: the first host (Section 4.5) it may run on, else the first other ECU it may run on, with the first type of that ECU it allows, in the order big, little, MCU, lock-step. The *fragmentation rate* is the number of cuts over the number of possible cuts (one per pair of consecutive tasks).

The schedulability test "alone on a core" in Algorithm 3 uses chain order inside a pattern and rate-monotonic order between patterns, with Eq. (6). The sub-chains of all task-chains are finally sorted by decreasing reference utilization (stable sort), the order in which they are allocated.

7.2 Step 2: allocation

Sub-chains are placed one by one in that order (Algorithm 4). The load a sub-chain adds to a core is its utilization at the speed and type of the core (Eq. (2)), plus the change of the LET load: T^{LET} becomes the GCD of the old and new periods (Eq. (3)). A core *can take* f if (C1), (C4) and (C5) hold and the new load is within φ (C3).

Tiers. Choose(f, *tier*) considers the cores that can take f, restricted by the tier:

- SIBLINGS: cores that already run another sub-chain of the same task-chain (affinity; it saves a communication);
- PREFERRED: for a free sub-chain that may run on the central computer, its cores only (except with CADU); otherwise all allowed cores;
- ANY: all allowed cores.

In every tier, for a free sub-chain that may run on the central computer, the zone cores *added* during the allocation are excluded: they were added for the pinned sub-chains of their zone, and filling them with free work would force yet more zone cores. Here and below, "the central computer" stands for the hosts of Section 4.5 (the central computers, or, on a platform without one, the ECUs with the fastest cores), and "zone cores" for the cores of the other ECUs.

Rules. Among the candidates, the rule picks:

Algorithm 4: Allocate($\mathcal{F}$)

Input : sub-chains by decreasing utilization; initial cores $\mathcal{P}_0$
Output : π, possibly with added cores, or *infeasible*

$\mathcal{P} \leftarrow \mathcal{P}_0$; $\mathcal{R}_k \leftarrow \sum_{f \in \mathcal{F}_k} \mu_f$ for all k ▷ *reserved memory*
foreach $f \in \mathcal{F}$ do
  for *attempt* $\leftarrow$ 1 to 2 do
    $p \leftarrow \perp$
    if *affinity* then $p \leftarrow$ Choose(f, SIBLINGS)
    if $p = \perp$ then $p \leftarrow$ Choose(f, PREFERRED)
    if $p = \perp$ then $p \leftarrow$ Choose(f, ANY)
    if $p \neq \perp$ then $\pi(f) \leftarrow p$; **break**
    if *attempt* $= 2$ ***or*** *fixed hardware* then return *infeasible*
    $k \leftarrow$ first ECU (hosts first) in A_f whose memory, replicas and $M_k^{\max}$ admit f
    if *none* then return *infeasible*
    add a core to k of the added type of k, or, if f may not use it, of the type in $\Theta(k) \cap \Theta_f$ on which f is lightest
  if f *pinned to* k then $\mathcal{R}_k \leftarrow \mathcal{R}_k - \mu_f$

- BFDU (best fit decreasing utilization): the core left with the least room;
- WFDU (worst fit): the core left with the most room;
- FFDU (first fit): the first core in creation order;
- CADU (communication-aware, new): the core that is lexicographically best for (i) the number of inter-ECU communications the placement creates with the neighbouring sub-chains of the task-chain, (ii) the number of communications between cores of one ECU it creates, (iii) being a core of the central computer, and (iv) the least room left.

For CADU, the neighbours of f are the previous and next sub-chains of its task-chain. A neighbour already placed on core q creates no communication if $q = p$, a communication inside the ECU if $\kappa(q) = \kappa(p)$, and an inter-ECU one otherwise. A neighbour not yet placed but pinned to ECU $k' \neq \kappa(p)$ creates an inter-ECU communication. Bin packing alone sends a free sub-chain to the central computer whenever a core there can take it, unless affinity first finds a core that runs its own task-chain. CADU instead keeps a computation next to the sensor or actuator it serves, when the zone controller has room for it.

Memory reservation. A zone controller with a memory limit could see its memory taken by free sub-chains placed early, and then fail to host its own sensor and actuator drivers, which can run nowhere else. The allocation therefore reserves, on every ECU k, the memory $\mathcal{R}_k$ of the pinned sub-chains not yet placed. A free sub-chain may use only $M_k - \mathcal{R}_k$, minus what is already used.

Proposition 5 (Memory). *If $\sum_{f \in \mathcal{F}_k} \mu_f \leq M_k$ for every ECU k, no pinned sub-chain is ever refused by (C4) during Algorithm 4.*

Algorithm 5: Consolidate(π)

repeat
  foreach *core p in use, by increasing load* do
    $\pi' \leftarrow \pi$
    foreach *sub-chain f of p, by decreasing utilization* do
      $Q \leftarrow$ cores $q \neq p$ in use that can take f in π'
      sort Q by (inter-ECU communications of f on q, room left)
      $q \leftarrow$ first $q \in Q$ schedulable with f (first-fit priorities)
      if *none* then **next** p ▷ *keep π*
      move f from p to q in π'
    $\pi \leftarrow \pi'$ ▷ *p is empty*
    restart
until *no core was emptied*

Proof. Let u_k be the memory used on k. The invariant $u_k + \mathcal{R}_k \leq M_k$ holds initially. Placing a pinned sub-chain on k moves μ_f from $\mathcal{R}_k$ to u_k; a free sub-chain is admitted only if $u_k + \mu_f + \mathcal{R}_k \leq M_k$. Both keep the invariant. When a pinned sub-chain f of k is placed, $\mu_f \leq \mathcal{R}_k$, so $u_k + \mu_f \leq M_k$. ■

7.3 Step 3: core consolidation

Decreasing-utilization bin packing places each sub-chain once and never revisits it. A core opened early for a large sub-chain can stay lightly loaded, while the smaller sub-chains allocated after it fill the gaps that the pinned work leaves on other cores. The exact formulation of Section 8 showed that this costs one to three cores on small and constrained instances. Consolidation (Algorithm 5) removes this loss. It takes the cores in use from the lightest, and tries to move all their sub-chains to other cores in use. A move must respect (C1)–(C5) and the cap φ (and, as in the allocation, a free sub-chain does not go to a zone core added during the allocation), and the target core must stay schedulable with the first-fit pass of Algorithm 1. Among the targets, the sub-chain goes to the one that leaves the fewest inter-ECU communications with its neighbours in the task-chain, then to the fullest, so that consolidation does not undo the work of affinity and of CADU. The moves are kept only if the whole core is emptied; otherwise the core is left as it was. Consolidation is not applied to WFDU, which spreads the load on purpose.

Each kept round empties one core, so there are at most M rounds, and every move keeps (C1)–(C5) and (C3). Schedulability is checked on every target with the optimal test of Theorem 1, so consolidation never makes a schedulable core unschedulable.

7.4 Contention: the number of competing cores

With contention, the WCETs depend on the number n_k of cores of ECU k that compete for its memory, and n_k depends on the allocation. The method resolves this circularity as follows.

1. The allocation starts with n_k equal to the number of cores of k in the architecture. A core added to k during the allocation gets the current count.
2. If the allocation took more cores on some ECU than assumed, the design point is run again from step 1 with that many. More cores mean more contention, longer WCETs and possibly more cores, so this is a fixed point. It stops when the counts no longer grow, after at most eight runs; otherwise the design point is declared infeasible.
3. After step 6, only the cores that run tasks compete: n_k becomes the number of cores of k in use. The WCETs can only decrease, and the response times are computed again with the same priorities.

Proposition 6 (Contention). *In the final mapping, every WCET is at least the one of Eq. (2) with n_k the number of cores of k that run tasks, and every core declared schedulable is schedulable with those WCETs.*

Proof. At the end of the fixed point, the count assumed for every ECU is at least the number of its cores, so every WCET used by steps 2 to 6 is at least the final one. Item 3 of the list above recomputes the WCETs with the number of cores in use, which no later step changes, and recomputes the response times with the final priorities. The response time of Eq. (6) is monotone in the WCETs. ■

7.5 Step 4: priorities

Every core with tasks gets its priorities from Algorithm 1 with the robust choice. Bin packing only checks loads, so a core within φ can still be unschedulable: the automotive periods are not harmonic (2 ms and 5 ms do not divide each other). Such cores are repaired in step 5.

7.6 Step 5: relocation moves

This step moves sub-chains away from unschedulable cores, without adding cores to the platform. The sub-chains of the unschedulable cores are visited one at a time, the cores in their order and the sub-chains in the order of their core. A sub-chain f whose core has become schedulable is skipped. Otherwise f moves to the core that (i) satisfies (C1), (C4), (C5) and the cap φ with it, (ii) is schedulable, (iii) remains schedulable with f after Algorithm 1, and (iv) keeps the most room. Cores without tasks (initial cores not used yet, or cores emptied by step 3) are tried only when no loaded core qualifies, which is how this step can raise M. The source core then gets new priorities.

Remark 1 (Swap moves). An earlier version of this step first exchanged pairs of sub-chains between two unschedulable cores, and kept an exchange that made one of them schedulable. The tool still offers these swap moves as an option, off by default. Of the 9600 design points of Section 11, 9516 have an admissible mapping, and 84 of these have unschedulable cores after step 4. The relocation moves, where enabled, made all of them schedulable. In the variant of the ablation study with swap moves (Section 11.10), the swap moves made none of these cores schedulable.

Algorithm 6: LocalSearch(π)

```
S ← AnalysisInit(π); N ← N(S)
for pass ← 1 to 8 do
    kept ← false
    foreach task-chain ξ schedulable but late in S do
        foreach communication h of ξ, move μ ∈ {pull back, push on} do
            if μ violates (C1)–(C5), or the target is unschedulable with first-fit priorities then
                continue
            apply μ (cores p, q changed); n ← Update(S, p, q)
            if n > N then robust priorities on the target; n ← Update(S, q)
            if n > N then
                N ← n; kept ← true; restart with the same ξ
            else
                undo μ; Update(S, p, q)
    if not kept then break
```

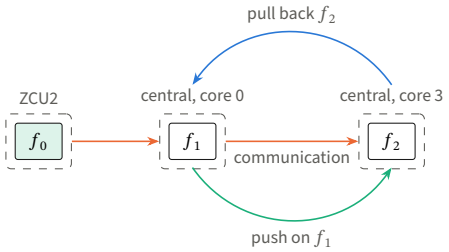


Figure 7. The two moves tried around a communication. Around the communication from ZCU2, only the pull back of f_1 exists: f_0 is pinned to its zone.

7.7 Step 6: deadline-driven local search

A task-chain whose tasks are all schedulable can still miss its end-to-end deadline, because every communication costs LET sampling delays and, between ECUs, a network transfer. A task-chain can also cross an overloaded port. The local search removes communications of such task-chains (Algorithm 6). Around each communication it tries two moves (Figure 7): *pull back* moves the first sub-chain after the communication onto the core before it, and *push on* moves the last sub-chain before the communication onto the core after it. A move must respect (C1)–(C5), leave the target core schedulable, and leave the source core schedulable (or no worse). The source keeps its priorities, because removing tasks cannot increase a response time. A move is *kept* only if the number of task-chains that meet their deadline, over the whole application, strictly increases. The count must be global: a move changes the response times of two cores and the loads of the ports, and hence other task-chains. Trials are screened with first-fit priorities on the target; a promising move gets the robust priorities and is counted again.

Incremental analysis. Counting the task-chains that meet

their deadline from scratch after every trial costs the whole communication analysis: collecting and sorting all sub-chains, rebuilding all segments and analysing the network. The local search instead keeps a state $\mathcal{S}$: the descriptor of every sub-chain (its core, ECU, the response times that Eqs. (17) and (18) need, and the LET parameters of its core), the segments of every task-chain, and the summary. Update($\mathcal{S}, p, q$) recomputes the descriptors of the sub-chains now on p and q, and rebuilds only the segments of their task-chains. It then reruns Algorithm 2 on all flows, whose cost is linear in the number of flows, and sums the bounds again.

Proposition 7 (Exactness). *After any sequence of moves, each followed by Update on the cores it changed, $\mathcal{S}$ equals the result of a full analysis of the current mapping.*

Proof. A sub-chain descriptor depends only on the core that holds the sub-chain (the response times of its tasks and the LET task of the core). A move changes only the two cores it touches, so the descriptors of the sub-chains on all other cores are unchanged, and the sub-chains that changed core are now on p or q. The segments of a task-chain depend only on the descriptors of its sub-chains; the network analysis and the sums are recomputed from all segments. The full analysis is a deterministic function of the same data, in the same order. ∎

Undoing a rejected move is a move too, and is followed by Update. The regression tests check Proposition 7 after random moves.

7.8 Step 7: analysis and metrics

Finally, the segments of all task-chains are built, the flows are analysed together (Algorithm 2), and the bounds of Proposition 3 are computed. The mapping is verified: no core above φ, and priorities in precedence order within every pattern. The method also computes the critical scaling factor of every core (Section 6.5) and the lower bound M_{LB} (Proposition 4).

What a design point guarantees

Guaranteed, under the model of Section 4: every mapping returned meets (C1)–(C6); every core either meets (C7) or is reported as unschedulable; every response time (Eq. (6)) reported within its period, every port delay (Section 6.3), and every end-to-end delay and maximum reaction time (Proposition 3) reported is an upper bound; a task above its period is reported without a response time, and a port loaded above its reserved rate, or crossed by a flow of such a task, is reported as giving no bound, and so is every task-chain that depends on them; M_{LB} is a lower bound on the cores of any mapping of the same sub-chains (Proposition 4). **Not guaranteed**: that M is optimal (Section 8 measures the gap); that every deadline is met, since deadlines are an objective (O2) and the late task-chains are reported; and anything outside the model: WCETs, request counts and message sizes are inputs, and the bounds across ECUs assume clocks that run at the same rate, though not with aligned phases (Section 12).

7.9 Sizing for a margin

A WCET margin at design time inflates every WCET and hopes that the resulting mapping keeps enough slack. The sizing variant of the problem asks for $\lambda(\pi) \geq \lambda^\star$ directly. A lower utilization cap leaves more room on every core and, in general, a larger scaling factor. The method therefore searches for the largest φ that reaches the target. It runs the design point at the given φ_0 and stops if $\lambda \geq \lambda^\star$. Otherwise it runs at $\varphi = \sigma$, the smallest value at which sub-chains still fit; if that run misses the target too, it is returned with the flag *target missed*. Otherwise, seven bisection steps on $[\sigma, \varphi_0)$ keep the largest φ that reached the target, so at most nine design points are run. The scaling factor is not exactly monotone in φ, because the mapping changes with it, so the result is the best φ found, not a guaranteed maximum. A run reaches the target when its scaling factor is at least $\lambda^\star(1 - 10^{-3})$ (a tolerance for floating point) and every core is schedulable.

7.10 Complexity

Let n be the number of tasks, N of sub-chains and P of cores. Partitioning is linear in n times the cost of a response-time analysis of one sub-chain. Allocation evaluates at most P cores per sub-chain and tier, $O(NP)$. A round of consolidation tries at most P cores, each costing a copy of the cores and one priority assignment per moved sub-chain and target. Relocation tries at most P targets for each sub-chain of an unschedulable core, a set that is empty in most runs at $\varphi \leq 0.8$. The local search tries $O(H)$ moves per pass for H communications of late task-chains, each costing two priority assignments and one incremental update, linear in the flows. Section 11.13 reports the measured growth.

8 Exact formulation

In brief. To measure how far the heuristic is from the optimum of (O1), the allocation of the sub-chains is also solved exactly. A constraint program chooses the cores and the LET periods; schedulability is added lazily, as cuts derived from unschedulable cores. Every optimal allocation it returns is checked again by the implementation of the heuristic.

Scope. The exact model takes the sub-chains of step 1 as given, since partitioning does not depend on the allocation rule, and solves the rest of (O1): the allocation, the LET periods and the schedulability of every core, under (C1)–(C7). It uses the same WCETs, utilizations (in parts per million of a core, rounded up) and LET tasks as the heuristic, so the two are compared on exactly the same problem. Contention is left out, as it would make the WCETs depend on the number of cores per ECU. The candidate cores of an ECU are its initial cores, then cores of its added type up to two more

than the larger of its initial count and the largest number any heuristic uses there (within $M_k^{\max}$). The optimum is the optimum over these candidates. A solution that uses every candidate of an ECU is reported as possibly restricted; this happened once (Section 11.3). To check that two spare cores suffice, all 240 instances of Section 11.3 were solved again with four: no allocation with fewer cores was found on any of them, none was restricted, and every optimum proven in both runs was the same.

8.1 Variables and constraints

For every sub-chain f and allowed candidate core p (by (C1)), a Boolean $x_{f,p}$ says that f runs on p, and u_p that p is used. Let $\mathcal{G}$ be the closure under GCD of the GCDs g_f of the periods of the sub-chains; for every core, a Boolean $y_{p,g}$, $g \in \mathcal{G}$, chooses the period of its LET task. With $U_{f,p}$ the utilization of f on p and C_p^{LET} the WCET of the LET task:

$$\sum_p x_{f,p} = 1, \qquad x_{f,p} \le u_p \qquad \forall f, p \qquad (20)$$

$$\sum_{g\in\mathcal{G}} y_{p,g} = u_p, \qquad x_{f,p} \le \sum_{g \mid g_f} y_{p,g} \qquad \forall f, p \qquad (21)$$

$$\sum_f U_{f,p}\, x_{f,p} + \sum_g \left\lceil \frac{C_p^{\mathrm{LET}}}{g} \right\rceil y_{p,g} \le \varphi\, u_p \qquad \forall p \qquad (22)$$

$$\sum_{f,\, p\in k} \mu_f\, x_{f,p} \le M_k \qquad \forall k \qquad (23)$$

$$\sum_{\xi\in\varrho} z_{\xi,k} \le 1, \quad x_{f,p} \le z_{\xi(f),\kappa(p)} \qquad \forall \varrho, k \qquad (24)$$

Eq. (21) lets a sub-chain run on a core only if the LET period of the core divides the periods of its tasks. The chosen g then divides every period on the core, so it is at most their GCD, and the load it reserves is at least the true LET load: every solution is admissible. Conversely, the GCD of the periods on a core is in $\mathcal{G}$, so an allocation that is admissible with the true LET periods stays feasible with g equal to that GCD, and no allocation is lost. Eq. (22) is the cap (C3), Eq. (23) the memory (C4), and Eq. (24) keeps the task-chains ξ of a replica group ϱ on distinct ECUs (C5), with an auxiliary $z_{\xi,k}$ that is true if ξ has a sub-chain on ECU k; the core policy replaces ECUs by cores. Co-location (C2) is already ensured by partitioning. The objective is min $\sum_p u_p$.

Symmetry breaking. Cores of one ECU and one kind (type, speed, LET WCET) are interchangeable. Within such a group, $u_{p_{j+1}} \le u_{p_j}$: cores are used in order. Moreover, with the sub-chains numbered by decreasing utilization, sub-chain i may only use the first $i+1$ cores of a group. Any solution can be relabelled to satisfy this, by sorting the used cores of each group by the smallest index of their sub-chains: the j-th core then holds no sub-chain of index below j.

8.2 Schedulability as lazy constraints

Constraints (C6) and (C7) are not linear. They are added lazily (Algorithm 7), in the spirit of logic-based Benders decomposition [55]. After each solve, every used core is checked with the response-time analysis and the priority assignment of Algorithm 1, which is optimal (Theorem 1). For an unschedulable core with sub-chains S, the check removes sub-chains from S one at a time, as long as the rest stays unschedulable. The result S' is unschedulable, but

Algorithm 7: Exact($\mathcal{F}$)

```
1  K ← ∅                                                  ▷ cuts
2  repeat
3      solve Eqs. (20) to (24) with the cuts K, minimizing ∑ u_p
4      if infeasible then return infeasible
5      foreach used core p with sub-chains S, unschedulable do
6          S' ← S
7          foreach f ∈ S' do
8              if S' \ {f} unschedulable then S' ← S' \ {f}
9          add Eq. (25) for S' on every core of the kind of p to K
10 until every used core is schedulable
11 return the allocation and ∑ u_p
```

it becomes schedulable if any of its sub-chains is removed. The cut

$$\sum_{f\in S'} x_{f,q} \le |S'| - 1 \qquad (25)$$

is then added for every candidate core q of the same kind as the failing one, and the model is solved again, starting from the previous solution.

Proposition 8 (Validity of the cuts). *No cut removes an allocation in which every core is schedulable. Algorithm 7 therefore returns an optimal allocation, and the objective bound of any of its solves is a lower bound of the optimum.*

Proof. Schedulability under the optimal assignment is monotone with respect to removing sub-chains: removing tasks shrinks every higher-priority set and can only make the LET period longer, so the order that was schedulable stays schedulable for the remaining tasks. Hence every superset of the unschedulable set S' is unschedulable on a core of that kind, and the WCETs, utilizations and LET task depend only on the kind. An allocation that puts all of S' on such a core is not admissible, which is what Eq. (25) forbids. The cuts only remove inadmissible allocations, so every solve is a relaxation of the problem and its bound is a lower bound. The loop ends: a new cut is never satisfied by the solution that produced it, so no cut is added twice, and there are finitely many. When the last solve is proved optimal, its solution is admissible and optimal for a relaxation of the problem, hence optimal for the problem. When the time limit stops a solve, the best admissible solution and the bound are reported instead. ∎

Implementation and checking. The model is solved with the CP-SAT solver of OR-Tools [73], with a time limit; if the limit is reached, the best admissible solution and the bound are reported. The schedulability checker is a separate implementation of Eq. (6) and Algorithm 1. It is first compared with the heuristic on the heuristic's own mappings. Every exact allocation is then placed again by the implementation of the heuristic, with all its admissibility checks and its priority assignment; the experiments stop if it is rejected. None was.

9 Life of a platform: software updates

In brief. A vehicle platform is built for years, while its software keeps growing. The lifecycle analysis sizes the hardware for the software at the start of production, then adds the task-chains of successive software updates to that fixed hardware. It measures how many updates the platform absorbs, and how its timing margin and deadline satisfaction evolve, against the cost of the hardware.

The question. The hardware of a vehicle platform is chosen before the SOP and is then fixed for the life of the vehicle line. Software updates, over the air or at service, add functions to it for years. A platform sized tightly for the SOP software is cheap, but the first update may not fit. A platform with room to spare costs more cores from the first car on. The lifecycle variant of the problem (Section 5) makes this trade-off measurable. It is related to the extensibility of a distributed system [54], but here the room is consumed by whole new task-chains, with their own pinned drivers and network streams, not by growing WCETs.

Procedure. The SOP mapping is the design point of Section 7 for the initial application Ξ_0, at a *provisioning* cap φ_{SOP}. A lower cap buys more cores. The platform is then fixed: the cores this mapping uses, nothing more. Update $u = 1, 2, \ldots$ brings a set Ξ_u of new task-chains. It is processed as follows (Algorithm 8).

1. The new task-chains are partitioned like the initial ones (Algorithm 3).
2. Their sub-chains are allocated with the rule of the SOP mapping, onto the cores of the platform only (Algorithm 4 with fixed hardware: no core may be added). The sub-chains already deployed stay where they are, because an update re-flashes software and should not move what has been validated. The cores may now be filled up to the cap of the updates, $\varphi_{\text{upd}} \geq \varphi_{\text{SOP}}$.
3. The priorities of every core are assigned again (Algorithm 1). The tasks keep their cores, but their priorities may change, which a real update would have to re-validate.
4. The update is *absorbed* if every new sub-chain found a core and every core is schedulable. The end-to-end timing of all task-chains, old and new, is then analysed again: a new stream can delay old ones on a shared port.

The first update that is not absorbed ends the life of the platform: its software would need new hardware. The results are the number of updates absorbed, and the critical scaling factor, the load and the share of task-chains that meet their deadline after each update.

Choices. The analysis evaluates one update policy, not the best possible one: a policy that re-allocated deployed software would absorb more updates (Section 12). The update does not use consolidation, relocation moves or the local search, since these would move deployed software; the deadline satisfaction after an update is therefore that of the new allocation as it is. The cap of the updates is the design cap of the platform ($\varphi_{\text{upd}} = 0.8$ in the experiments), while the SOP cap is the provisioning decision. The new task-chains are generated with the same statistics as the initial ones and numbered after them, so that their pinned drivers land in random zones, as new functions with new sensors would.

Algorithm 8: Lifecycle($\Xi_0, \Xi_1, \ldots, \Xi_K$)

```
π ← design point of Ξ_0 at φ_SOP
P ← the cores π uses                          ▷ the platform, fixed
for u ← 1 to K do
    F_u ← Partition of Ξ_u
    π' ← π extended by Allocate(F_u) on P, cap φ_upd, no new core
    if infeasible then return u − 1 updates absorbed
    Priorities on every core of π'
    if a core is unschedulable then return u − 1 updates absorbed
    π ← π'; record λ(π), load, task-chains on time
return K updates absorbed
```

10 A worked example

In brief. This section follows one core and one task-chain through the analyses, with the default parameters: $c_{\text{LET}} = 50\,\mu\text{s}$, $\varphi = 0.8$, a 1 Gbit/s star with one CBS class, no contention.

One core. A big core p of the central computer holds two sub-chains: $a_1 \to a_2$ (one pattern of task-chain A, $C = 1000$ and $2000\,\mu\text{s}$, $T = 10\,\text{ms}$) and b_1 ($C = 2400\,\mu\text{s}$, $T = 6\,\text{ms}$). Then $T_p^{\text{LET}} = \gcd(10, 10, 6)\,\text{ms} = 2\,\text{ms}$, and the LET load is $50/2000 = 2.5\,\%$. The load is $0.1 + 0.2 + 0.4 + 0.025 = 0.725 \leq \varphi$.

Priorities (Algorithm 1). At the lowest level, the candidates are a_2 (the last task of its pattern) and b_1; a_1 is not a candidate while a_2 is unassigned. With $W = 50 + 5400 = 5450$, a_2 has the slack bound $(10000 - 5450)/10000 = 0.455$. Its WCRT below $\{a_1, b_1, \text{LET}\}$ starts at $\max(2000 + 1000 + 2400 + 50,\ 2000/0.475) = 5450$ and iterates to $2000 + 1000 + 2400 + \lceil 5450/2000 \rceil \cdot 50 = 5550$, a fixed point: slack 0.445. The bound of b_1, $(6000 - 5450)/6000 = 0.09 < 0.445$, stops the scan, and a_2 takes the lowest level. At the next level, a_1 (now a candidate) has $R = 1000 + 2400 + 2 \cdot 50 = 3500$ (slack 0.65), which beats the bound $(6000 - 3450)/6000 = 0.425$ of b_1. Last, b_1 has $R = 2400 + 2 \cdot 50 = 2500 \leq 6000$. The final order is LET, b_1, a_1, a_2, with response times 2500, 3500 and $5550\,\mu\text{s}$, and precedence holds (a_1 above a_2). The critical scaling factor is 1.25: at $\lambda = 1.25$, $R_{a_2} = 1.25(2000 + 1000 + 2 \cdot 2400) + 5 \cdot 50 = 10\,000 = T$.

One task-chain. Task-chain A is $s \to a_1 \to a_2 \to u$, all at 10 ms, with a 64-byte message per task. Its sensor driver s ($C^{\text{ref}} = 200$) is pinned to ZCU2 and its actuator driver u ($C^{\text{ref}} = 150$) to ZCU4. Partitioning cuts at both changes of binding: $\{s\}$, $\{a_1, a_2\}$, $\{u\}$. With BFDU, $\{a_1, a_2\}$ goes to the core p above. On the zone cores (speed 50), $C_s = 400$ and $C^{\text{LET}} = 100$, so $R_s = 500$ and $R_u = 300 + 100 = 400$, with

$T^{\mathrm{LET}} = 10$ ms on both.

Network (Section 6.3). A 64-byte message is one frame of $b = (64+42)\cdot 8 = 848$ bits, and $\rho = 848/10000 = 0.0848$ bit/µs. Assume these are the only flows. With one class, $c^{\max} = I\bar{l}/r = 750 \cdot 12336/1000 = 9252$ bits and $L = 9252/750 + 250 \cdot 848/(1000 \cdot 750) = 12.619$ µs. The communication h_1 (ZCU2 to the central computer) has $J = 500 + 10000$, so $\sigma = 848 + 0.0848 \cdot 10500 = 1738.4$ and $d = 12.619 + 1738.4/750 = 14.937$ at the output port of ZCU2. At the switch port the burst grows by ρd to 1739.7, which gives $d = 14.938$. With the switch latency, $d_{h_1} = 31.9$, rounded up to 32 µs. In the same way, h_2 (central computer to ZCU4, $J = 5550 + 2000$) gives 32 µs.

End-to-end bounds (Proposition 3). The holistic sum is $\Lambda = 500 + 5550 + 400 + 32 + 32 = 6514$ µs. The bound of the maximum reaction time adds the sampling of the sensor (10 000 µs) and, for each communication, the copy-out delay w_h and the wait of the consumer. With implicit communication,

$$\Delta^{\mathrm{b}} = 10000 + 6514 + (10000 + 100 + 10000) + (2000 + 50 + 10000) = 48\,664\ \mu\mathrm{s}.$$

The deadline (300 % of the single pattern period) is 30 ms, so the task-chain is *late*: the two communications alone cost 32 ms of copy and sampling delays. With strict LET, $w_{h_1} = 10000 - 500 + 100 = 9600$ and $w_{h_2} = 10000 - 5550 + 50 = 4500$. The flows now have a jitter of only C^{LET} (100 and 50 µs), so $d_{h_1} = d_{h_2} = 30$, $\Lambda = 6510$ and $\Delta^{\mathrm{b}} = 50\,610$ µs: 1 946 µs more than with implicit communication.

Local search (Algorithm 6). Pulling $\{a_1, a_2\}$ back onto the core of s (load $(400+2000+4000)/10000+0.01 = 0.65$) removes h_1. The run s, a_1, a_2 now ends with $R_{a_2} = 4000 + 2000 + 400 + 100 = 6500$, and $\Delta^{\mathrm{b}} = 10000 + 6500 + 400 + 34 + (10000 + 100 + 10000) = 37\,034$ µs. The task-chain is still late: the communication from ZCU2 to ZCU4 links two pinned sub-chains and cannot be removed. The move is kept only if another task-chain meets its deadline thanks to it, since this one does not. This is the limit of the local search discussed in Section 11.7. CADU would have put $\{a_1, a_2\}$ on a core of ZCU2 from the start, with one inter-ECU communication instead of two, and affinity then puts s on the same core.

11 Experimental evaluation

In brief. The evaluation answers ten questions, from the safety of the bounds (RQ0) to the robustness of the results to the assumptions of the generator (RQ9), on synthetic applications that follow industrial statistics, a small detection chain and a whole-vehicle application. The main findings are collected in a box at the end of the section.

RQ0 Are the network and end-to-end bounds safe, and how pessimistic are they?

RQ1 How many cores does the method use, how far is it from the optimum, and how does it compare with general-purpose metaheuristics?

RQ2 What does communication awareness buy, and at what cost?

RQ3 How do network design choices (rate, topology, shaping, replication) and the communication scheme (implicit or strict LET) change end-to-end timing?

RQ4 What do spatial constraints, heterogeneous cores and memory contention cost?

RQ5 How effective are the deadline-driven local search and the sizing for a margin?

RQ6 How many software updates does a platform absorb, for what hardware cost?

RQ7 What do the same functions need on a domain, hybrid or distributed architecture?

RQ8 What does each part of the method contribute, and how does the method compare with the rules of practice?

RQ9 Do the conclusions hold when the assumptions of the generator change?

Two case studies (Section 11.12) and a scalability study (Section 11.13) complete the picture.

11.1 Setup

Applications. Real automotive software is not public, so the applications are generated from the published statistics of an industrial engine-management software, the WATERS 2015 benchmark of Bosch [8] (Appendix A). A task-chain has 1 to 3 activation patterns, and each pattern 2 to 5 tasks, with the probabilities of the benchmark. The periods are drawn with the benchmark's shares from $\{1, 2, 5, 10, 20, 50, 100, 200, 1000\}$ ms. Chain utilizations are log-uniform between 0.1 and 8 % of the reference core, plus 2 % of heavy chains (perception, planning) of up to 80 %. A quarter of the task-chains are local to one zone; of the others, 60 % start with a sensor and 50 % end with an actuator in a random zone. Task utilizations come from UUniFast [74], redrawn while a task exceeds 30 % of a core, messages range from 4 bytes to 16 kB, and the end-to-end deadline is 300 % of the sum of the pattern periods. The default application has 300 task-chains and 1299 tasks on average (1242 to 1395), about the consolidated software of several domain ECUs.

Protocol. Each *point* of an experiment fixes the parameters and is run on 20 applications (*instances*), the same applications at every point. Means are shown with their 95 % confidence interval, and differences between rules are paired by instance. Mapping metrics are averaged over the feasible instances. A paired difference is given as its mean ± the half-width of its 95 % Student t-interval over the instances; a difference whose interval excludes zero is called significant. RQ1 to RQ6 report BFDU as the reference packing rule and RQ7 to RQ9 FFDU, the default of the tool; both give almost the same cores at the default point (14.45 and 14.40, Figure 9). All results come from the open implementation, single-threaded per design point, on an Intel Xeon at 2.1 GHz. The 24 experiments run 9600 design points in about three minutes on two cores; the exact formulation adds 250 instances, the metaheuristics 400 runs (18 min-

utes), and the two simulators 23 flow sets and 45 mapped applications. The user manual gives the commands that reproduce every number.

11.2 RQ0: the bounds against simulation

The proofs of Section 6 carry the safety of the bounds. A simulation can only refute a bound, never prove it, so the two simulators are a check of the implementation and of the modelling assumptions: a bound below an observed delay would reveal an error.

The network. The Network Calculus bounds are compared with a discrete-event, frame-level simulation of the same network. The simulator models store-and-forward switches; credit-based shapers with the idle slopes of the analysis; TAS gates with guard bands; strict priority with per-flow token-bucket regulators at every switch for ATS; replication and elimination of frames by sequence number for FRER; and a saturated best-effort class. The flow sets are of two kinds: those of real mappings (all four rules, a ring with three classes with and without FRER, a tree with TAS, a tree with ATS, a custom topology with FRER, the domain and the distributed platforms of Table 4), and random sets loaded to 40 to 70 % of every port they cross (all topologies and mechanisms). Each set is simulated in 24 release scenarios: synchronous and random offsets; uniform, extreme and alternating jitters; best effort on and off. Two limits apply: the simulator regulates ATS flows one by one where the analysis assumes interleaved regulators, which can only lower the simulated delays; and the 20 ms horizon covers a single message of the flows of long periods, whose worst case therefore rests on the scenarios rather than on the length of the run.

Table 6 reports the largest ratio of observed delay to bound for each flow set. No simulated delay exceeded its bound, over 3377 flows. The largest ratio, 0.87, is for ATS with three classes on a tree: with bursts reset at every switch, the ATS bounds come closest to the simulated delays. On the CBS mappings the bounds are 1.5 to 2.3 times the worst observed delay. The factor is larger with TAS and on the random loads, because random jitters rarely align the bursts of many flows.

The whole system. A second, full-system simulator checks the end-to-end bounds of Section 6: the tasks on their cores under preemptive fixed priorities, their LET tasks, and network delays drawn between zero and their bound. Every core starts at a random phase (the cores are not synchronized); a job reads its input when it starts and writes its output when it ends; the LET task of the producer core copies the last output written by its release (with strict LET, the output of the last producer job whose period has ended), and that of the consumer core reads, at the release of the consumer job, the last message arrived. Each sample of the first task is tracked, by its timestamp, to the first job of the last task that uses it. This gives an observed *end-to-end delay*, from the release of the sampling job to the end of that last job, whose bound is $\Gamma^{\mathrm{b}}_{\xi}$ (Eq. (17)), and an observed maximum reaction time, bounded by $\Delta^{\mathrm{b}}_{\xi}$. Nine settings (the zonal platform with FFDU and CADU, a ring with FRER and three classes, a tree with TAS, shared-memory contention at 20 ns, strict LET on the star and on the ring, the domain and the hybrid platforms) are mapped for five applications of 200 task-chains, and each mapping is simulated in four scenarios: execution times at their WCET and network delays at their bound, or both drawn uniformly (WCETs in $[0.5C, C]$), with new phases every time.

Table 6. Network bounds against a frame-level simulation (24 scenarios per flow set, 20 ms horizon, two random seeds). Max ratio: largest observed delay over its bound.

flow set	network	flows	max ratio	violations
mapping BFDU	star, CBS 1	129	0.600	0
mapping WFDU	star, CBS 1	193	0.452	0
mapping FFDU	star, CBS 1	129	0.600	0
mapping CADU	star, CBS 1	46	0.507	0
mapping FFDU	ring, CBS 3	128	0.613	0
mapping FFDU, FRER	ring, CBS 3	128	0.644	0
mapping FFDU	tree, TAS 1	129	0.233	0
mapping FFDU	tree, ATS 3	129	0.868	0
mapping FFDU, FRER	custom, CBS 3	128	0.686	0
domain, FFDU	tree, CBS 1	153	0.440	0
distributed, FFDU	star, ATS 1	180	0.559	0
load 50 %	star, CBS 1	183	0.242	0
load 70 %	star, CBS 1	275	0.212	0
load 60 %	tree, CBS 1	228	0.096	0
load 60 %	ring, CBS 1	248	0.083	0
load 60 %	star, CBS 3	111	0.320	0
load 50 %	star, TAS 3	116	0.204	0
load 60 %	star, ATS 3	94	0.408	0
load 60 %	ring, ATS 1	215	0.219	0
load 40 %, FRER	ring, CBS 1	79	0.102	0
load 40 %, FRER	ring, ATS 3	45	0.295	0
load 40 %, FRER	custom, CBS 1	205	0.244	0
load 40 %, FRER	custom, ATS 3	106	0.383	0
all		**3377**	**0.868**	**0**

No observed delay exceeded its bound, over 9000 task-chains (Figure 8). The end-to-end bound is reached exactly by 3795 task-chains, all of them without an inter-ECU communication. With one, the median observed delay is 81 % of the bound with implicit communication and 94 % with strict LET, whose timing depends less on the response times, and the largest is 99.4 % and 99.997 %; the largest observed reaction time is 99.7 % of its bound. The bounds are therefore attained, or nearly, in the worst case, and pessimistic by about a fifth for a typical task-chain that crosses the network. The network delays are drawn up to their bound here, so this simulation checks the task-level part of the analysis, and the frame-level simulation above the network bounds.

11.3 RQ1: number of cores and optimality gap

Cores and the lower bound. Figure 9(a) shows the cores used as the sub-chain limit σ varies, with the lower bound of Proposition 4. BFDU and FFDU use 14.45 and 14.40 cores at the default point, 1.21 times the lower bound of 11.95 (1.15 to 1.36 over the instances). Consolidation empties 1.4 cores on average. WFDU spreads the load over all 28 initial cores by construction, so its M measures the spreading, not

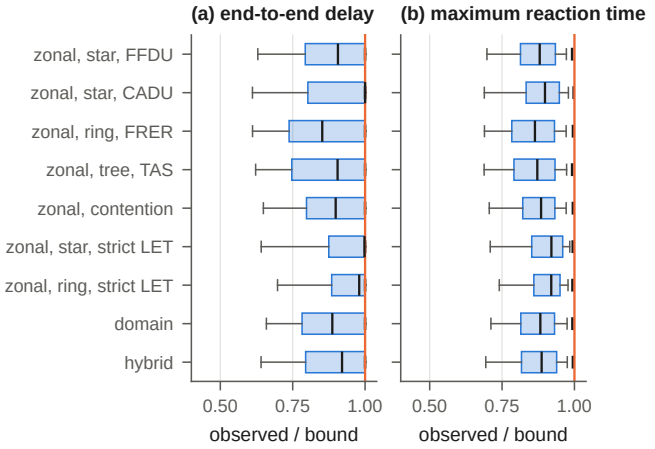


Figure 8. Full-system simulation: observed delay over its bound for every task-chain (9000 task-chains, 5 applications of 200 per setting, 4 scenarios each). Box: quartiles and median; whiskers: 5th and 95th percentiles; tick: largest.

a need. σ barely changes the number of cores. Most task-chains use less than 8 % of a core, so the cuts are mostly forced by bindings: the fragmentation rate falls from 49 % at $\sigma = 0.02$ to 24 % at $\sigma \geq 0.2$. Small sub-chains only add communications, as Figure 9(b,c) shows.

Optimality gap. The lower bound ignores the LET load, the schedulability gaps and the integrality of each zone, so the ratio above overstates the gap. The exact formulation of Section 8 measures it. It was run on 150 instances: 20 to 300 task-chains, 10 instances per size, on three platforms (default; zone cores at 25 % speed; and 20 % of replicated task-chains, 10 % of zone-only task-chains and 8 MB of memory per zone controller). On the constrained platform, no instance of 300 task-chains has an admissible mapping: the pinned tasks of some zone alone exceed its memory, which both methods detect. Of the 140 other instances, 134 were solved to proven optimality within 60 s (5.3 s on average) and 6 stopped at the limit with their best solution, against which the gap is then measured; the tool accepted every exact allocation. The optimum is that of the sub-chains of step 1, without contention, over the candidate cores of Section 8. CADU maps only 2 of the 10 constrained instances of 200 task-chains, which BFDU, FFDU and the exact solver all map: it fills the 8 MB of zone memory with free sub-chains, after which a replica finds every ECU with memory left already running another replica of its task-chain. Figure 10 and Table 7 give the result. With consolidation, BFDU and FFDU are 2.3 % above the best allocation found on average, and equal to it in 83 % of the instances. Counting only proven optima, BFDU is optimal in 113 of the 140 instances (81 %), and, measured against the lower bound of the solver on the 6 unproven instances, its mean gap is at most 2.9 % (2.85 % for FFDU). Without consolidation they were 20 % above it, one to three cores on instances that need five to twenty. The loss comes from the order of the packing: a core opened early for a large sub-chain stays lightly loaded, while the smaller sub-chains allocated later fill the gaps that the pinned work leaves elsewhere; emptying such cores is what step 3 does.

Table 7. Exact formulation against the heuristics (10 instances per row; instances without an admissible mapping left out). Zonal platform: default, slow zone controllers, spatial constraints; then the domain and hybrid platforms of Table 4. *proven*: solved to optimality within 60 s; *opt.*: mean optimum (best found if not proven); gaps in % of the optimum, with consolidation / without.

platform	chains	proven	M_{LB}	opt.	BFDU	FFDU	CADU
default	20	10/10	5.0	5.0	0 / 20	0 / 20	0 / 16
default	50	10/10	5.0	5.0	0 / 34	0 / 34	4 / 28
default	100	10/10	5.0	6.1	0 / 31	0 / 31	21 / 41
default	200	10/10	7.4	9.1	0 / 13	0 / 13	35 / 54
default	300	10/10	11.6	13.7	3 / 11	3 / 11	37 / 50
slow zones	20	10/10	5.0	5.2	0 / 16	0 / 16	2 / 16
slow zones	50	10/10	5.0	6.0	0 / 14	0 / 14	15 / 37
slow zones	100	9/10	5.4	8.2	1 / 10	1 / 10	39 / 68
slow zones	200	10/10	10.0	13.1	2 / 6	2 / 6	58 / 72
slow zones	300	7/10	15.7	19.5	1 / 5	1 / 4	44 / 54
constraints	20	10/10	5.0	5.0	0 / 20	0 / 20	0 / 14
constraints	50	10/10	5.0	5.0	4 / 30	4 / 30	4 / 30
constraints	100	8/10	5.0	5.9	7 / 39	7 / 38	19 / 48
constraints	200	10/10	7.6	9.1	14 / 30	13 / 29	32 / 53
domain	20	10/10	5.0	5.0	0 / 14	0 / 0	0 / 4
domain	50	10/10	5.0	5.0	0 / 50	0 / 18	6 / 24
domain	100	10/10	5.0	5.1	10 / 83	2 / 37	38 / 63
domain	200	10/10	7.0	8.1	7 / 50	11 / 26	36 / 71
domain	300	10/10	10.8	12.4	5 / 34	11 / 34	26 / 53
hybrid	20	10/10	5.0	5.0	0 / 20	0 / 20	0 / 16
hybrid	50	10/10	5.0	5.0	0 / 34	0 / 34	2 / 24
hybrid	100	10/10	5.0	5.7	2 / 41	2 / 41	14 / 44
hybrid	200	10/10	7.2	8.7	1 / 19	1 / 19	30 / 49
hybrid	300	10/10	11.2	13.0	3 / 15	2 / 14	33 / 48

CADU is further from the optimum (22 %), on purpose: it gives up cores to keep computations next to their devices (RQ2).

Other architectures. The same study on the domain and hybrid platforms of Table 4 (100 more instances, all proven optimal, in 1.4 s on average and 11.4 s at worst, every exact allocation accepted by the tool; one domain instance used every candidate core of an ECU, and solving it again with six spare cores instead of two gave the same optimum) gives a gap of 4.4 % for BFDU and 4.7 % for FFDU on the domain platform, optimal in 68 % and 62 % of the instances, and of 1.2 % and 1.1 % on the hybrid one, optimal in 88 % and 90 % (Table 7). The domain platform is the harder case: without a central computer, the free sub-chains have no single obvious host. This study changed the method. In its first version, a platform without a central computer gave the free sub-chains to any ECU, in the order of the platform, which let them fill microcontroller cores where they cost twice the utilization: on the domain platform, BFDU was 19 % above the optimum on average (36 and 37 % at 200 and 300 task-chains) and FFDU 7.2 %. Giving them first to the ECUs with the fastest cores (Section 4.5) brought the averages down to 4.4 and 4.7 %. The zonal and hybrid platforms, which have a central computer, are not affected.

Against metaheuristics. Simulated annealing (SA) and a genetic algorithm (GA) solve the same problem: the same sub-chains, every constraint and the schedulability test, with the number of cores as the objective (Appendix B).

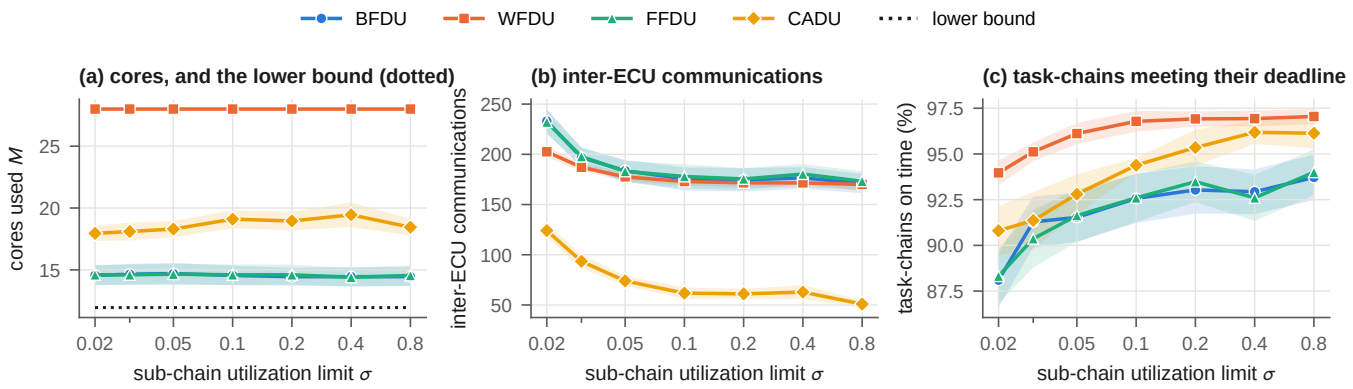


Figure 9. Sub-chain limit σ and allocation rule (300 task-chains, 20 instances, 95 % confidence bands). BFDU and FFDU coincide in cores. (a) Cores used, with the lower bound of Proposition 4 (dotted). (b) Inter-ECU communications. (c) Task-chains that meet their deadline.

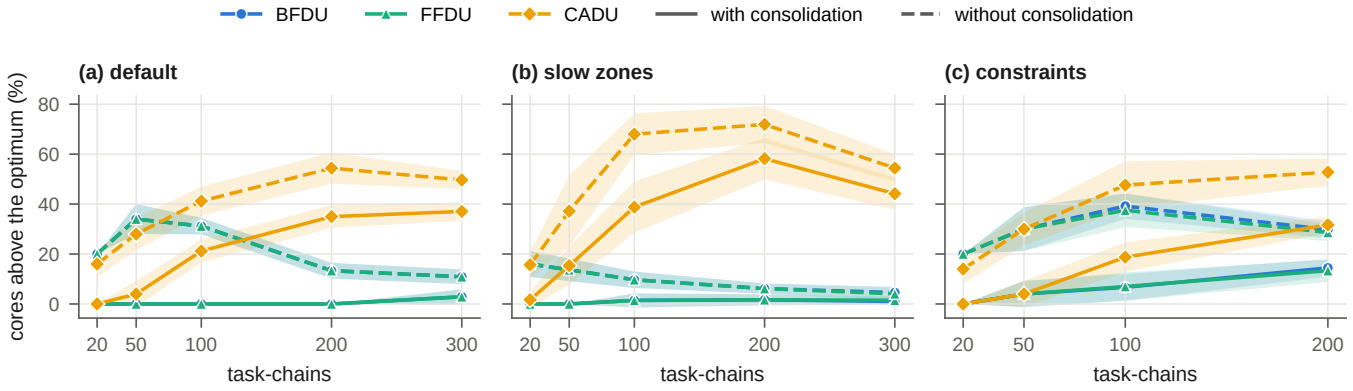


Figure 10. Cores above the optimum found by the exact formulation (10 instances per size, 95 % confidence bands), with (solid) and without (dashed) consolidation. (a) Default platform. (b) Zone cores at 25 % speed. (c) Replicas, zone-only task-chains and 8 MB of zone memory.

Figure 11 plots the gap to the proven optimum against the time, on the ten default instances of each size. The packing rules are optimal on all of them up to 200 task-chains and 2.9 % above at 300, in 2 to 32 ms. SA needs 10^6 moves, 3.7 to 13.0 s, to come as close (2.0, 0, 0 and 0.8 %); with $2 \cdot 10^5$ moves it stays 2.2 to 6.7 % above. The GA degrades with the size: 0, 3.3, 6.6 and 17.9 % after 100 generations, which take up to 61 s. On these instances the exact solver (0.27 to 6.3 s on average on the default platform) is faster than both metaheuristics and proves optimality. Both metaheuristics minimize the cores only, so they are given the deadline local search of the heuristic on their best mapping. They then keep as many task-chains on time as the packing rules at 50 task-chains (97 %), and 2.6 to 6.4 points fewer beyond (88 to 89 % at 300, against 94 %).

The harder platforms. The default platform is where the packing rules are best, so Table 8 repeats the comparison on the three platforms where they are furthest from the optimum. With slow zones, the packing rules stay within 1.7 % of the best allocation known, the GA within 0.9 % (in 4 to 44 s), and SA is no better than the rules (1.0 to 3.9 %). On the domain platform, the packing rules are up to 11.1 % above (FFDU; BFDU up to 9.7 %), and the GA closes most of that gap (0 to 4.0 %, in 7 to 77 s), while SA does not (up to 20 %). On the constrained platform, no method is close at 200 task-chains: the packing rules are 13 to 14 % above the optimum, the GA 21 %, and SA finds no admissible mapping on half of the instances. The exact solver handles all these instances in 0.1 to 30 s on average, proves 104 of the 110 optima, and finds an allocation at least as good as every other method on every instance. A metaheuristic therefore adds nothing on the default platform; where the packing rules fall short, a GA run for tens of seconds recovers most of the gap on two of the three platforms, and the exact formulation all of it, in a comparable time. The packing rules are the tool for exploring a design space, and the exact formulation, or a GA beyond its reach, the tool for the final sizing.

Figure 12 varies the speed of the zone cores. At a quarter of the speed of the central computer, the pinned work of the zones needs 12.8 zone cores (BFDU), and the task-chains lose a point of deadline satisfaction because every response time in a zone grows. From 75 % speed upwards the zones need 6 cores or fewer, and the central computer is the bottleneck. A cheap microcontroller-class zone controller thus costs cores in proportion to its pinned work, not to the whole application.

11.4 RQ2: communication awareness

CADU keeps a computation next to the device it serves whenever the zone has room. At the default point it removes

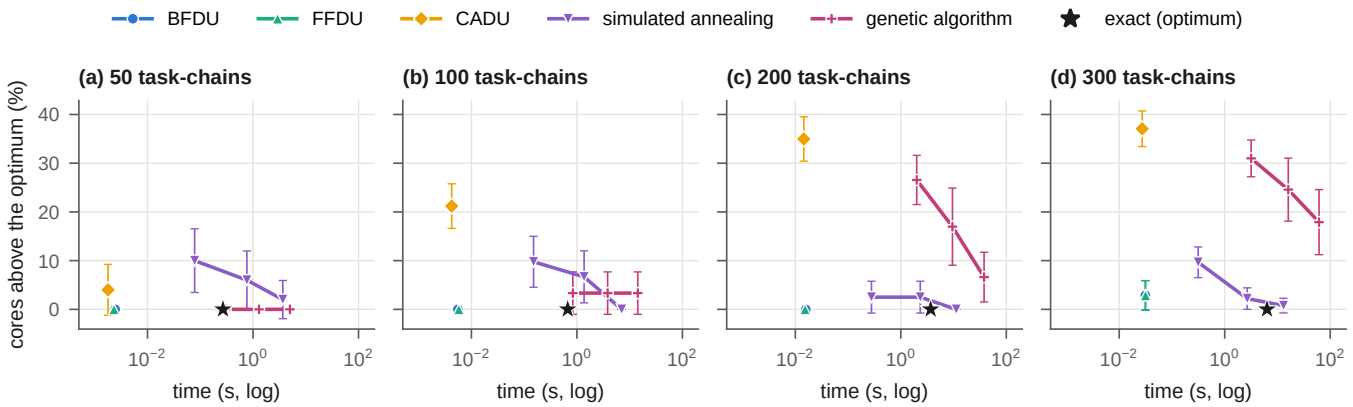


Figure 11. Cores above the optimum against the time to map, default platform (10 instances per size, 95 % confidence intervals). The heuristic rules with consolidation (BFDU and FFDU coincide), simulated annealing with $2 \cdot 10^4$, $2 \cdot 10^5$ and 10^6 moves, the genetic algorithm with 5, 25 and 100 generations (CPU time), and the exact formulation (wall time, two threads).

Table 8. The heuristic against the metaheuristics on the hard platforms of Table 7 (10 instances per size; on the constrained platform, no instance of 300 task-chains is feasible). Gap: cores above the best allocation known (the exact solver or any method), %, mean; on time: task-chains that meet their deadline, %, with the deadline local search applied to every method; SA: 10^6 moves, GA: 100 generations, with their CPU times; –: no admissible mapping found on some instances (SA on 5 of the 10 constrained instances of 200 task-chains).

		gap (%)				on time (%)		time (s)	
chains	inst.	BFDU	FFDU	SA	GA	FFDU	SA	SA	GA
slow zones									
50	10	0.0	0.0	3.7	0.0	96.6	97.0	2.9	4.4
100	10	1.4	1.4	3.9	0.0	94.4	92.9	5.0	12
200	10	1.7	1.7	3.7	0.8	93.5	87.8	7.9	28
300	10	1.0	1.5	1.0	0.9	92.4	86.3	9.0	44
constraints									
50	10	4.0	4.0	0.0	0.0	96.6	96.4	2.9	4.8
100	10	6.7	7.0	7.3	5.7	97.8	94.6	5.9	14
200	10	14.4	13.3	–	21.4	93.9	–	3.8	21
domain									
50	10	0.0	0.0	0.0	0.0	98.8	98.8	4.4	6.6
100	10	9.7	2.0	20.0	4.0	96.0	97.7	8.6	19
200	10	7.4	11.1	8.9	0.0	96.2	96.3	16	46
300	10	4.9	10.6	8.2	2.6	94.4	92.3	20	77

64 % of the inter-ECU communications (63 instead of 177; paired difference -114 ± 15). The largest network bound falls from 1.40 ms to 0.58 ms, and the share of task-chains that meet their deadline rises from 92.9 to 96.2 % (paired difference $+3.3 \pm 1.2$ points). The price is $+5.0 \pm 0.4$ cores: the work kept in the zones runs on cores at half speed, so the application load grows from 10.2 to 13.7 cores. Figure 13 shows the trade-off for every instance. BFDU and FFDU are the cheapest. CADU dominates WFDU in cores, at almost the same deadline satisfaction. WFDU's good timing comes from spreading: every core is lightly loaded and response times are short. Affinity (trying the cores of the same task-chain first) removes all communications between cores of one ECU for WFDU, but changes the bin-packing rules by less than 0.5 point, because those rules already pack the sub-chains of a task-chain together.

Table 9. Time to map one application. Exact: wall time of the constraint program (two threads, 60 s limit), mean and worst over the three zonal platforms of Table 7, and the instances proven optimal. The others: mean CPU time on the default platform (10 instances), the heuristic with consolidation, the simulated annealing with 10^6 moves and the genetic algorithm with 100 generations.

	exact (s)			heuristic (ms)		baselines (s)	
chains	mean	worst	proven	FFDU	CADU	SA	GA
50	0.37	1.8	30/30	2	2	3.7	5.0
100	7.83	60.2	27/30	6	4	7.1	14.2
200	4.45	13.1	30/30	15	14	11.2	37.9
300	17.93	60.9	17/20	32	27	13.0	61.4

11.5 RQ3: network design and communication

Rate, preemption, message sizes. Figure 14(a,b) varies the link rate, frame preemption and the message sizes. At 100 Mbit/s, the port of the central computer carries all the traffic between zones in a bin-packed mapping: its load reaches 57 %, and only 75 % of the task-chains meet their deadline, against 95 % with CADU, whose port load is three times lower. With a third of the messages large (up to 64 kB, a sensor-heavy application), even 1 Gbit/s overloads that port with bin packing (69 % load, close to the 75 % idle slope, and some ports above it): 73 % of the task-chains meet their deadline, against 95 % with CADU. Frame preemption changes almost nothing, because the blocking of one best-effort frame (12 µs at 1 Gbit/s) is small next to the LET sampling delays of milliseconds.

Topology and shaping. Figure 14(c) compares topologies at the default load. A tree or a ring adds switch ports to the routes, which raises the largest bound from 1.4 ms (star) to 2.2 ms (tree) and 2.9 ms (ring) with BFDU. On the ring, the cyclic dependencies make the bursts grow around the ring. ATS cuts this growth: the regulators reset the bursts at every switch, and the largest bound falls to 1.6 ms on the tree and 2.0 ms on the ring, with the same 1 Gbit/s links. A 10 Gbit/s backbone brings both topologies near 1.1 ms. Deadline satisfaction moves by less than one point: at this load, LET

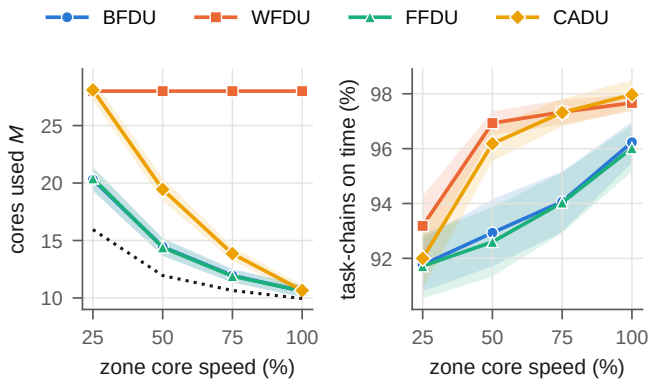


Figure 12. Speed of the zone cores (central computer at 100 %). Left: cores used and lower bound (dotted). Right: task-chains that meet their deadline.

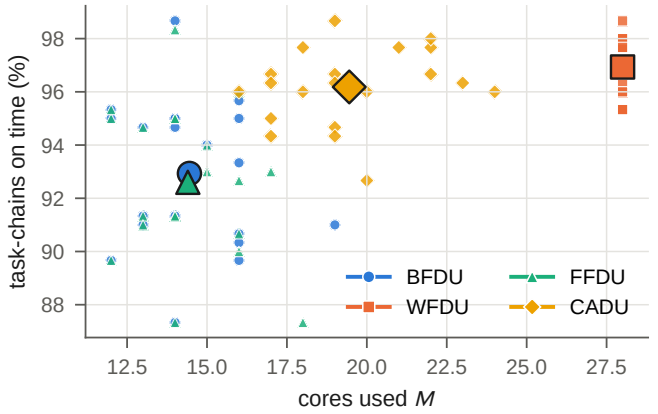


Figure 13. Cores against task-chains that meet their deadline, every instance of the default point (small markers) and means (large markers): CADU trades cores for deadline satisfaction.

sampling, not the network, dominates the reaction times.

Frame replication. Figure 15 adds FRER on the ring for 10 to 30 % of replicated task-chains. Replication sends a second member stream around the other side of the ring, so the busiest backbone link carries up to 1.4 points more load, and the largest bound grows by about 2 ms with CBS (from 2.1 to 4.1 ms at 30 %) and by 1.4 ms with ATS (from 1.5 to 2.9 ms). The bound of a replicated flow is that of its longer route, and the burst after the elimination grows with the difference between the two routes (Proposition 2). Deadline satisfaction drops by at most 0.4 point. Making the redundancy of the replicated task-chains survive a link or switch failure therefore costs little at this load, and less with ATS than with CBS, since the member streams do not accumulate bursts around the ring.

Implicit communication or strict LET. Table 10 maps the same applications with the two schemes of Section 4.3. The cores do not change, and the allocation barely does: only the local search, which sees other deadlines, moves a few sub-chains. Strict LET removes the response time from the release jitter of every flow ($J_h = C^{\mathrm{LET}}$ instead of $R_{\mathrm{prod}} + T^{\mathrm{LET}}$), so the largest network bound falls by 19 % with FFDU (by 270 µs ± 80) and by 15 % on the ring. But every crossing between cores now waits for the end of the period of its producer,

Table 10. Implicit communication or strict LET between cores (300 task-chains, 20 instances). on time: task-chains that meet their deadline (%); $\bar{\Delta}$: mean bound of the maximum reaction time (ms); d: largest network bound (µs).

	FFDU			CADU		
	on time	$\bar{\Delta}$	d	on time	$\bar{\Delta}$	d
default point						
implicit	92.6	205.6	1415	96.2	191.2	578
strict LET	62.4	244.6	1145	79.7	207.0	402
deadlines at 500 %						
implicit	99.5	208.9	1496	99.6	195.6	573
strict LET	95.3	246.6	1109	98.1	222.4	435
ring, three classes						
implicit	91.3	208.7	37135	96.0	191.7	40615
strict LET	61.7	248.5	31650	79.3	207.5	35916

so the mean reaction bound grows by 39 ms with FFDU (by 19 %) and by 16 ms with CADU, and the task-chains on time drop by 30 points with FFDU and 16 with CADU (± 2.2 and 1.4). The cause is structural: a single-pattern task-chain of period T with c crossings has a reaction bound of at least $(c+1)T$ with implicit communication and $(2c+1)T$ with strict LET, so a deadline of $3T$ admits one crossing with the first and none with the second. With deadlines at 500 % the gap narrows to 4.2 and 1.5 points. The determinism of strict LET is therefore paid in deadlines, and the fewer crossings of CADU halve the bill: strict LET needs a communication-aware allocation, or deadlines set for it.

The whole vehicle. On the whole vehicle (Table 15, 2000 task-chains), the network matters more. Raising the backbone and the link of the central computer to 10 Gbit/s increases the task-chains that meet their deadline from 81.4 to 88.3 % (BFDU). Splitting the LET traffic into classes by period does not help with CBS. With two classes, the low class (periods above 20 ms) gets bounds of up to 99 ms, and deadline satisfaction falls to 83 %. Three classes with proportional idle slopes (Eq. (10)) recover 88.8 %, close to one class. TAS with one window is close to CBS (88.0 %); with three windows it loses four points, because each class only gets its own window. ATS gives the lowest bounds with one class (2.8 ms, against 3.9 ms for CBS) and also with three (3.1 ms, against 36 ms), and the best deadline satisfaction (89.0 % with three classes).

11.6 RQ4: spatial constraints, core types and contention

Spatial constraints. Figure 16(a) adds constraints one at a time. Replicating 10 to 30 % of the task-chains with ECU separation costs at most 0.5 core (BFDU). The replicas replace other task-chains, so that the application keeps its number of task-chains, and only task-chains that are not heavy are replicated; the applications with replicas are therefore slightly lighter than those without, and this figure compares the two only roughly. Two replicas may not share the central computer, so one of each pair runs in a zone (central cores fall from 6.7 to 4.7, zone cores rise from 7.8 to 10.2), mostly on zone cores that were already there. Separating replicas by core only needs 13.7 cores instead of 14.6, but

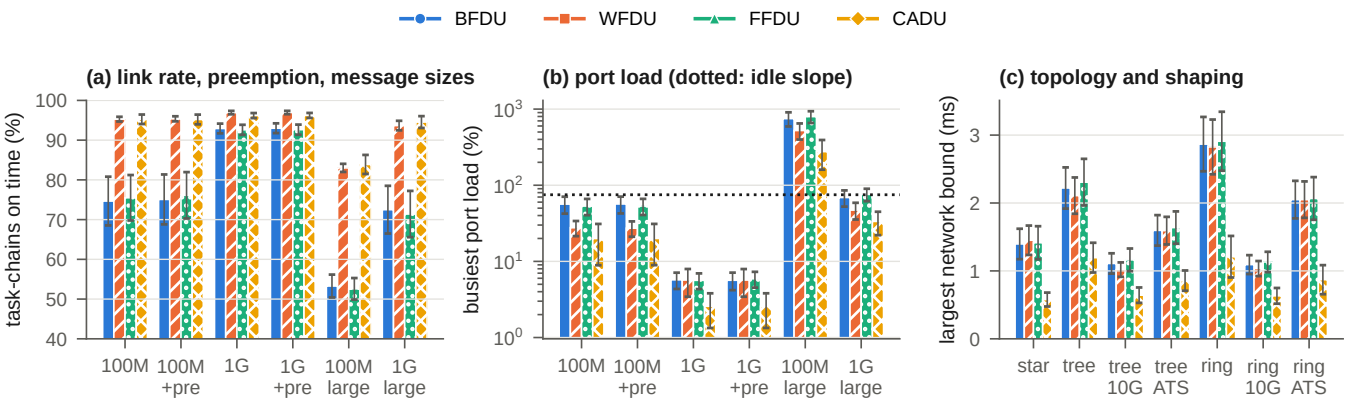


Figure 14. Network design. (a) Task-chains that meet their deadline and (b) busiest port load (log scale; above the idle slope, flows have no bound), for link rates of 100 Mbit/s and 1 Gbit/s, with frame preemption (+pre) and with large sensor messages. (c) Largest network bound for the three topologies, with a 1 or 10 Gbit/s backbone, and with ATS instead of CBS.

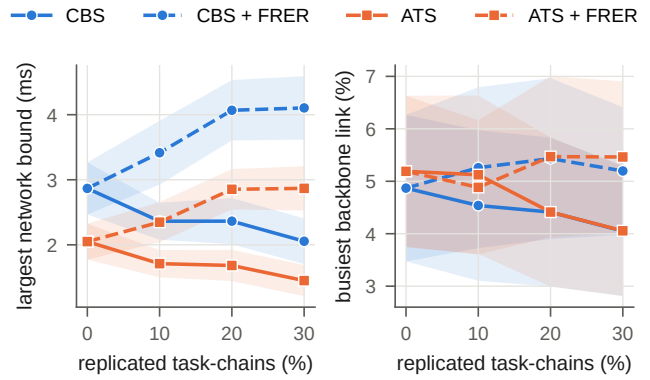


Figure 15. Frame replication on the ring (BFDU): largest network bound and busiest backbone link, with CBS or ATS, against the share of replicated task-chains.

it puts both replicas on the central computer, where one failure of that ECU loses both. Forbidding the central computer to 20 % of the task-chains (zone-only safety functions) costs 1.2 cores. Co-location of 30 % of consecutive free tasks costs nothing measurable. A memory of 8 MB per zone controller makes 55 % of the applications infeasible. In all of them, the pinned tasks of some zone alone exceed 8 MB, which no mapping can fix; by Proposition 5, and because a free sub-chain can always go to a new core of the central computer here, this is the only possible cause.

Core types. Figure 16(b) varies the core types, with per-type WCET factors drawn within ±30 % of the speed ratio. Replacing half of the big cores of the central computer by twice as many little ones (40 % speed) adds 3.7 cores for BFDU. Replacing half of the zone cores by lock-step cores, for the 10 % of safety task-chains that need them, adds 1.8 cores; both together add 5.5 cores (19.8 instead of 14.4). Splitting the zone cores between safety and other work removes packing flexibility in every zone. This cost of dedicated safety hardware is visible only with a type-aware model.

Memory contention. Figure 17 sets the latency of a memory request from 0 to 100 ns, with 1000 requests per millisecond of WCET. At 100 ns a request may wait 0.3 µs on a zone controller of four cores and 0.7 µs on the central computer of eight (more when cores are added there), a contention of 30 to 70 % of the reference WCET. The application load grows from 10.2 to 15.0 cores, and BFDU needs 22.2 cores instead of 14.4 (+54 %). CADU suffers less (+22 %, from 19.5 to 23.7 cores), because its zone cores run at half speed, so the same wait is a smaller share of their WCETs, and its central computer uses few cores. The gap between packing and communication awareness thus shrinks from five cores to 1.5. Ignoring contention, as WCET-inclusive models do, would hide that the number of cores a heavily loaded central computer needs grows with the cores themselves.

11.7 RQ5: deadline local search and margins

Local search. Figure 18 varies the deadline from 200 to 600 % of the sum of the pattern periods. At the default 300 %, without the local search, the bin-packing rules leave 88.8 % of the task-chains on time; the local search raises this to 92.9 % with 12 kept moves, in 6 ms of CPU time per design point. It is most effective where cores have room: it lifts WFDU from 79 to 97 % at 300 % and from 49 to 87 % at 200 %, while BFDU only goes from 55 to 60 % at 200 %: tightly packed cores leave little room for the moves. No move can remove a communication between two pinned sub-chains in different zones, so a task-chain whose deadline is shorter than the sampling delays of such communications stays late (Section 10).

LET cost and utilization cap. The LET cost scales with c_{LET}. At 200 µs the LET tasks load 4.6 cores, and BFDU needs 18.9 cores instead of 14.4. The utilization cap sets the margin: from $\varphi = 1$ to 0.6, the critical scaling factor grows from 1.00 to 1.67, while the cores grow from 11.5 to 21.2 (BFDU). At $\varphi = 1$, 0.7 % of the cores are unschedulable after priority assignment, and the relocation moves repair all of them.

Margins. Figure 19 compares two ways to obtain a margin on the real WCETs. A WCET margin of m % inflates every WCET and gives the factor $\lambda(1+m/100)$ on the real WCETs. Sizing for $\lambda^\star$ (Section 7.9) lowers φ instead. For the same guaranteed factor, both need about the same number of cores: 18.3 cores for a 20 % margin (factor 1.49) and 18.5 for $\lambda^\star = 1.5$ (BFDU). Sizing, however, guarantees the target

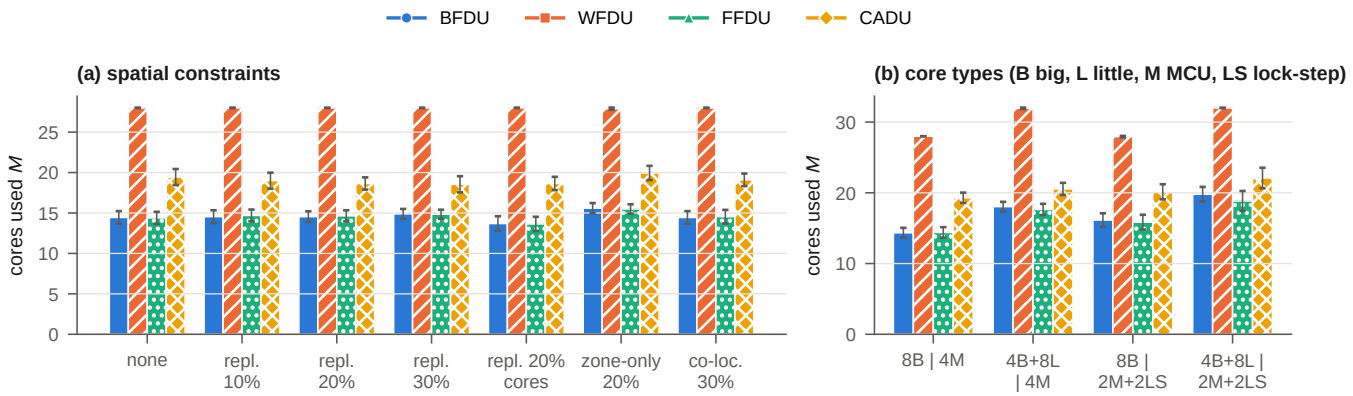


Figure 16. Cores used under (a) spatial constraints and (b) heterogeneous core types, with every WCET within ±30 % of its speed ratio on each type (B: big, L: little, M: MCU, LS: lock-step; 10 % of the task-chains on lock-step cores when LS cores exist).

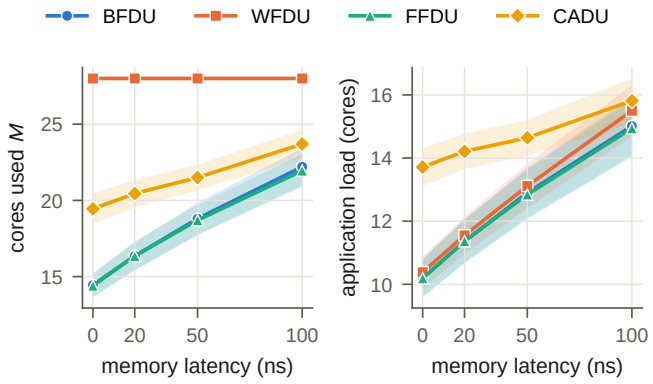


Figure 17. Shared-memory contention: cores used and application load against the latency of a memory request (1000 requests per ms of WCET, round robin).

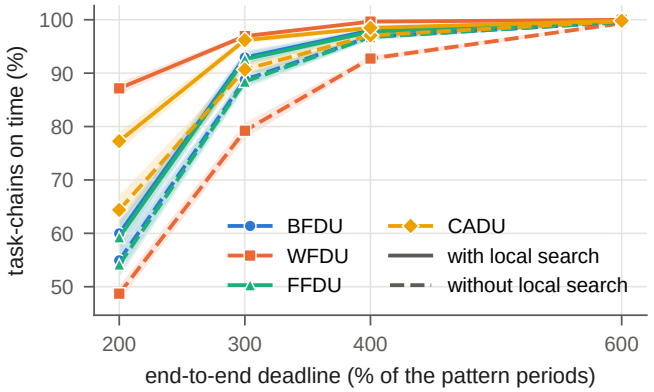


Figure 18. End-to-end deadline, with (solid) and without (dashed) the deadline-driven local search.

directly and reports it, runs at most nine design points, and keeps the real WCETs for every other metric.

11.8 RQ6: the life of a platform

Figure 20 sizes the platform at the SOP for 300 task-chains with a provisioning cap φ_{SOP} from 0.5 to 0.8, then adds ten software updates of 15 task-chains each (+50 % of software in all) up to the design cap of 0.8. A platform sized at the design cap absorbs no update with BFDU: its cores are full, and the new pinned drivers need room in their zones. Every step of provisioning buys updates: 1.3 updates at $\varphi_{\text{SOP}} = 0.7$ for 2.8 more cores, 3.8 at 0.6 for 6.8 more, and 6.5 at 0.5 for 12.1 more (26.6 cores). The relation is close to linear in the cores, about one update for two cores here. WFDU is a special case: in these experiments it uses all 28 initial cores (29 at $\varphi_{\text{SOP}} = 0.5$), and so absorbs about five updates whatever the cap, but at a cost close to that of the most provisioned BFDU platform. CADU absorbs updates at the price of its extra cores, with the best deadline satisfaction along the life (96 % against 91–93 %). The critical scaling factor of a BFDU platform falls to 1.26–1.39 after the first update and to about 1.26 after the second, whatever its provisioning (Figure 20(b)): an update fills some core up to the design cap, and the factor of the platform is that of its fullest core. Provisioning therefore buys updates, not margin.

11.9 RQ7: other E/E architectures

Figure 21 maps the same applications (the same generator seeds) of 100 to 300 task-chains onto the four platforms of Table 4. The sensors and actuators are wired to the ECUs that are not central computers, so the bindings follow each architecture: five zone controllers in the zonal platform, five domain controllers in the domain one, the ADAS controller and the four zones in the hybrid one, all twelve ECUs in the distributed one.

Cores. At 300 task-chains the hybrid platform needs the fewest cores with FFDU (13.2, lower bound 11.4), then the domain one (13.35, 11.1) and the zonal one (14.4, 11.95); the ratio to the lower bound is 1.16, 1.20 and 1.21. Both alternatives save about a core over the zonal platform for the same reason: part of the pinned work runs on ECUs with big cores, a fifth of it on the ADAS controller of the hybrid platform and two fifths on the ADAS and infotainment controllers of the domain one, instead of on microcontroller cores at half speed, which lowers the bound itself. The distributed platform runs out of room. It needs at least one core per ECU (12), and with at most two cores per small ECU and four on the ADAS controller, FFDU maps 95 % of the applications of 200 task-chains and 50 % of those of 300 (CADU: 100 and 75 %). The failures are sub-chains that find every allowed ECU at its largest number of cores; placing

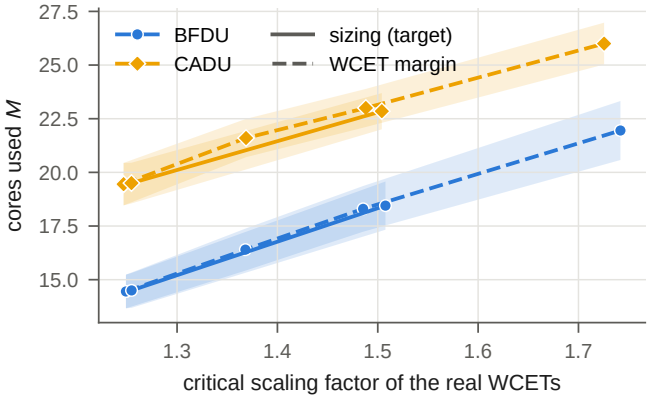


Figure 19. Cores against the critical scaling factor guaranteed on the real WCETs: WCET margins of 0, 10, 20, 40 % (dashed) and sizing for targets 1.1, 1.25, 1.5 (solid).

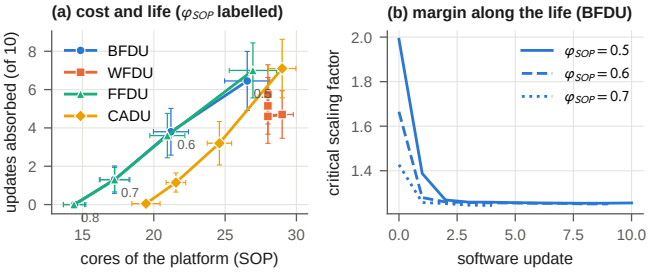


Figure 20. Life of a platform: (a) cores bought at the SOP against the software updates absorbed (of ten updates of 15 task-chains), for φ_{SOP} from 0.5 to 0.8; (b) critical scaling factor after each update (BFDU), averaged over the platforms that have absorbed every update so far.

the bound sub-chains before the free ones, so that these cannot take the cores of the ADAS controller, maps fewer applications, not more. Its 100 Mbit/s network is the most loaded (23 % on the busiest port at 300 task-chains with FFDU, against at most 6 % elsewhere).

Communications and timing. At 300 task-chains, FFDU creates 166 to 180 inter-ECU communications on the three platforms that map every application, and CADU 62 to 74: a reduction of 65 % on the zonal and hybrid platforms and of 56 % on the domain one. The share of task-chains on time is within one point between them (92.6 to 93.1 % with FFDU, 96.2 to 96.7 % with CADU). The zonal platform is therefore not the cheapest in cores for this software: what matters most is the speed of the cores that run the pinned work.

11.10 RQ8: ablation and the rules of practice

Ablation. Table 11 removes one part of the method at a time, on the default point and on a stressed one ($\varphi = 1$, so that some cores are unschedulable after step 4, and deadlines at 200 %). Consolidation saves 1.40 ± 0.28 cores with FFDU and 1.85 ± 0.35 with CADU (default), and 0.90 ± 0.21 and 1.50 ± 0.32 (stressed). The cores it empties also leave less room, so without it 3.65 ± 1.19 points more task-chains are on time with FFDU (default). The deadline-driven local search adds 4.18 ± 0.96 points of task-chains on time with FFDU and 5.50 ± 1.37 with CADU (default), and 4.60 ± 1.26 and 12.83 ± 2.44 (stressed), without changing the cores. The relocation moves only matter when cores are unschedulable: in the stressed setting, 0.7 % of the cores of FFDU are unschedulable after the priority assignment, and without relocation they stay so (the apparent saving of 0.20 ± 0.19 cores is that of a mapping that is not admissible). Adding the swap moves of the first version of step 5 before the relocation moves changed no result in either setting, nor in any other experiment (Section 7.6); they are now an option, off by default. Affinity makes no significant difference to the packing rules (every paired interval contains zero). Without all of them, FFDU needs 1.40 ± 0.28 more cores and loses 6.58 ± 1.11 points of task-chains on time (default), and 15.45 ± 2.28 points when stressed.

Rules of practice. Table 12 compares the method with three rules a practitioner might apply. Keeping task-chains whole, cut only where their bindings force it ($\sigma = \varphi$), costs no significant core ($+0.15 \pm 0.23$), and keeps 1.40 ± 1.06 points more task-chains on time with 7.2 ± 6.1 fewer communications on this point; the next paragraph shows where it fails. Putting all free software on the central computer costs 0.75 ± 0.40 cores, adds 55 ± 10 inter-ECU communications ($+31$ %) and loses 6.93 ± 1.01 points of task-chains on time: the zone cores that the pinned work leaves partly empty are not used for nearby computations. On this point, applying both rules gives the cores and communications of the central rule alone. Balancing the load (WFDU) spreads the software over all 28 initial cores ($+13.6 \pm 0.8$) for 4.33 ± 1.38 points more on time. CADU gets most of this timing benefit ($+3.58 \pm 1.35$ points) for 5.05 ± 0.49 more cores, and removes two thirds of the communications.

The sub-chain limit. Table 13 raises σ from 0.4 to 0.6 and to $\varphi = 0.8$ (whole task-chains). On the default point, most task-chains are far smaller than σ (RQ1), so the limit cuts little more than the bindings do: a larger σ changes the cores of FFDU by no significant amount, and saves CADU 1.00 ± 0.64 cores and 12.0 ± 5.1 communications at $\sigma = 0.8$. The limit matters when a sub-chain grows after the cut. With shared-memory contention at 50 ns, the WCETs grow with the number of cores that compete, and a large sub-chain no longer fits any core within φ once its LET task and the contention are added: only 45 % of the applications can be mapped at $\sigma = 0.8$, and 95 % at 0.6, against all of them at 0.4. With 600 task-chains, the cores are the same within 0.6 for every σ. The default $\sigma = 0.4$ is therefore the largest value tested that never made an application infeasible; without contention, CADU saves about a core with $\sigma = 0.6$ (0.90 ± 0.68).

11.11 RQ9: robustness to the generator

The applications are synthetic, so Table 14 changes each assumption of the generator in turn, to a lower and a higher value. The numbers of cores follow the load: halving or doubling the largest utilization of a task-chain gives 10.2 or 24.3 cores with FFDU, and the ratio to the lower bound stays between 1.16 and 1.39 at every point. The share of task-

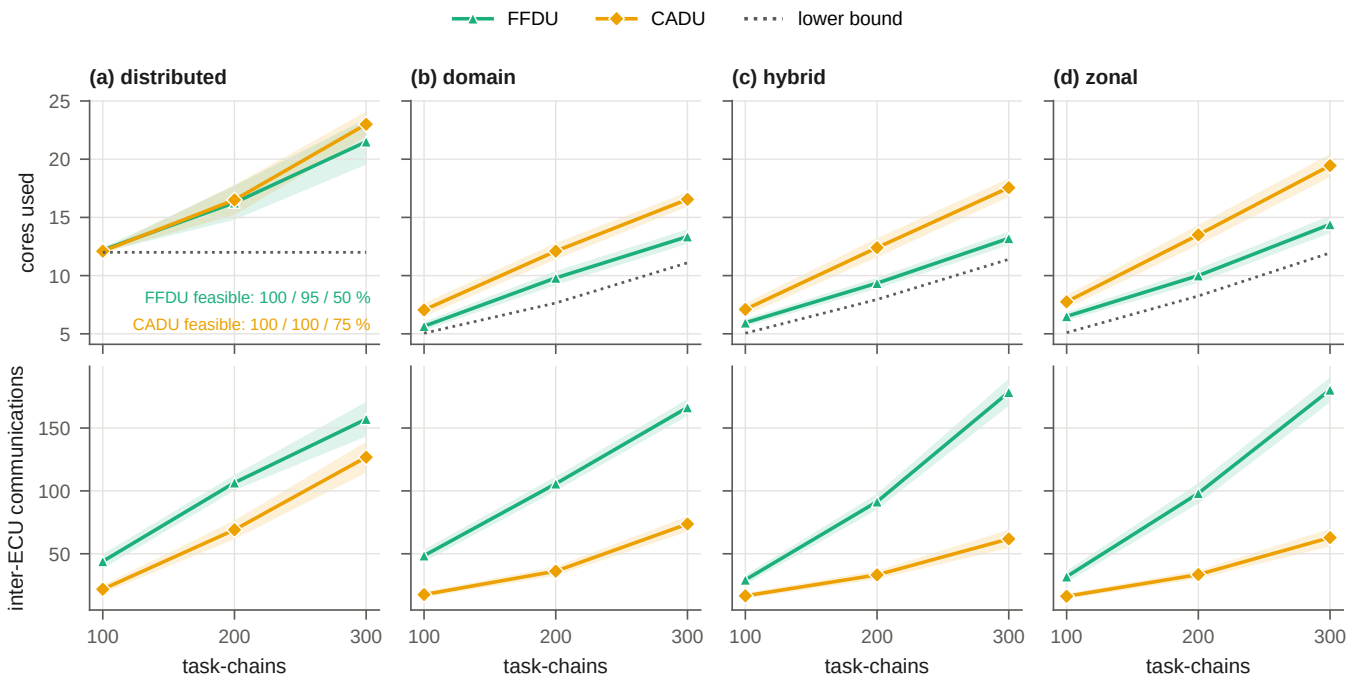


Figure 21. The same applications on four E/E architectures (Table 4; 20 instances, 95 % confidence bands, means over the feasible instances). Top: cores used and lower bound (dotted). Bottom: inter-ECU communications. Only the distributed platform has instances without an admissible mapping; their share is given in (a), and its means are over fewer, different instances for each rule and size.

Table 11. Ablation: the method with one part removed (300 task-chains, 20 instances). Default: the default platform. Stressed: utilization cap $\varphi = 1$ and deadlines at 200 % of the sum of the pattern periods. M: cores used; on time: task-chains that meet their deadline (%); sched.: cores schedulable at the end (%, FFDU).

	default					stressed				
	FFDU		CADU			FFDU		CADU		
variant	M	on time	M	on time	sched.	M	on time	M	on time	sched.
full method	14.40	92.6	19.45	96.2	100.0	11.45	61.3	16.15	80.9	100.0
no affinity	14.40	92.6	19.40	96.2	100.0	11.45	61.2	16.20	80.7	100.0
no consolidation	15.80	96.2	21.30	97.9	100.0	12.35	57.1	17.65	87.0	100.0
with swap moves	14.40	92.6	19.45	96.2	100.0	11.45	61.3	16.15	80.9	100.0
no relocation moves	14.40	92.6	19.45	96.2	100.0	11.25	60.7	15.95	80.8	99.3
no local search	14.40	88.4	19.45	90.7	100.0	11.45	56.7	16.15	68.1	100.0
none of them	15.80	86.0	21.30	90.7	100.0	12.30	45.6	17.55	68.6	99.3

Table 12. The method against the rules of practice (300 task-chains, 20 instances). M: cores used, on the central computer / the zone controllers; hops: inter-ECU communications; port: load of the busiest port (%); on time: task-chains that meet their deadline (%).

rule	M	central/zones	hops	port	on time
method, FFDU	14.40	6.65/7.75	180.3	5.6	92.6
method, CADU	19.45	1.85/17.60	62.9	2.6	96.2
whole task-chains	14.55	6.70/7.85	173.2	5.2	94.0
central computer only	15.15	8.30/6.85	235.6	9.2	85.7
whole and central only	15.15	8.30/6.85	235.6	9.2	86.0
load balancing (WFDU)	28.00	8.00/20.00	171.6	5.7	96.9

chains with a sensor and an actuator matters most for the timing: at 90 % / 80 %, 83.6 % of the task-chains are on time with FFDU and 92.9 % with CADU, because more task-chains cross the network twice. Message sizes change the load of the busiest port (1.0 to 15.3 % with FFDU), not the cores. The conclusions of RQ1 and RQ2 hold at every point: FFDU uses the fewest cores, and CADU the fewest communications and the most task-chains on time, with paired differences of +2.5 to +6.5 cores, −58 to −141 communications and +0.7 to +9.3 points on time, all significant.

11.12 Case studies

A detection chain. A small ADAS detection chain, an illustrative example of nine tasks that is not taken from a benchmark (Appendix A), forms one task-chain with patterns at 50, 25 and 10 ms and 3.7 cores of load, all pinned to the central computer. At $\varphi = 1$, all rules use 5 cores, which is optimal. The three tasks above half a core (0.65, 0.56, 0.56) cannot share a core with each other or with a task of half a core, and the two tasks of exactly half a core cannot share one either once the LET task is added. At $\varphi = 0.8$ the method uses 6 cores ($\sigma \leq 0.3$) or 7, with a critical scaling factor of 1.26 to 1.53.

The software of a whole vehicle. A premium car has 70 to 100

Table 13. The sub-chain limit σ (300 task-chains unless stated, 20 instances; $\varphi = 0.8$). feas.: instances with an admissible mapping (%); M: cores used; hops: inter-ECU communications; on time: task-chains that meet their deadline (%); means over the feasible instances.

	FFDU				CADU			
	feas.	M	hops	on time	feas.	M	hops	on time
default								
$\sigma = 0.4$	100	14.40	180.3	92.6	100	19.45	62.9	96.2
$\sigma = 0.6$	100	14.55	175.4	93.9	100	18.55	51.0	96.2
$\sigma = 0.8$	100	14.55	173.2	94.0	100	18.45	50.9	96.1
contention, 50 ns								
$\sigma = 0.4$	100	18.70	176.8	92.1	100	21.50	69.3	95.5
$\sigma = 0.6$	95	18.47	173.7	92.4	95	20.84	56.0	95.7
$\sigma = 0.8$	45	17.56	175.3	93.2	45	20.00	55.1	96.2
600 task-chains								
$\sigma = 0.4$	100	33.25	386.9	92.7	100	34.25	342.8	92.9
$\sigma = 0.6$	100	33.10	384.9	92.9	100	33.65	319.3	93.0
$\sigma = 0.8$	100	33.15	384.5	93.1	100	33.65	318.4	93.1

ECUs or more [1]. An engine control unit runs 30 to 60 task-chains [8], while many body ECUs run fewer. Taking about 20 task-chains for each of some 100 ECUs gives 2000 task-chains. Table 15 maps 2000 task-chains (8654 tasks) on a central computer with 16 initial cores. Its utilizations are those of automotive microcontrollers, so the reference core is a microcontroller core. With reference cores on the central computer and zone cores at half that speed (*reference*), the vehicle needs 89 cores, 1.17 times the lower bound. With central cores four times faster than the reference and zone cores at the reference speed (*fast*), it needs 29.5 cores. The 1 Gbit/s network is then the bottleneck of the timing: 28 to 33 % load on the busiest port, bounds of 8 ms, and 81 to 88 % of the task-chains on time. On the vehicle, CADU removes almost no communications on the reference platform, because the zones are saturated by their own pinned work (46 zone cores) and have no room left for free sub-chains. On the fast platform, CADU keeps computations on zone cores four times slower than the central ones: it needs 11.5 more cores, its task-chains are on time less often (80.9 against 88.1 %), and its greedy choices, made one sub-chain at a time, even end with more inter-ECU communications (1369 against 1189). The benefit of communication awareness therefore depends on spare zone capacity and on zone cores that are not much slower than the central ones.

11.13 Scalability

Figure 22 shows the CPU time of a design point from 50 to 3200 task-chains (226 to 13 869 tasks on average). The largest application is mapped in 1.4 to 2.1 s, depending on the rule (1.8 s with FFDU). A least-squares fit over the whole range gives a growth of $n^{1.64}$ to $n^{1.72}$. The deadline-driven local search grows fastest ($n^{1.9}$ to $n^{2.3}$ from 200 task-chains on) and takes 0.7 to 1.4 s at 3200 task-chains; the priority assignment and the consolidation take at most 1.0 s, and the relocation moves had nothing to do. A system architect can therefore sweep hundreds of design points for a whole vehicle in minutes, and check the heuristic against the exact formulation on the instance sizes of a domain controller.

Table 14. Robustness to the assumptions of the generator, one at a time (default value in parentheses; 300 task-chains, 20 instances). LB: lower bound on the cores; hops: inter-ECU communications; on time: task-chains that meet their deadline (%). Messages above one frame: 1.5 to 16 kB; without them, 90 % of the messages have 4 to 64 B and 10 % one frame.

		FFDU			CADU		
assumption	LB	M	hops	on time	M	hops	on time
default	11.95	14.40	180.3	92.6	19.45	62.9	96.2
task-chains local to one zone (25 %)							
0 %	10.10	13.50	220.2	90.6	18.75	79.2	94.4
50 %	13.70	15.95	99.0	96.2	20.10	41.5	97.4
start at a sensor / end at an actuator (60 / 50 %)							
30 / 25 %	11.75	14.10	95.6	97.5	16.60	12.3	98.2
90 / 80 %	12.05	14.30	259.7	83.6	20.75	139.0	92.9
largest utilization of a task-chain (8 %)							
4 %	8.35	10.15	149.6	95.9	14.15	51.3	97.6
16 %	18.20	24.25	181.7	90.6	27.70	124.0	93.5
heavy task-chains (2 %)							
0 %	9.10	10.95	168.4	94.0	14.60	45.9	96.8
5 %	16.35	22.80	183.8	91.0	26.30	101.2	94.6
messages above one frame (5 %)							
0 %	11.95	14.40	180.6	93.0	19.45	63.0	96.2
20 %	11.95	14.40	180.6	91.7	19.45	62.8	96.2

Table 15. Whole vehicle: 2000 task-chains, 8654 tasks (2 instances). Top: two central computers. Bottom: network options, reference central computer, BFDU.

central	rule	M	M_{LB}	central/zones	on time	CPU
reference	BFDU	89.0	76.0	43.0/46.0	81.4 %	1.0 s
reference	CADU	90.5	76.0	41.5/49.0	80.3 %	0.9 s
fast	BFDU	29.5	26.5	12.0/17.5	88.1 %	1.0 s
fast	CADU	41.0	26.5	8.5/32.5	80.9 %	1.1 s

network (reference central, BFDU)	largest bound	port load	on time
1 CBS class, 1 Gbit/s	8.4 ms	33 %	81.4 %
1 CBS class, 10 Gbit/s backbone	3.9 ms	15 %	88.3 %
2 CBS classes, 10 Gbit/s	98.5 ms	15 %	83.1 %
3 CBS classes, 10 Gbit/s	35.8 ms	15 %	88.8 %
1 TAS window, 10 Gbit/s	4.6 ms	15 %	88.0 %
3 TAS windows, 10 Gbit/s	11.1 ms	15 %	83.9 %
1 ATS class, 10 Gbit/s	2.8 ms	15 %	88.3 %
3 ATS classes, 10 Gbit/s	3.1 ms	15 %	89.0 %

Main findings

- No simulated delay exceeded its bound: 3377 network flows (all four TSN mechanisms) and 9000 task-chains simulated as a whole system, where the end-to-end bounds are reached exactly or within 1 % (RQ0).
- With consolidation, the packing rules are 2.3 to 2.9 % above the optimal allocation on average, and proven optimal in 81 % of the instances of up to 300 task-chains, in milliseconds; simulated annealing needs seconds to come as close on the default platform, and a genetic algorithm does not; on the domain platform and with slow zones, a genetic algorithm run for tens of seconds recovers most of the gap of the packing

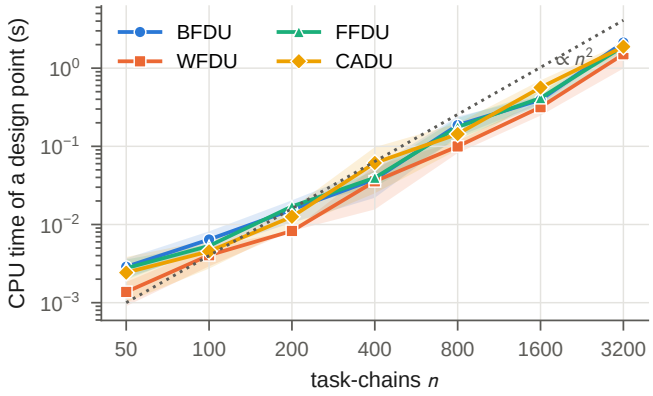


Figure 22. CPU time of a design point against the number of task-chains (log-log; dotted: slope 2).

rules, and the exact formulation all of it (RQ1).

- Communication awareness removes about two thirds of the inter-ECU communications for a third more cores (RQ2).
- ATS removes the burst growth of rings; frame replication for the replicated task-chains costs little bandwidth and deadline satisfaction; strict LET cuts the network bounds by 15 to 19 % but costs 30 points of deadlines with FFDU and 16 with CADU (RQ3).
- Memory contention can add half the cores of a packed mapping, much less for a communication-aware one (RQ4).
- Each core bought at the SOP buys about half a software update; the margin after the first update is set by the design cap, not by the provisioning (RQ6).
- The method maps domain, hybrid and distributed platforms as well, within 4.7 % of the optimum on the domain platform and 1.2 % on the hybrid one (RQ1); both need about a core less than the zonal platform, and a distributed one holds only half the applications of 300 task-chains (RQ7).
- Consolidation saves 0.9 to 1.9 cores and the local search adds 4 to 13 points of task-chains on time; putting all free software on the central computer costs cores, communications and 7 points on time; whole task-chains fail under memory contention (RQ8).
- These conclusions hold when each assumption of the generator is set to a lower or a higher value (RQ9).

12 Discussion

In brief. The experiments suggest ten design rules for an early platform. This section states them, then the limits of the model and the threats to the validity of the results.

12.1 Design rules

R1. *Size the zones by their pinned work.* The zone controllers need cores for the drivers and local task-chains wired to them. A slow microcontroller costs cores in proportion to that work (Figure 12); on a whole vehicle, the zones are saturated by it.

R2. *Consolidate after bin packing.* Decreasing-utilization packing is one to three cores above the optimum on small and constrained platforms; emptying the lightest cores closes most of this gap at little cost (Figure 10).

R3. *Keep computation in the zones when they have room and timing matters.* Communication-aware allocation trades about a third more cores for three times fewer inter-ECU communications and fewer late task-chains. The gain disappears when the zones are full, and the cost shrinks when memory contention penalizes a crowded central computer.

R4. *Budget the backbone for the whole vehicle, not for one domain.* At the default load the network hardly matters. At vehicle scale the link of the central computer is the bottleneck, and a 10 Gbit/s backbone is worth seven points of deadline satisfaction.

R5. *Prefer ATS on rings and with several classes.* Asynchronous traffic shaping removes the burst growth of cyclic topologies and keeps the bounds of low-priority classes small; with CBS, keep LET traffic in one class or allocate the idle slopes per port. Frame replication for the replicated task-chains is affordable.

R6. *Count the contention.* When the cores of an ECU share a memory, the cores needed grow with the cores used; a WCET measured in isolation underestimates the platform by up to half here.

R7. *Provision for updates in cores, size for margin in WCETs.* Each core bought at the SOP buys about half a software update, while the timing margin of the platform after its first update is set by the design cap. Sizing for a target scaling factor costs the same as an equivalent WCET margin, but guarantees and reports it.

R8. *Put fast cores where the work is bound.* The same software needs about a core less on a hybrid or a domain platform, whose ADAS (and infotainment) controllers run part of the pinned work on big cores, than on the zonal one; small ECUs with a fixed number of cores cannot absorb its growth (Figure 21).

R9. *Cut task-chains before they reach a core.* Most task-chains are far below any sensible limit, but a task-chain cut only by its bindings may no longer fit a core once the contention is counted; a limit of half the cap kept every application feasible (Table 13).

R10. *Pay for strict LET with fewer crossings.* Strict LET makes the data flow independent of response times and shrinks the network bounds, but each crossing between cores waits a full producer period. Combine it with communication-aware allocation, and set the deadlines with that wait in mind (Table 10).

12.2 Limits of the model

Platform. The contention model is a single shared memory per ECU with round-robin arbitration and a known number of requests per job. Caches, interconnects with other arbitration policies, or request counts that depend on co-

runners would need a richer model, such as the multicore response-time analysis of Davis et al. [29]. Legacy buses (CAN, LIN) behind the zone controllers are outside the network model. Each core has one LET task at the GCD of its periods; one LET group per period would lower the overhead on cores with fine-grained periods. Its WCET is a constant per instant, whatever the amount of data copied, which suits short signals but not large sensor data: a copy time that grows with the bytes copied would make the LET load depend on the allocation.

Timing. The analysis assumes synchronous releases and no offsets, which is safe but pessimistic. The response time of Eq. (6) is valid only up to the period; a mapping with a task above its period has no bound for that task, its task-chain or the flows it produces, and the method treats it as unschedulable instead of analysing the backlog [71]. LET across ECUs assumes clocks that run at the same rate, as a synchronized time base provides [40]; their phases need not be aligned, since a message waits at most one consumer period whatever the phase. Clock drift is not modelled. The network bounds assume a best-effort class that is always present, which is safe again. Frame replication needs two routes without a common link or intermediate switch: the ring and custom topologies offer them, a star or a tree does not, and a flow without them is not replicated. Mixed-criticality scheduling is not modelled.

Optimization. The heuristic optimizes (O1) by decomposition and (O2) by a local search that accepts only strictly improving moves and stops at a local optimum. The exact formulation measures the gap of (O1) for the sub-chains of the partitioning, without contention, on up to 300 task-chains on the zonal, domain and hybrid platforms, but not on the distributed one; the optimum of (O2), and of the whole problem with partitioning and network, remains unknown. The lifecycle analysis does not move deployed software; allowing an update to re-allocate it would absorb more updates, at the price of re-validating more functions.

12.3 Threats to validity

Construct validity. The applications are synthetic. Their structure (patterns, periods, tasks per pattern) is that of an industrial engine-management software [8]. The zone bindings, message sizes, memory footprints, request densities and replication rates are assumptions, and every experiment that depends on them varies them; RQ9 varies the others, one at a time. The four platforms of RQ7 are examples, not products. Utilizations are relative to a reference core and scale with its speed (the *fast* case). The whole-vehicle size is an order-of-magnitude estimate, and the updates of the lifecycle are drawn from the same statistics as the initial software. *Internal validity.* Errors in the implementation are mitigated by golden-result regression tests, by a randomized checker that recomputes response times, loads, constraints and end-to-end bounds independently of the library, by the frame-level network simulator and the full-system simulator, by round-trip tests of the input files, and by the independent schedulability checker of the exact formulation, whose solutions are re-checked by the library. *External validity.* The conclusions hold for the parameter ranges explored. The tool and the data are provided so that they can be re-run for another platform or application.

13 Conclusion

Sizing a zonal E/E platform means deciding, together, where the tasks run, how they are prioritized, how they communicate between cores at LET instants, and how their messages cross a TSN backbone, under the wiring, memory and safety constraints of a vehicle and for the whole life of the platform. This paper stated that problem formally and gave a complete method to solve it. The timing analyses come with proofs: an optimal priority assignment that respects precedence, response times with shared-memory contention, bounds on the maximum reaction time of multi-rate task-chains, and Network Calculus bounds for credit-based, time-aware and asynchronous shaping and for frame replication. The heuristic combines bin packing or a new communication-aware rule with core consolidation, relocation moves, and a deadline-driven local search with an exact incremental re-analysis. An exact formulation with lazy schedulability cuts shows that the heuristic is close to optimal, and a lifecycle analysis turns the provisioning of the hardware into a measurable trade-off with the number of software updates the platform absorbs.

On applications built from industrial statistics, no bound was exceeded in a frame-level simulation of the network or in a simulation of the whole system, and the end-to-end bounds were reached, or nearly; the packing rules with consolidation came within a few percent of the optimal allocation in milliseconds on the default platform, where simulated annealing needs seconds and a genetic algorithm falls behind, while on harder platforms a genetic algorithm run for tens of seconds, and the exact formulation, recover most of the gap; communication awareness removed about two thirds of the inter-ECU communications at the price of a third more cores; asynchronous traffic shaping removed the burst growth of rings; strict LET shrank the network bounds but cost deadlines, one producer period per crossing; memory contention could add half the cores of a packed mapping; and each core bought at the start of production bought about half a software update. Consolidation and the deadline-driven local search account for most of the benefit of the method, the conclusions hold when the assumptions of the generator change, and the same method maps domain, hybrid and distributed architectures, within 5 % of the optimum on the domain and hybrid ones. Measuring that gap also improved the method: it showed where free software should go on a platform without a central computer. A whole vehicle of 2000 task-chains is mapped in about a second, so a system architect can compare architectures, network options and provisioning policies with a worst-case analysis behind every timing result it reports.

The natural next steps are richer contention models

(caches, interconnects), mixed-criticality scheduling, reallocation of deployed software during updates, and an exact treatment of the end-to-end objective, which this paper improves but does not yet optimize.

Availability. The method, the exact formulation, the metaheuristic baselines, the two simulators and the experiments are implemented in the open-source tool `mapping` (C11 and Python, Apache-2.0), available at `https://github.com/ifupauline/mapping` and archived at `https://doi.org/10.5281/zenodo.24157732`; this paper and the manual are under CC BY 4.0. Its user manual, distributed with it, describes the installation, every option, the output, the data files, and the commands that reproduce every table and figure of this paper.

A The applications

Generator. An application of N task-chains is drawn chain by chain from a seeded pseudo-random generator (xoshiro256** seeded by splitmix64). The steps and their parameters are the following.

1. *Structure* (WATERS 2015 rules [8]): 1, 2 or 3 activation patterns with probabilities 70, 20 and 10 %; each pattern has 2, 3, 4 or 5 tasks with probabilities 30, 40, 20 and 10 %. The patterns of a task-chain have distinct periods, drawn with the shares of Table 16.
2. *Chain utilization*: log-uniform between 0.1 and 8 % of the reference core; 2 % of the task-chains are heavy, uniform between 8 and 80 %.
3. *Bindings*: 25 % of the task-chains are local to a random zone (heavy ones never are). Of the others, 60 % start with a sensor task and 50 % end with an actuator task, each pinned to a random zone. On a platform without zones, a random ECU that is not a central computer plays the role of the zone. A task-chain whose tasks would all be drivers keeps its last task free.
4. *Task utilizations*: each driver takes 5 % of the chain utilization; the computation tasks share the rest by UUniFast [74], redrawn while a task exceeds 30 % of a core. Then $C = \mathrm{round}(UT)$, at least 1 µs and below T.
5. *Messages*: 4–64 bytes (70 %), 65–1500 bytes (25 %), or 1501 bytes to 16 kB (5 %).
6. *Deadline*: 300 % of the sum of the periods of the patterns of the task-chain, on its maximum reaction time.
7. *Constraints*: memory per task log-uniform in 4–256 KB. Optionally, the experiments add: co-location of consecutive free tasks of a pattern; zone-only task-chains; per-type WCET factors log-uniform in $[(1+\upsilon)^{-1}, 1+\upsilon]$, lockstep equal to MCU; safety task-chains restricted to lockstep cores; and replicas. A replica is a copy of a random non-heavy task-chain, with its pinned tasks moved to zones that the original does not use; the free tasks of each are kept out of the zones the other is pinned to.

The task-chains are finally sorted by decreasing utilization and numbered. The task-chains of a software update (Section 9) are drawn the same way, from a seed derived from the seed of the application and the number of the update, and numbered after the existing ones.

Table 16. Periods of the WATERS 2015 benchmark and their shares of the runnables; the 15 % of angle-synchronous runnables are left out.

period (ms)	1	2	5	10	20	50	100	200	1000
share (%)	3	2	2	25	25	3	20	1	4

The detection chain. This illustrative ADAS chain is not taken from a benchmark. It is one task-chain of nine tasks pinned to the central computer, in the order GPS grabber (7/50), LIDAR grabber (12/50), camera grabber (2/25), localization (28/50), detection (28/50), EKF (6.5/10), fusion (25/50), planner (5/10) and control (4.5/10), where C/T is in ms. The messages are 64, 16 000, 16 000, 256, 1024, 256, 1024, 256 and 64 bytes.

B The metaheuristic baselines

Both baselines take the sub-chains of step 1 (Section 7.1) and map them onto a fixed set of *candidate cores*: the initial cores of the platform, then, for every ECU and core type, as many added cores as the sub-chains allowed on that ECU whose lightest type it is there would fill (their utilization on that type over φ, rounded up), within $M_k^{\max}$ and at most 64. A mapping is admissible when it meets every constraint of Section 5, schedulability included (the robust optimal priority assignment of step 4 on every core). Both minimize the energy

$$E = M - \frac{0.9}{M} \sum_{p \text{ used}} \left(1 - \frac{u_p}{\varphi}\right)^2, \tag{26}$$

where M is the number of cores in use and u_p their loads: fewer cores first, then the free capacity gathered on few cores, the next ones to empty. Since E grows with the load, a sub-chain also prefers a faster core. Neither runs the consolidation, and neither models shared-memory contention. Their best mapping then gets the deadline-driven local search of step 6, as in the heuristic, so that the task-chains on time compare the searches. Both are deterministic for a seed, and run on the default, slow-zone, constrained and domain platforms of Table 7.

Simulated annealing [75]. It starts from a first fit of the sub-chains by decreasing utilization, each to the first candidate core that takes it, the fastest cores first. A move relocates one sub-chain to another candidate core (probability 0.7) or exchanges two sub-chains of two cores (0.3). Half of the time, the moved sub-chain is the one, of two drawn at random, on the less loaded core; the target is a core in use with probability 0.8. A move that breaks a constraint is not made, and one that raises the energy by d is accepted with probability $e^{-d/T}$, the temperature decreasing geometrically from 0.03 to 0.0003 over the budget of moves. Every mapping visited is admissible: a core that loses a sub-chain stays schedulable, since its response times can only decrease.

Genetic algorithm. An individual is an order of the sub-chains, decoded by a packing heuristic, as in the hybrid genetic algorithms for bin packing of Reeves [76]. It is decoded by first fit: each sub-chain goes to the first core in use that takes it (the fastest first, then in the order they were opened), else opens the first candidate core that does, the fastest first. The population has 40 individuals: the order by decreasing utilization and random orders. Parents are chosen by binary tournaments, recombined by order crossover (OX1, probability 0.9) and mutated by a swap (probability 0.3); the two best individuals are kept.

References

[1] R. N. Charette, "This car runs on code," IEEE Spectrum, Feb. 2009.

[2] V. Bandur, G. Selim, V. Pantelic, and M. Lawford, "Making the case for centralized automotive E/E architectures," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 2, pp. 1230–1245, 2021.

[3] H. Askaripoor, M. Hashemi Farzaneh, and A. Knoll, "E/E architecture synthesis: Challenges and technologies," *Electronics*, vol. 11, no. 4, p. 518, 2022.

[4] L. Lo Bello and W. Steiner, "A perspective on IEEE time-sensitive networking for industrial communication and automation systems," *Proceedings of the IEEE*, vol. 107, no. 6, pp. 1094–1120, 2019.

[5] IEEE, "IEEE standard for local and metropolitan area networks—bridges and bridged networks," IEEE Std 802.1Q-2022, 2022.

[6] L. Mauser and S. Wagner, "Centralization potential of automotive E/E architectures," *Journal of Systems and Software*, vol. 219, p. 112220, 2025.

[7] N. Feiertag, K. Richter, J. Nordlander, and J. Jonsson, "A compositional framework for end-to-end path delay calculation of automotive systems under different path semantics," in *Workshop on Compositional Theory and Technology for Real-Time Embedded Systems (CRTS)*, 2008.

[8] S. Kramer, D. Ziegenbein, and A. Hamann, "Real world automotive benchmarks for free," in *6th International Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems (WATERS)*, 2015.

[9] M. Dürr, G. von der Brüggen, K.-H. Chen, and J.-J. Chen, "End-to-end timing analysis of sporadic cause-effect chains in distributed systems," *ACM Transactions on Embedded Computing Systems*, vol. 18, no. 5s, pp. 1–24, 2019.

[10] M. Günzel, H. Teper, K.-H. Chen, G. von der Brüggen, and J.-J. Chen, "On the equivalence of maximum reaction time and maximum data age for cause-effect chains," in *Proc. 35th Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 262, 2023, pp. 10:1–10:22.

[11] C. M. Kirsch and A. Sokolova, "The logical execution time paradigm," in *Advances in Real-Time Systems*. Springer, 2012, pp. 103–120.

[12] A. Hamann, D. Dasari, S. Kramer, M. Pressler, and F. Wurst, "Communication centric design in complex automotive embedded systems," in *Proc. 29th Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 76, 2017, pp. 10:1–10:20.

[13] A. Biondi and M. Di Natale, "Achieving predictable multicore execution of automotive applications using the LET paradigm," in *Proc. 24th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2018, pp. 240–250.

[14] L. Zhao, P. Pop, and S. Steinhorst, "Quantitative performance comparison of various traffic shapers in time-sensitive networking," *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2899–2928, 2022.

[15] K. W. Tindell, A. Burns, and A. J. Wellings, "Allocating hard real-time tasks: An NP-hard problem made easy," *Real-Time Systems*, vol. 4, no. 2, pp. 145–165, 1992.

[16] Q. Zhu, H. Zeng, W. Zheng, M. Di Natale, and A. Sangiovanni-Vincentelli, "Optimization of task allocation and priority assignment in hard real-time distributed systems," *ACM Transactions on Embedded Computing Systems*, vol. 11, no. 4, pp. 1–30, 2012.

[17] R. I. Davis and A. Burns, "A survey of hard real-time scheduling for multiprocessor systems," *ACM Computing Surveys*, vol. 43, no. 4, pp. 35:1–35:44, 2011.

[18] R. I. Davis, L. Cucu-Grosjean, M. Bertogna, and A. Burns, "A review of priority assignment in real-time systems," *Journal of Systems Architecture*, vol. 65, pp. 64–82, 2016.

[19] M. Becker, D. Dasari, S. Mubeen, M. Behnam, and T. Nolte, "End-to-end timing analysis of cause-effect chains in automotive embedded systems," *Journal of Systems Architecture*, vol. 80, pp. 104–113, 2017.

[20] M. Günzel, K.-H. Chen, N. Ueter, G. von der Brüggen, M. Dürr, and J.-J. Chen, "Timing analysis of asynchronized distributed cause-effect chains," in *Proc. 27th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2021.

[21] J. Martinez, I. Sañudo, and M. Bertogna, "Analytical characterization of end-to-end communication delays with logical execution time," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 11, pp. 2244–2254, 2018.

[22] P. Pazzaglia, A. Biondi, and M. Di Natale, "Optimizing the functional deployment on multicore platforms with logical execution time," in *Proc. 40th IEEE Real-Time Systems Symposium (RTSS)*, 2019, pp. 207–219.

[23] F. Paladino, A. Biondi, E. Bini, and P. Pazzaglia, "Optimizing per-core priorities to minimize end-to-end latencies," in *Proc. 36th Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 298, 2024, pp. 6:1–6:25.

[24] J. A. R. De Azua and M. Boyer, "Complete modelling of AVB in network calculus framework," in *Proc. 22nd International Conference on Real-Time Networks and Systems (RTNS)*, 2014.

[25] E. Mohammadpour, E. Stai, M. Mohiuddin, and J.-Y. Le Boudec, "Latency and backlog bounds in time-sensitive networking with credit based shapers and asynchronous traffic shaping," in *Proc. 30th International Teletraffic Congress (ITC)*, 2018, pp. 1–6.

[26] L. Zhao, P. Pop, Z. Zheng, H. Daigmorte, and M. Boyer, "Latency analysis of multiple classes of AVB traffic in TSN with standard credit behavior using network calculus," *IEEE Transactions on Industrial Electronics*, vol. 68, no. 10, pp. 10 291–10 302, 2021.

[27] L. Zhao, P. Pop, and S. S. Craciunas, "Worst-case latency analysis for IEEE 802.1Qbv time sensitive networks using network calculus," *IEEE Access*, vol. 6, pp. 41 803–41 815, 2018.

[28] J.-Y. Le Boudec, "A theory of traffic regulators for deterministic networks with application to interleaved regulators," *IEEE/ACM Transactions on Networking*, vol. 26, no. 6, pp. 2721–2733, 2018.

[29] R. I. Davis, S. Altmeyer, L. S. Indrusiak, C. Maiza, V. Nelis, and J. Reineke, "An extensible framework for multicore response time analysis," *Real-Time Systems*, vol. 54, no. 3, pp. 607–661, 2018.

[30] R. Henia, A. Hamann, M. Jersak, R. Racu, K. Richter, and R. Ernst, "System level performance analysis – the SymTA/S approach," *IEE Proceedings – Computers and Digital Techniques*, vol. 152, no. 2, pp. 148–166, 2005.

[31] C. Park, C. Cui, and S. Park, "Analysis of E2E delay and wiring harness in in-vehicle network with zonal architecture," *Sensors*, vol. 24, no. 10, p. 3248, 2024.

[32] K. Aghababaiyan, B. Coll-Perales, L. Lusvarghi, and J. Gozalvez, "Deterministic task scheduling in in-vehicle networks for software-defined vehicles," in *IEEE/CIC International Conference on Communications in China (ICCC)*, 2025.

[33] K. Tindell and J. Clark, "Holistic schedulability analysis for distributed hard real-time systems," *Microprocessing and Microprogramming*, vol. 40, no. 2–3, pp. 117–134, 1994.

[34] M. Joseph and P. Pandya, "Finding response times in a real-time system," *The Computer Journal*, vol. 29, no. 5, pp. 390–395, 1986.

[35] N. C. Audsley, A. Burns, M. Richardson, K. Tindell, and A. J. Wellings, "Applying new scheduling theory to static priority preemptive scheduling," *Software Engineering Journal*, vol. 8, no. 5, pp. 284–292, 1993.

[36] N. C. Audsley, "On priority assignment in fixed priority scheduling,"

Information Processing Letters, vol. 79, no. 1, pp. 39–44, 2001.

[37] J. P. Lehoczky, L. Sha, and Y. Ding, "The rate monotonic scheduling algorithm: Exact characterization and average case behavior," in *Proc. 10th IEEE Real-Time Systems Symposium (RTSS)*, 1989, pp. 166–171.

[38] D. S. Johnson, "Fast algorithms for bin packing," *Journal of Computer and System Sciences*, vol. 8, no. 3, pp. 272–314, 1974.

[39] T. A. Henzinger, B. Horowitz, and C. M. Kirsch, "Giotto: a time-triggered language for embedded programming," *Proceedings of the IEEE*, vol. 91, no. 1, pp. 84–99, 2003.

[40] K.-B. Gemlau, L. Köhler, R. Ernst, and S. Quinton, "System-level logical execution time: Augmenting the logical execution time paradigm for distributed real-time automotive software," *ACM Transactions on Cyber-Physical Systems*, vol. 5, no. 2, pp. 1–27, 2021.

[41] IEEE, "IEEE standard for local and metropolitan area networks—bridges and bridged networks—amendment 34: Asynchronous traffic shaping," IEEE Std 802.1Qcr-2020, 2020.

[42] J. Specht and S. Samii, "Urgency-based scheduler for time-sensitive switched Ethernet networks," in *28th Euromicro Conference on Real-Time Systems (ECRTS)*, 2016.

[43] IEEE, "IEEE standard for local and metropolitan area networks—frame replication and elimination for reliability," IEEE Std 802.1CB-2017, 2017.

[44] D. Thiele and R. Ernst, "Formal worst-case performance analysis of time-sensitive Ethernet with frame preemption," in *Proc. 21st IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2016, pp. 1–9.

[45] R. L. Cruz, "A calculus for network delay, part I: Network elements in isolation," *IEEE Transactions on Information Theory*, vol. 37, no. 1, pp. 114–131, 1991.

[46] J.-Y. Le Boudec and P. Thiran, *Network Calculus: A Theory of Deterministic Queuing Systems for the Internet*, ser. Lecture Notes in Computer Science. Springer, 2001, vol. 2050.

[47] D. Zaheri and H.-C. Reuss, "ASIL-decomposition based resource allocation optimization for automotive E/E architectures," arXiv:2505.07881, 2025.

[48] ISO, "Road vehicles — functional safety," ISO 26262:2018, 2018.

[49] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W. H. Freeman, 1979.

[50] K. Lakshmanan, R. Rajkumar, and J. P. Lehoczky, "Partitioned fixed-priority preemptive scheduling for multi-core processors," in *Proc. 21st Euromicro Conference on Real-Time Systems (ECRTS)*, 2009, pp. 239–248.

[51] G. Dósa, "The tight bound of first fit decreasing bin-packing algorithm is $FFD(I) \leq 11/9\,OPT(I)+6/9$," in *Combinatorics, Algorithms, Probabilistic and Experimental Methodologies (ESCAPE)*, ser. LNCS, vol. 4614, 2007, pp. 1–11.

[52] S. Baruah, "Task partitioning upon heterogeneous multiprocessor platforms," in *Proc. 10th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2004, pp. 536–543.

[53] W. Zheng, Q. Zhu, M. Di Natale, and A. Sangiovanni-Vincentelli, "Definition of task allocation and priority assignment in hard real-time distributed systems," in *28th IEEE International Real-Time Systems Symposium (RTSS)*, 2007, pp. 161–170.

[54] Q. Zhu, Y. Yang, E. Scholte, M. Di Natale, and A. Sangiovanni-Vincentelli, "Optimizing extensibility in hard real-time distributed systems," in *15th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2009.

[55] H. Cambazard, P.-E. Hladik, A.-M. Déplanche, N. Jussien, and Y. Trinquet, "Decomposition and learning for a hard real time task allocation problem," in *Principles and Practice of Constraint Programming (CP)*, ser. LNCS, vol. 3258, 2004, pp. 153–167.

[56] A. Davare, Q. Zhu, M. Di Natale, C. Pinello, S. Kanajan, and A. Sangiovanni-Vincentelli, "Period optimization for hard real-time distributed automotive systems," in *Proc. 44th Design Automation Conference (DAC)*, 2007, pp. 278–283.

[57] A. Bhat, S. Samii, and R. Rajkumar, "Practical task allocation for software fault-tolerance and its implementation in embedded automotive systems," in *Proc. 23rd IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2017, pp. 87–98.

[58] R. I. Davis and A. Burns, "Robust priority assignment for fixed priority real-time systems," in *Proc. 28th IEEE Real-Time Systems Symposium (RTSS)*, 2007, pp. 3–14.

[59] E. Bini, M. Di Natale, and G. Buttazzo, "Sensitivity analysis for fixed-priority real-time systems," *Real-Time Systems*, vol. 39, no. 1–3, pp. 5–30, 2008.

[60] M. Günzel, K.-H. Chen, N. Ueter, G. von der Brüggen, M. Dürr, and J.-J. Chen, "Compositional timing analysis of asynchronized distributed cause-effect chains," *ACM Transactions on Embedded Computing Systems*, vol. 22, no. 4, pp. 1–34, 2023.

[61] J. Martinez, I. Sañudo, and M. Bertogna, "End-to-end latency characterization of task communication models for automotive systems," *Real-Time Systems*, vol. 56, no. 3, pp. 315–347, 2020.

[62] R. Queck, "Analysis of Ethernet AVB for automotive networks using network calculus," in *Proc. IEEE International Conference on Vehicular Electronics and Safety (ICVES)*, 2012, pp. 61–67.

[63] S. S. Craciunas, R. S. Oliver, M. Chmelík, and W. Steiner, "Scheduling real-time communication in IEEE 802.1Qbv time sensitive networks," in *Proc. 24th International Conference on Real-Time Networks and Systems (RTNS)*, 2016, pp. 183–192.

[64] L. Thomas, A. Mifdaoui, and J.-Y. Le Boudec, "Worst-case delay bounds in time-sensitive networks with packet replication and elimination," *IEEE/ACM Transactions on Networking*, 2022.

[65] M. Paolieri, E. Quiñones, F. J. Cazorla, G. Bernat, and M. Valero, "Hardware support for WCET analysis of hard real-time multicore systems," in *36th Annual International Symposium on Computer Architecture (ISCA)*, 2009, pp. 57–68.

[66] Y. Zhu, Y. Gao, B. Zhang, X. Gong, B. Jiang, L. Gong, W. Lou, T. Wang, C. Wang, X. Li, and X. Zhou, "Reducing end-to-end latency of cause-effect chains with shared cache analysis," arXiv:2601.20427, 2026.

[67] S. S. Craciunas and R. S. Oliver, "Combined task- and network-level scheduling for distributed time-triggered systems," *Real-Time Systems*, vol. 52, no. 2, pp. 161–200, 2016.

[68] P. Pop, M. L. Raagaard, S. S. Craciunas, and W. Steiner, "Design optimisation of cyber-physical distributed systems using IEEE time-sensitive networks," *IET Cyber-Physical Systems: Theory & Applications*, vol. 1, no. 1, pp. 86–94, 2016.

[69] A. Arestova, W. Baron, K.-S. J. Hielscher, and R. German, "ITANS: Incremental task and network scheduling for time-sensitive networks," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 369–387, 2022.

[70] F. Eisenbrand and T. Rothvoß, "Static-priority real-time scheduling: Response time computation is NP-hard," in *29th IEEE Real-Time Systems Symposium (RTSS)*, 2008, pp. 397–406.

[71] J. P. Lehoczky, "Fixed priority scheduling of periodic task sets with arbitrary deadlines," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 1990, pp. 201–213.

[72] L. Thomas, J.-Y. Le Boudec, and A. Mifdaoui, "On cyclic dependencies and regulators in time-sensitive networks," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 2019.

[73] L. Perron and F. Didier, "CP-SAT," https://developers.google.com/optimization/cp/, Google, google OR-Tools, version 9.15.

[74] E. Bini and G. C. Buttazzo, "Measuring the performance of schedulability tests," *Real-Time Systems*, vol. 30, no. 1–2, pp. 129–154, 2005.

[75] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[76] C. Reeves, "Hybrid genetic algorithms for bin-packing and related problems," *Annals of Operations Research*, vol. 63, no. 1, pp. 371–396, 1996.